Universidad de Oviedo

Escuela de Ingeniería Informática
Universidad de Oviedo

UNIVERSIDAD DE OVIEDO
ESCUELA DE INGENIERÍA INFORMÁTICA
GRADO EN INGENIERÍA INFORMÁTICA DEL SOFTWARE

TRABAJO DE FIN DE GRADO

IR-Board

Requirements Management Platform

Author:
Javier Carrasco Arango
Tutors:
Jorge Álvarez Fidalgo
Benjamín López Pérez

2026-06-26

Declaration of originality

I, Javier Carrasco Arango, with DNI 71905728T and UO294532, hereby declare that this work is completely original and all sources used during the development of it have been correctly cited. In Avilés, Asturias, on the xx of xx of 2026,

Signed:

Abstract

IR-Board is a Requirements Management Platform (RMP) designed to support the complete lifecycle of software requirements engineering. The project addresses the need for a unified environment that combines the rigor of traditional requirements specification standards with the flexibility of modern Agile methodologies. The platform is developed in accordance with the IEEE 830 and ISO/IEC/IEEE 29148 standards, providing structured support for the elicitation, documentation, validation, and maintenance of software requirements.

The system incorporates a hybrid documentation approach, enabling the management of traditional requirements engineering artifacts such as use cases, flowcharts, and decision tables, alongside Agile practices including user stories and story mapping. To ensure secure and granular access control, IR-Board implements a Relation-Based Access Control (ReBAC) model within a Zero-Trust architecture, allowing permissions to be determined dynamically according to user relationships with projects and functionalities.

In addition, the platform provides quality of life automations through controlled state transitions and approval workflows, improving traceability and governance. Collaborative features, including stakeholder management and concurrency control mechanisms, facilitate coordinated work among multiple users while preventing data conflicts. Finally, an integrated observability framework provides basic monitoring, operational metrics, and audit logging to support system reliability and administration. Through the integration of these capabilities, IR-Board offers a comprehensive solution for modern requirements engineering and project governance.

Keywords

Requirements Management Platform, Software Requirements Engineering, IEEE 830, ISO/IEC/IEEE 29148, Agile Methodology, Hybrid Documentation, User Stories, Use Cases, Relation-Based Access Control (ReBAC), Zero-Trust Architecture, Requirements Lifecycle Management, Traceability, Collaborative Engineering, Concurrency Control, Observability, Audit Logging, Project Governance, Workflow Automation.

About the document

This document follows a template provided by Jorge Álvarez Fidalgo.

It has been adapted and modified to fit the specific needs of the project during its development.

Special thanks to

Index

Figure Index

Table Index

Chapter 1. Introduction

1.1 Object

The primary object of this project is the design and implementation of **IR-Board**, a comprehensive Requirements Management Platform (RMP) designed to support the full lifecycle of software requirements engineering. The system aims to bridge the gap between traditional documentation standards and modern agile methodologies, providing a centralized environment for eliciting, refining, and managing requirements.

Key objectives of the system include:

- **Methodological Compliance:** Implementing a framework that follows international standards for requirements specification, specifically the **IEEE 830** guidelines and the **ISO/IEC/IEEE 29148** standard.
- **Hybrid Documentation Support:** Providing tools for both traditional modeling (use cases, flowcharts, and decision tables) and Agile practices (user story mapping and management).
- **Relation-Based Access Control (ReBAC):** Developing a sophisticated security model where permissions are not merely role-based but determined by the dynamic relationship between users and specific entities (Projects and Functionalities), implemented through a **Zero-Trust** architecture.
- **Lifecycle and State Management:** Automating the management of requirement states (Pending Approval, Approved, Deactivated, etc.) and project lifecycles to ensure data integrity and traceability.
- **Collaborative Engineering:** Facilitating stakeholder management and real-time concurrency control to prevent data conflicts during collaborative editing sessions.
- **System Observability:** Integrating a high-standard monitoring stack to provide administrators with full visibility into the infrastructure's health and security audit logs.

1.2 Background

Software requirements engineering is a fundamental phase in the software development lifecycle, as it defines what a system must achieve before design and implementation begin. It also helps bridge the gap between client needs and the technical rigor required to reduce misunderstandings, contractual ambiguity, and potential legal issues. To ensure quality and consistency, standardized approaches such as IEEE 830 and ISO/IEC/IEEE 29148 have been widely adopted. These standards emphasize structured documentation, traceability, validation, and completeness of requirements, making them particularly suitable for large-scale and regulated software systems.

In contrast, modern software development has increasingly shifted toward Agile methodologies, which prioritize iterative development, rapid feedback, and lightweight documentation. Techniques such as user stories and story mapping allow teams to respond quickly to changing requirements and stakeholder needs. How-

ever, Agile approaches often reduce formal structure, which can make long-term traceability and governance more challenging.

As a result, many organizations now operate in hybrid environments where both traditional and Agile requirements engineering practices are used simultaneously.

1.3 Current state

Despite the coexistence of traditional and Agile methodologies, existing requirements management tools tend to favor one approach over the other. Traditional tools focus heavily on structured documentation and compliance with formal standards, while Agile-oriented platforms emphasize backlog management and iterative workflows. This separation often leads to fragmented processes, requiring teams to use multiple disconnected tools.

This fragmentation can result in reduced traceability, inconsistencies between requirement representations, and difficulties in maintaining lifecycle integrity across different development methodologies. Additionally, many existing systems provide limited support for advanced access control models, relying mainly on role-based access control (RBAC), which lacks flexibility in dynamic, collaborative environments.

At the same time, there is a broad ecosystem of commercial requirements management and product lifecycle tools that aim to address these challenges by offering more integrated workflows. Platforms such as IBM Engineering Requirements Management DOORS Next, Jama Connect, Polarion ALM, and modern lifecycle tools like Atlassian Jira (with extensions) or Azure DevOps provide end-to-end support for requirements tracking, traceability, collaboration, and integration with development pipelines. These systems are often highly mature and feature-rich, supporting compliance workflows, test management, and cross-team collaboration within large enterprises.

However, despite their extensive capabilities, these solutions are typically proprietary, complex to configure, and associated with significant licensing costs. They are often optimized for enterprise-scale environments, which makes them less accessible for small teams, academic projects, or organizations seeking flexible customization. Furthermore, while they provide strong workflow integration, their support for modern architectural security models such as Relation-Based Access Control (ReBAC) and Zero-Trust principles remains limited or heavily abstracted.

At the same time, there is an increasing demand for more sophisticated system capabilities such as real-time collaboration, automated workflow and state management, and integrated observability for monitoring system health and security. Emerging security models such as ReBAC and Zero-Trust architectures are not yet widely adopted in most requirements management platforms.

Within this landscape, open-source alternatives remain relatively limited in scope and maturity. Existing open-source tools tend to focus on either issue tracking or lightweight Agile backlog management rather than providing a full requirements engineering lifecycle aligned with formal standards such as IEEE 830 and ISO/IEC/IEEE 29148. As a result, there is a clear gap in the market for an open, exten-

sible, and standards-compliant platform that combines structured requirements engineering, Agile methodologies, advanced access control mechanisms, and collaborative lifecycle management within a single coherent system.

1.4 Definitions and Abbreviations

1.4.1 General concepts

- **RMP (Requirements Management Platform)**: A software system designed to support the full lifecycle of software requirements, including elicitation, specification, validation, traceability, and change management.
- **Requirements Engineering (RE)**: A discipline of software engineering focused on defining, documenting, and maintaining software requirements throughout the system lifecycle.
- **IEEE 830**: A legacy IEEE standard for software requirements specification, defining structure and content of Software Requirements Specifications (SRS).
- **ISO/IEC/IEEE 29148**: The current international standard for requirements engineering processes and requirements specification quality.
- **Agile Methodology**: An iterative software development approach that emphasizes incremental delivery, collaboration, and adaptability to change.
- **User Story**: A short description of a feature from an end-user perspective, commonly used in Agile development.
- **Use Case**: A structured description of a system's behavior in response to external actors.

1.4.2 Security and access control

- **ReBAC (Relation-Based Access Control)**: An authorization model where access permissions are determined by relationships between entities (e.g., user–project, user–role–resource), rather than static roles.
- **RBAC (Role-Based Access Control)**: A traditional access control model where permissions are assigned to roles, and users inherit permissions through assigned roles.
- **Zero-Trust Architecture**: A security model that assumes no implicit trust in any user or system component, requiring continuous verification of identity and permissions.
- **OIDC (OpenID Connect) (implied via Ory ecosystem context)**: An authentication layer built on OAuth 2.0 used for identity verification.

1.4.3 Protocols and data modelling concepts

- **SMTP (Simple Mail Transfer Protocol)**: The standard protocol used for sending emails between servers.
- **Internal Unique Identifier (ID)**: A system-generated identifier used internally to uniquely identify an entity in the database.
- **Dynamic Identifier**: A human-readable structured identifier generated from relationships and context (e.g., FR-UM-001).
- **Slug**: A URL-safe unique string identifier, typically used for routing or external references.
- **Floating Point Ordering**: A technique for ordering entities (e.g., requirements) using floating-point values to allow efficient reordering without full renumbering.

- **Entity**: A domain object within the system such as a project, requirement, stakeholder, or document.
- **Audit Log**: A record of system actions used for traceability and accountability.

1.4.4 Abbreviations

- API – Application Programming Interface
- DNS – Domain Name System
- UI – User Interface
- UX – User Experience
- SRS – Software Requirements - Specification
- MTA – Mail Transfer Agent
- TLS – Transport Layer Security
- JWT – JSON Web Token
- CLI – Command Line Interface
- CI/CD – Continuous Integration / Continuous Deployment

1.5 Scope

The scope of this project is the design and implementation of IR-Board, a Requirements Management Platform focused on supporting the complete lifecycle of software requirements engineering. The system aims to provide a centralized environment where software projects can define, organize, validate, and maintain requirements while preserving traceability between the different elements involved in the requirements process.

The project covers the analysis, design, development, and validation of the core platform functionalities required to manage software requirements in a structured manner.

The implemented solution combines traditional requirements engineering practices with selected Agile-oriented concepts, allowing users to manage requirements, stakeholders, documents, and related project information from a unified interface.Furthermore, it includes support for the complete management lifecycle of project entities. This includes the creation, modification, validation, deactivation, and archival of projects, functionalities, requirements, stakeholders, and associated documentation. The platform provides mechanisms to maintain the relationships between these entities, enabling bidirectional traceability and ensuring that changes affecting one element can be identified across related elements.

About document management, the system allows users to upload, associate, update, and manage documents linked to project entities. These documents contribute to requirements traceability by allowing requirements and other elements to reference supporting material throughout the development lifecycle.

Another objective within the scope of the project is the implementation of a secure architecture following Zero-Trust principles. The system incorporates identity management, authentication, and authorization mechanisms designed to prevent implicit trust between components. Fine-grained access control is implemented through a Relation-Based Access Control (ReBAC) approach, allowing permissions to be determined dynamically according to relationships between users, projects,

functionalities, and other system entities. More on the selection of a ReBAC access control on the section alternatives analysis. About the user management, the system covers user invitation, authentication workflows, credential management, and permission assignment. Users can participate in projects according to their assigned relationships and responsibilities, enabling collaborative work while maintaining controlled access to each project.

The project also includes the implementation of basic system observability capabilities. The platform provides mechanisms for collecting operational information such as application logs and relevant system metrics. These capabilities are intended to assist development, debugging, maintenance, and basic operational monitoring of the deployed solution.

The platform is designed with extensibility and future integration in mind. Although some external services are not implemented as part of this project, the architecture has been prepared to allow their incorporation without requiring significant changes to the core system. This includes the possibility of replacing development components with production-ready alternatives when deploying the system in a real environment.

The following elements are considered outside the scope of this project:

- Full production-grade security hardening of the observability infrastructure is not included. Although logging and metric collection mechanisms are implemented, securing, isolating, auditing, and managing the observability ecosystem itself is considered an infrastructure operation outside the objectives of this project.
- Integration of the observability stack as part of the business application is not included. The platform exposes the necessary information for monitoring, but advanced dashboards, alerting systems, automated incident response, and operational intelligence, included in comprehensive and unified panels within the frontend are outside the project scope.
- Integration with external third-party platforms is not included. This includes external project management tools, development platforms, issue trackers, or other enterprise systems.
- Production email infrastructure is not included. During development, email-related functionality is supported through development-oriented components intended for testing purposes, that is, a local development test server called mailpit. Configuration and deployment with real SMTP providers, domain verification, email reputation management, SPF, DKIM, DMARC, and production mail delivery processes, all necessary to allow a mail to even reach the spam folder when self hosting, are outside the scope of this project.
- Deployment automation for large-scale production environments is not included. While the system is containerized and designed for reproducible deployment, advanced orchestration, scaling strategies, and cloud-specific infrastructure management are considered future work.

The final result of this project is therefore a functional and extensible prototype of a requirements management platform that demonstrates secure architecture

design, requirements lifecycle management, traceability, collaboration capabilities, and basic operational visibility, while leaving production infrastructure concerns and external ecosystem integrations as future extensions.

1.6 Assumptions and Constraints

The development of IR-Board is based on several assumptions and constraints regarding the environment, users, and expected usage of the platform.

It is assumed that the system will primarily be used by software development teams or academic/project environments where requirements need to be collected, structured, reviewed, and maintained throughout a software lifecycle. The platform is not intended to replace complete enterprise Application Lifecycle Management (ALM) suites, but rather provide a focused requirements engineering environment combining formal documentation practices with Agile techniques.

The project is constrained by the available development time and the scope of a bachelor's thesis. Therefore, some enterprise-level functionalities commonly found in commercial requirements management platforms are not implemented.

It is assumed that users interacting with the system have basic knowledge of software development concepts and requirements engineering terminology. Although the platform provides mechanisms to guide the creation and validation of requirements, it does not attempt to automatically determine whether a requirement is correct from a business perspective; this responsibility remains with stakeholders and project members, and therefore those actions are fundamentally manual by design.

The system assumes that requirements are created and maintained following a structured lifecycle. Requirements may evolve over time, and changes are expected to occur through controlled modification, validation, and approval processes rather than direct deletion. Therefore, the platform assumes that maintaining historical information and traceability is more valuable than permanently removing data in a quick manner.

The project assumes that the group using the system may have different projects at a time, operated by different groups of developers, and therefore static global roles are unfit for access control.

The project assumes that external security services such as identity management, authentication, and authorization providers can be integrated through standardized interfaces. Therefore, solutions such as the Ory ecosystem are treated as replaceable infrastructure components rather than business logic dependencies.

It is assumed that PostgreSQL provides enough flexibility for the domain model, including structured relational data and semi-structured information through JSONB fields. The system does not require a dedicated NoSQL database because the main entities require strong consistency and traceability.

The document management subsystem assumes that uploaded documents act mainly as requirement-related artifacts. Advanced document processing, version

control systems, collaborative document editing, or automatic content extraction are outside the expected functionality.

The observability layer assumes that basic logging, metrics, and monitoring capabilities are sufficient for system administration and debugging. The project does not assume the need for security monitoring, automated incident response, or a complete observability platform.

The deployment environment assumes containerized execution. Components are expected to run through container orchestration tools such as Docker Compose during development and testing, allowing the same architecture to be adapted to more advanced deployment environments if required.

Chapter 2. Theoretical Background

To better understand the system developed, here is a brief summary of the concepts related and used on it.

2.1 Software requirements engineering

Requirements engineering is the structured and systematic approach to formalizing the needs and expectations of stakeholders for a system. It encompasses different processes for identifying, eliciting, analyzing, specificating, validating and their management.

2.1.1 Concepts

But first, it is necessary to explain what a **stakeholder** is. A stakeholder is someone with a "stake" or share on a project, or in general, someone interested in it. Stakeholders are the ones who, directly or indirectly define what a system must do for it to succeed. Common examples of stakeholders are end users, investors, industry competitors, and the development team. Understanding the stakeholders related to a project and their needs is imperative for a project to be accepted and provide value.

A **requirement** represents a formal, documented statement describing a capability, condition, or constraint that a system must satisfy. Requirements act as the communication bridge between stakeholders and the development team, allowing the expected behavior of a system to be understood, evaluated, and implemented.

2.1.2 Criteria

Generally, requirements are classified according to different criteria, including their abstraction level and whether they are functional or non-functional. Regarding abstraction, requirements are commonly organized in a hierarchical structure, where high-level requirements are progressively refined into more specific ones.This hierarchy allows requirements to be represented using identifiers, commonly composed of a unique identifier followed by a structured notation for their refinements. Sub-requirements can be nested within parent requirements, often using dot-separated numbering schemes that represent their position within the hierarchy.

At the highest level, **business requirements** describe the objectives and needs that motivate the development of a system. They are closer to the concerns of stakeholders and are therefore usually expressed in a more abstract way, making them easier for non-technical stakeholders to understand and validate. As requirements are refined into lower levels of abstraction, they become increasingly detailed and closer to the technical aspects of the system. These **technical requirements** describe specific behaviors, constraints, or implementation-related aspects necessary to satisfy the higher-level objectives.

This separation between abstraction levels facilitates communication between stakeholders and development teams, while also improving requirements traceability by allowing each detailed requirement to be linked back to the original business need that originated it.

Regarding the other criteria, **functional requirements** describe the behaviors and services that the system must provide, defining what the system should do. Examples include allowing a user to create an account, generating reports, or managing project information. **Non-functional requirements**, on the other hand, define quality attributes and constraints that affect how the system performs. These include aspects such as security, performance, reliability, usability, and maintainability.

2.1.3 Lifecycle

The requirements engineering process usually follows several interconnected activities:

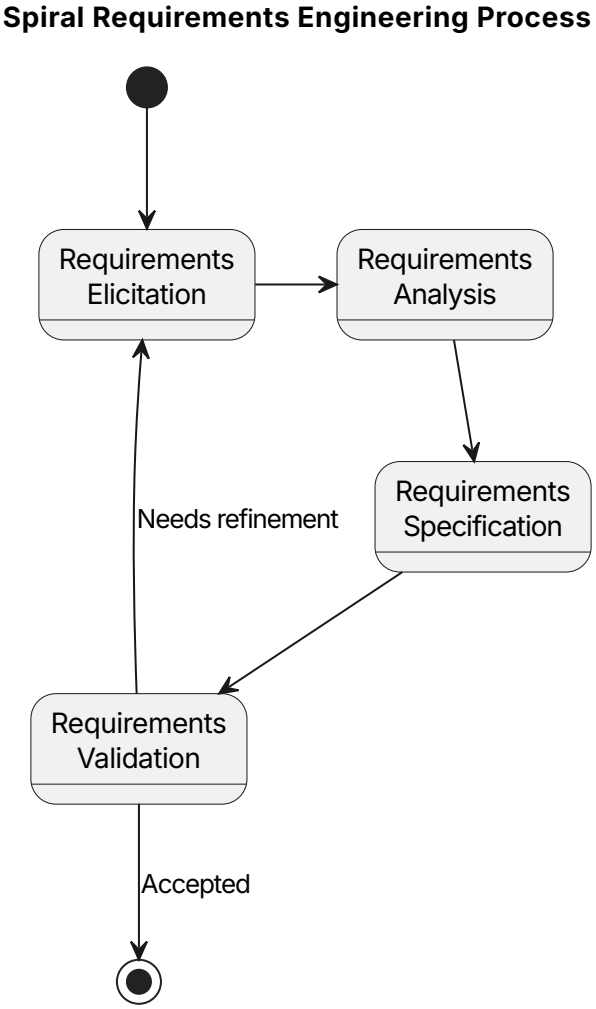

Figure 1: Requirement engineering processes

The first phase is **requirements elicitation**, where information is gathered from stakeholders through techniques such as interviews, observation, workshops, or analysis of existing documentation. The objective of this phase is to understand the problem domain, its scope, and identify the actual needs behind the requested system. Due to the inherently human-centered nature of requirements elicitation, many of these techniques are influenced by methods from fields such as social sciences, qualitative research, and human-computer interaction. This is because requirements are not always explicitly known or easily communicated by stake-

holders; they often need to be discovered through structured interaction, analysis, and interpretation.

After elicitation, requirements are analyzed and refined. During **requirements analysis**, ambiguous, conflicting, or incomplete requirements are identified and resolved. This stage also involves prioritizing requirements according to factors such as business value, feasibility, cost, and risk.

Once analyzed, requirements are documented during the **requirements specification** phase. The purpose of specification is to create a clear and structured representation of the requirements, reducing misunderstandings between stakeholders and developers. A well-written requirement should be understandable, consistent, testable, and traceable throughout the development process.

Requirements must also be validated before being considered complete. **Requirements validation** ensures that documented requirements accurately represent stakeholder needs and that they are realistic within the technical and organizational constraints of the project. Validation activities may include reviews, prototypes, and acceptance criteria definition. Logically, part of the requirements may be validated while others may need refinement. For this reason, the process is modelled as a spiral, iterating over the different phases until accepted.

Finally, requirements engineering includes **requirements management**, which handles changes to requirements during the lifecycle of a project. Software systems frequently evolve due to changes in business needs, regulations, technology, or user expectations. Managing these changes ensures that modifications are controlled and that the relationship between requirements, design decisions, implementation, and testing remains clear.

2.1.4 Traceability

A key concept within requirements management is **traceability**. Requirements traceability refers to the ability to follow the relationship between a requirement and other elements associated with it, such as stakeholders, design components, implementation artifacts, tests, or related requirements. Traceability improves impact analysis, change management, and verification by providing visibility into how modifications affect the overall system.

Forward traceability ensures that every requirement can be followed from its original stakeholder need towards its implementation and verification artifacts. This helps confirm that all requested functionality has been addressed.

Backward traceability performs the opposite analysis, allowing developers and analysts to determine why a specific implementation element exists and which requirement originated it. This is particularly useful for preventing unnecessary functionality and supporting maintenance activities.

Horizontal traceability refers to relationships between artifacts at the same abstraction level or between parallel elements of the system. Unlike hierarchical decomposition, horizontal traceability focuses on dependencies, interactions, and consistency between related elements. Examples include linking requirements that depend on each other, connecting a requirement with its related stakeholders, as-

sociating documents or models with the requirements they describe, or identifying conflicts and overlaps between different requirements. This form of traceability is especially important in complex systems, where a modification to one element may affect several other elements that are not directly above or below it in the hierarchy.

One common approach to implementing these concepts is **identifier-based** traceability, where each requirement is assigned a unique identifier that remains stable throughout its lifecycle. These identifiers provide a direct reference mechanism between requirements, design elements, source code components, test cases, documentation, and other related artifacts. In this project, unique and semantically meaningful identifiers are used. For functional requirements, the identifier is composed of the initials of the requirement category or containing functionality, followed by a dot-separated numbering scheme that represents its position within the requirement hierarchy. This structure allows requirements to be located and understood without inspecting their complete content.

Another commonly used strategy is **traceability matrices**, where relationships between different artifacts are represented in a structured table. For example, a requirements-to-test matrix can demonstrate that every requirement has associated validation activities, while a requirements-to-design matrix can show how each requirement is translated into system components.

2.2 Requirements Documentation Standards

Requirements documentation standards provide guidelines and recommendations for creating consistent, complete, and understandable descriptions of software requirements. Their main objective is to reduce ambiguity between stakeholders and development teams by defining common structures, terminology, and practices for representing system needs.

2.2.1 IEEE 830

The IEEE 830 standard, formally known as **IEEE Recommended Practice for Software Requirements Specifications**, defines recommendations for writing Software Requirements Specifications (SRS).

The standard establishes characteristics that a good requirement specification should have. A requirements document should be:

- **Correct:** The documented requirements should represent the actual needs of the stakeholders and system objectives.
- **Unambiguous:** Each requirement should have only one possible interpretation, avoiding unclear or subjective language.
- **Complete:** All relevant requirements, constraints, and system behaviors should be included.
- **Consistent:** Requirements should not contradict each other or define incompatible system behavior.
- **Verifiable:** Each requirement should be testable through objective validation methods.
- **Modifiable:** The document structure should allow changes without introducing inconsistencies.

- **Traceable:** Each requirement should be uniquely identifiable and linked to its origin and related artifacts.

Although modern Agile methodologies often favor lighter documentation approaches, the principles introduced by IEEE 830 remain applicable, especially regarding clarity, traceability, and verification.

2.2.2 ISO/IEC/IEEE 29148

The ISO/IEC/IEEE 29148 standard, **Systems and software engineering - Life cycle processes - Requirements engineering**, provides a more modern and comprehensive framework for requirements engineering. While IEEE 830 focuses mainly on the structure of the requirements specification document, ISO/IEC/IEEE 29148 covers the complete requirements lifecycle.

This standard defines processes for requirements definition, analysis, management, validation, and maintenance throughout the entire system lifecycle. It emphasizes that requirements should not be considered static documents, but rather managed entities that evolve together with the system.

ISO/IEC/IEEE 29148 introduces concepts such as:

- **Stakeholder requirements and system requirements:** The criteria about requirement abstraction based on levels seen on the criteria section. Stakeholder requirements are the equivalent of lower level, project-specific business requirements, as in some contexts, a business requirement may be broader, to affect the whole business.
- **Requirements management:** The continuous process of controlling requirement changes, relationships, versions, and status throughout the lifecycle.
- **Requirements verification and validation:** Activities that ensure requirements are correctly defined (verification) and that they represent the intended system (validation).

The standard also reinforces the importance of requirements attributes beyond their textual description. These attributes may include identifiers, priority, source, rationale, status, dependencies, and verification criteria. These elements support traceability and enable better impact analysis when requirements change.

Compared to IEEE 830, ISO/IEC/IEEE 29148 provides a broader lifecycle-oriented perspective, making it more suitable for modern development environments where requirements continuously evolve. For this reason, IR-Board follows the principles of both standards: using structured requirement documentation inspired by IEEE 830 while applying lifecycle management concepts defined by ISO/IEC/IEEE 29148.

2.3 Agile methodologies

Agile methodologies are software development approaches that prioritize adaptability, continuous feedback, and incremental delivery over rigid, sequential planning. They emerged as a response to traditional development models, where requirements were often defined completely at the beginning of the project and changes were expensive to introduce later.

Their approach is based on the idea that software requirements evolve as stakeholders gain a better understanding of their needs and as the project progresses. Instead of attempting to define the complete system beforehand, Agile teams work through short development iterations or sprints, frequently delivering functional increments of the system and incorporating feedback from users and stakeholders.

Common principles of Agile development include close collaboration between developers and stakeholders, continuous improvement, prioritization of valuable features, and responding to changes rather than strictly following an initial plan.

Although Agile methodologies generally favor lightweight documentation, they do not eliminate the need for requirements management. Instead, requirements are commonly represented through simpler and more flexible artifacts that can evolve during development, like user stories.

User stories are an Agile technique used to describe system requirements from the perspective of the user or stakeholder. Rather than focusing on technical implementation details, user stories express the expected value and purpose of a feature. They generally follow a structure like so:

As a [type of user], I want [goal], so that [reason or benefit].

2.4 Security principles

Software security principles are fundamental concepts used to design systems that protect data, users, and operations against unauthorized access, misuse, or failure. Security is not only implemented through specific mechanisms but also through architectural decisions and development practices.

A common principle is the principle of **least privilege**, which states that users and components should only have the permissions required to perform their intended tasks. This reduces the potential impact of compromised accounts or services.

Another important principle is defense in depth, where multiple independent security layers are applied instead of relying on a single protection mechanism. If one layer fails, additional controls can still prevent or limit damage.

Modern security approaches also emphasize concepts such as secure-by-design, where security considerations are incorporated from the beginning of the system lifecycle rather than added as a final step.

2.5 Software architectures

Software architecture describes the high-level structure of a software system, including its components, their responsibilities, and the communication mechanisms between them. Architectural decisions influence scalability, maintainability, security, and the ability of the system to evolve. It not only defines how software is organized internally, but also how different parts of the system interact with each other and with external services.

2.5.1 Monolithic Architecture

A monolithic architecture is a software architecture where the different system components are developed, deployed, and executed as a single application unit.

In a monolithic system, business logic, data access, authentication, and other application concerns usually exist within the same deployment artifact. This approach is simple to develop and deploy, especially for smaller systems, as communication between components occurs through internal method calls.

However, as the system grows, monolithic architectures can become harder to maintain because changes in one component may require redeploying the entire application. Scaling individual parts independently is also more difficult, since the complete application is scaled as a single unit.

2.5.2 Microservices Architecture

A microservices architecture divides an application into a collection of smaller, independent services. Each service is responsible for a specific business capability and communicates with other services through well-defined interfaces.

Microservices allow independent deployment, scaling, and development of different system components. They also provide stronger isolation, as failures in one service can potentially be contained without affecting the entire system.

However, this approach introduces additional complexity, including network communication, service discovery, distributed data management, monitoring requirements, and more complex deployment processes.

For this reason, microservices architectures are often combined with supporting infrastructure such as API gateways, centralized logging, authentication services, and monitoring systems.

2.6 Observability

Observability is the ability to understand the internal state and behavior of a software system by analyzing its external outputs. It allows operators and developers to detect failures, investigate problems, and understand system performance.

Modern distributed systems require observability because failures are often not limited to a single component. Instead, they may result from interactions between several services, infrastructure components, or external dependencies.

The three main pillars of observability are logs, metrics, and traces.

2.6.1 Logs

Logs are timestamped records of events produced by applications and infrastructure components. They provide detailed information about what happened at a specific point in time. Examples of logged events include application errors, authentication attempts, configuration changes, or important business operations.

Logs are especially useful for debugging and investigating incidents because they provide contextual information about individual events.

2.6.2 Metrics

Metrics are numerical measurements that represent the state or performance of a system over time. Unlike logs, which describe individual events, metrics provide aggregated information that can be analyzed through trends and comprehensive dashboards.

Common examples include CPU usage, memory consumption, request latency, error rates, or the number of active users.

2.6.3 Traces

Traces represent the path of a request as it travels through different components of a system. They are particularly important in distributed architectures where a single user action may involve multiple services.

A trace is composed of multiple spans, where each span represents an operation performed by a specific component. By analyzing traces, developers can identify bottlenecks, latency sources, or failures across service boundaries.

Together, logs, metrics, and traces provide a complete view of system behavior, supporting reliability, maintenance, and operational decision-making.

Chapter 3. Feasibility Study and Alternatives Analysis

Before starting the implementation of IR-Board, an analysis of the technical feasibility and the different alternatives available was performed. The objective of this analysis was to identify the most appropriate technologies and architectural decisions according to the characteristics of the system to be implemented, the expected complexity, the security requirements, and the available development resources.

The main factors considered during the evaluation were maintainability, scalability, security, development complexity, ecosystem maturity, integration capabilities, and suitability for the requirements management domain.

Given the collaborative nature of requirements engineering systems, the decision was made to develop IR-Board as a web-based platform rather than as a standalone desktop application. A web architecture provides easier deployment, multiplatform access, and a better foundation for collaborative workflows where several users may interact with the same project simultaneously.

The selected architecture separates the frontend and backend into independent projects. This separation improves maintainability and allows each layer to evolve independently. Additionally, it facilitates future deployment scenarios where the frontend could be served independently or replaced without requiring modifications to the core business logic.

3.1 Deferral or self-implementation of security environment

Security was identified as a critical aspect of the system due to the collaborative nature of requirements management and the need for controlled access between users, projects, and functionalities.

Two approaches were considered: implementing security mechanisms directly inside the application, or delegating security responsibilities to specialized external components.

Criteria	Custom implementation	External security ecosystem
Control	Maximum control	Controlled through configuration
Development effort	Very high	Lower
Security assurance	Depends on implementation quality	Based on mature solutions
Maintainability	Higher responsibility	Separated components

Table 1: Security implementation alternatives comparison

A custom implementation would provide maximum control over authentication, authorization, and session management. However, implementing a complete security environment would significantly increase the complexity of the project. Features such as secure password management, session handling, authorization policies,

token management, and protection against common vulnerabilities require extensive development and testing.

For this reason, the project decided to use specialized open-source identity and security components from the Ory ecosystem. Their specific licensing is documented over on the appendices.

Ory Kratos is responsible for identity management and user sessions. Ory Oathkeeper acts as an authorization gateway, validating requests before they reach internal services. Ory Keto provides relationship-based access control (ReBAC), which is particularly appropriate for the project because permissions depend on relationships between users, projects, and functionalities.

Similarly, Traefik was selected as the API gateway and entry point due to its maturity and previous experience with the technology.

The use of these components follows the security principle of defense in depth, separating responsibilities and reducing the amount of security-critical code maintained by the project.

3.2 Backend framework selection

The backend was approached as a system exposing a REST API from the beginning. Three main alternatives were considered: Spring Boot, Express.js, and FastAPI.

Framework	Advantages	Disadvantages
Spring Boot	Mature ecosystem, strong security support, dependency injection, Spring Data JPA integration, suitable for complex domain models	Higher initial complexity and more verbose configuration compared with lightweight alternatives
Express.js	Simple architecture, rapid development, large JavaScript ecosystem, direct compatibility with React and TypeScript	Minimal framework approach requires manual decisions for architecture, security, validation, and application structure
FastAPI	Modern API design, automatic OpenAPI generation, type-based validation, high performance, simple development model	Less opinionated architecture, fewer enterprise-oriented patterns, additional design decisions required for complex systems

Table 2: Backend framework alternatives comparison

3.2.1 Spring Boot

Spring Boot was selected as the backend framework due to its maturity, strong ecosystem, and extensive support for enterprise-oriented applications. Its opinionated structure provides several development and security advantages, including dependency management, validation support, integrated configuration management, security modules, and mature database access through Spring Data JPA.

Additionally, the IR-Board domain requires complex relationships between entities, lifecycle management, state transitions, and strict data consistency. The object-relational mapping capabilities provided by JPA allow these relationships to be represented naturally while maintaining control over the resulting database structure. The Spring ecosystem also provides mature integration possibilities with authentication systems, authorization layers, and observability tools.

3.2.2 Express.js

Express.js was considered due to its simplicity, lightweight nature, and strong integration with JavaScript-based frontend ecosystems. Since the frontend was planned using React and TypeScript, using a JavaScript-based backend would allow sharing the same programming language across the stack, potentially simplifying development and reducing context switching.

However, Express.js is intentionally minimal and provides fewer architectural constraints. This means that important aspects such as project structure, validation, security practices, dependency management, and error handling need to be manually defined. For a system where security, traceability, and long-term maintainability are important objectives, this additional responsibility increases implementation complexity and risk.

3.2.3 FastAPI

FastAPI was also considered as a modern alternative for REST API development. Its use of Python type hints and Pydantic models provides automatic validation, clear API contracts, and automatic OpenAPI documentation generation. Additionally, its asynchronous architecture can provide excellent performance for I/O-heavy applications while maintaining a relatively simple development experience.

Despite these advantages, FastAPI was not selected. Although it is a strong framework for API-focused applications, it provides fewer built-in architectural conventions for large domain-oriented systems compared with Spring Boot. More decisions regarding dependency injection patterns, application organization, security integration, and persistence architecture would need to be defined independently. While this flexibility is valuable in many contexts, the additional design decisions would increase the complexity of maintaining a consistent architecture.

The final decision was Spring Boot because the additional structure, mature security ecosystem, database integration capabilities, and established enterprise patterns better match the requirements of IR-Board. The framework reduces implementation risks and development efforts while providing a solid foundation for future expansion.

3.3 Frontend framework selection

The frontend is responsible for presenting the requirements management interface and enabling user interaction with complex entities such as requirements, documents, stakeholders, and project structures. Due to the collaborative and data-intensive nature of the system, the frontend needed to support reusable components, responsive layouts, maintainable state management, and a flexible design system.

The main frontend framework alternatives considered were React and Angular.

React with TypeScript was selected as the frontend framework due to its flexibility, ecosystem maturity, and previous experience with the technology. React provides a component-based development model, allowing the creation of reusable interface elements and facilitating the development of complex interactive views. The use of TypeScript adds static typing, reducing potential errors and improving maintainability in a project with a large number of entities and interactions.

Angular was considered because it provides a complete frontend framework including routing, dependency injection, and a predefined application structure. This approach improves consistency and can simplify development in large teams by reducing the number of architectural decisions required during implementation. However, Angular's broader framework scope, stricter conventions, and larger set of integrated concepts introduce additional complexity compared with lighter approaches. Since IR-Board requires flexibility in interface design and integration with external services, React offered a better balance between structure, adaptability, and control over the selected tooling.

Criteria	React + TypeScript	Angular
Architecture	Flexible component-based approach; additional tooling chosen according to project needs	Complete framework with predefined application structure
Flexibility	High; developers choose supporting libraries and patterns	Moderate; framework conventions guide implementation
Learning curve	Moderate; requires selecting additional tools	Higher initially due to wider framework scope
Ecosystem	Large ecosystem and extensive third-party integrations	Large ecosystem with official tooling
Decision	Selected	Rejected

Table 3: Frontend framework alternatives comparison

3.4 Database selection

The system requires persistent storage for projects, requirements, users, documents, relationships, and lifecycle information. The database solution needed to provide reliable persistence, transactional guarantees, support for complex relationships between entities, and enough flexibility to evolve as the system grows.

A relational database management system (RDBMS) was selected because the problem domain is primarily composed of structured entities with well-defined relationships. Requirements engineering involves maintaining consistency between projects, functionalities, requirements, stakeholders, and documents, where modifications to one entity may affect others. Therefore, transactional operations, referential integrity, and explicit relationship modelling are important characteristics for the system.

PostgreSQL was selected due to its maturity, reliability, and extensive feature set. It provides strong relational guarantees while also supporting semi-structured data

through features such as JSONB columns. This allows storing flexible attributes when required without abandoning the advantages of a relational model.

Alternative database solutions based on document-oriented databases, such as MongoDB, were considered due to their flexible schema and ability to represent changing data structures. However, this flexibility is less relevant for the IR-Board domain, where consistency, traceability, and explicit relationships between entities are critical. A document-oriented approach would require additional application-level logic to maintain relationships and enforce integrity constraints.

Another important consideration was the integration with the selected security architecture. The Ory ecosystem used by the project relies on relational database support, and PostgreSQL is a mature option compatible with these components, as commented here. Using the same database technology across the system reduces deployment complexity, simplifies maintenance, and avoids introducing unnecessary infrastructure differences.

The final decision was PostgreSQL because it provides the required balance between consistency, flexibility, ecosystem compatibility, and long-term maintainability.

Criteria	PostgreSQL	MongoDB	Evaluation
Data consistency	Strong ACID transactions and relational integrity	Flexible schema with weaker relationship enforcement	PostgreSQL preferred
Entity relationships	Native foreign keys and relational modelling	Relationships require additional application logic	PostgreSQL preferred
Traceability	Suitable for immutable identifiers, lifecycle states, and audit information	Possible but requires additional modelling	PostgreSQL preferred
Schema flexibility	JSONB allows semi-structured fields while maintaining relations	Highly flexible document model	Both viable
Ory ecosystem compatibility	Supported and commonly used	Not suitable for the selected deployment	PostgreSQL preferred

Table 4: Database alternatives comparison

Chapter 4. Initial Project Planning and Management

4.1 Theoretical client petition

To be able to appropiately create a project's budget aligned with the needs of the client, and ensure a complete enough context, the planning was done with the following fictional scenario in mind:

The theoretical client, NorthTech Solutions, is a company located in a nearby city to where the development team operates. The company requests the creation of a customized software platform to improve its internal processes and digital management.

The project involves collaboration between different parties, including NorthTech Solutions, the development team, the company's management department, technical staff, and the future users of the application. The requested work covers the complete software lifecycle, including requirements analysis, custom development, deployment on the internal network of the company, load testing, usability testing, and technical documentation.

The objective is to deliver a functional, maintainable, and scalable solution while applying professional software engineering practices within the scope of this TFG.

4.2 Stakeholder Identification

ID	Stakeholder	Description
ST-1	NorthTech Solutions	Fictional company requesting the software solution, defining business needs and validating the final product
ST-2	Company Management Department	Area responsible for organizational decisions, providing objectives, priorities, and project approval
ST-3	Technical Department	Employees responsible for IT systems, providing technical requirements and supporting deployment
ST-4	End Users (Software Development Teams)	Teams that will use the application daily for their development activities, providing usability feedback and validating workflows
ST-5	Development Team	TFG development group responsible for the solution, analyzing, designing, developing, testing, and documenting the system
ST-6	Client Development Team	Technical personnel from the client organization who may review the solution and support future improvements
ST-7	Future Maintenance Team	Hypothetical team responsible for future evolution, requiring documentation and maintainability information

ST-8	NorthTech Systems Administration Team	Hypothetical team responsible for infrastructure management, deployment support, and system monitoring

Table 5: Stakeholder list

Domain and infrastructure providers are not included as stakeholders, since the deployment environment will rely on the theoretical client's existing infrastructure and no external provisioning or management is required within the scope of this project.

4.3 OBS and PBS

This section presents the Product Breakdown Structure (PBS) and Organization Breakdown Structure (OBS) of the IR-Board project. The PBS provides a hierarchical view of the main components and modules that compose the final product, while the OBS defines the organizational structure and the distribution of responsibilities among the roles involved in the project.

The purpose of these structures is to establish a clear understanding of the system scope, its main elements, and the relationship between the product components and the theoretical teams responsible for their development and management.

4.3.1 PBS

The Product Breakdown Structure (PBS) shown below decomposes IR-Board into its main functional components. At the highest level, the system is divided into several modules that represent the main capabilities offered by the platform.

The security and access control module contains the mechanisms required to protect the system, including authentication, Relation-Based Access Control (ReBAC), and the Zero-Trust security model. The requirements management module represents the core functionality of the platform, covering functional and non-functional requirements, their lifecycle, and traceability between related elements.

Project management provides the necessary tools to organize projects, manage their functionalities, and control approval workflows. User management handles users and permissions, while the project element management module groups additional entities managed within projects, such as documents and stakeholders.

Finally, collaboration services provide support for concurrent work through entity locking, and the search and filtering module improves accessibility by allowing users to efficiently locate and organize project information.

This decomposition defines the main product boundaries and provides a high-level overview of the system architecture from a functional perspective.

Product Breakdown Structure - IR-Board

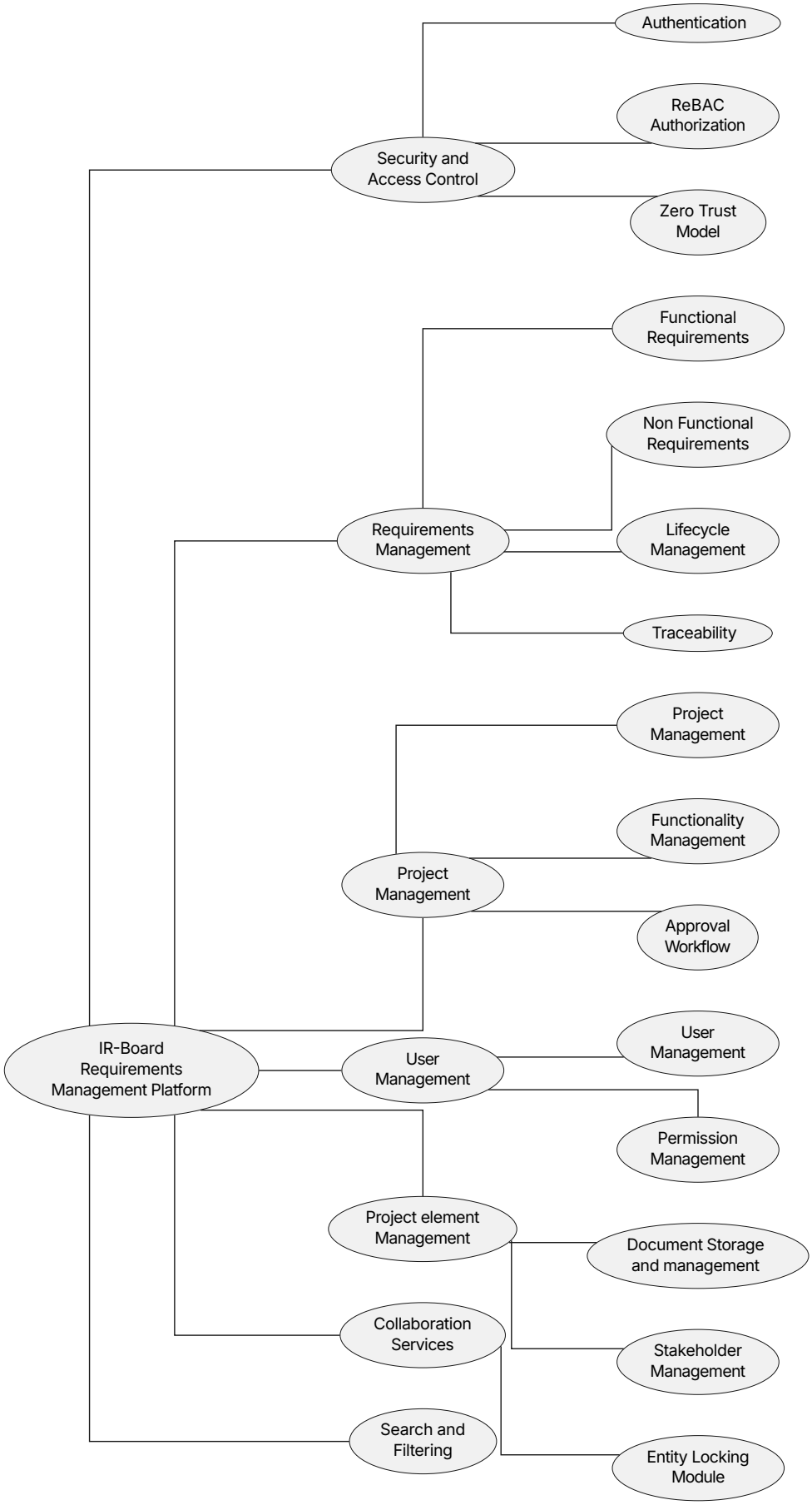

Figure 2: PBS

4.3.2 OBS

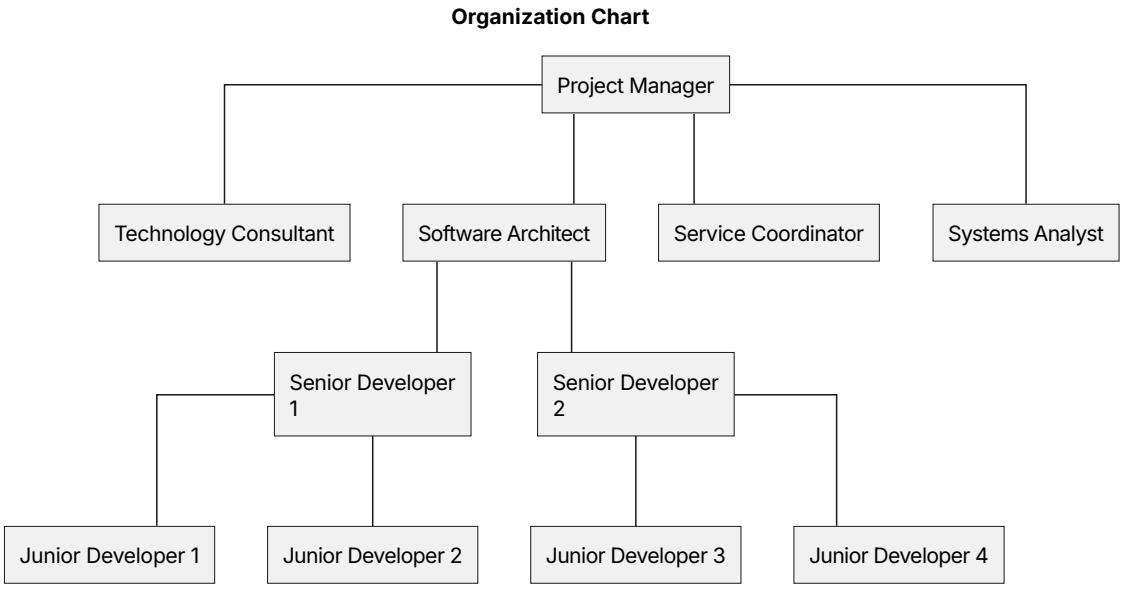

Figure 3: OBS

A RACI responsibility matrix has been used to define the responsibilities and participation of each stakeholder throughout the project. This matrix helps clarify the involvement of each role during the different project activities.

The RACI model defines the following categories:

- R (Responsible): Person or role responsible for carrying out the activity.
- A (Accountable): Person or role who approves the result and has the final responsibility.
- C (Consulted): Person or role whose expertise or opinion is required.
- I (Informed): Person or role who must be notified about the progress or results.

The roles considered in the project are:

- Project Manager (PM)
- Systems Analyst (SA)
- Service Coordinator (SC)
- Technology Consultant (TC)
- Software Architect (SWA)
- Senior Developers (SD)
- Junior Developers (JD)

The following matrix defines the responsibility distribution for the main project activities:

Activity	PM	SA	SC	TC	SWA	SD	JD
Project Management	R	I	I	I	I	I	I
Requirements Analysis	A	R	R	I	I	I	I
Software Design	I	A	C	R	R	I	I

Software Development	I	I	I	C	A	R	C
Testing and Validation	I	A	C	I	I	R	R
Documentation	A	I	I	I	C	R	R

4.4 Initial Planning

The initial project planning was developed based on the Product Breakdown Structure (PBS) of the system, which can be found here. The PBS was defined through an iterative process of refinement with the client (outside the theoretical context, this process was carried out with the project tutors) until a suitable system scope and set of functional requirements were established.

From the identified requirements, the main system modules were derived and used as the basis for creating the project breakdown structure and the initial planning. It should be noted that this planning does not consider strict execution deadlines or real resource constraints, as its main objective is to estimate the required resources and project budget. Therefore, no distinction has been made between individual team members within the same professional profile; for example, tasks assigned to junior developers are considered equivalent regardless of whether they are performed by Junior Developer 1 or Junior Developer 2.

The estimated effort, task durations, and assignment of professional profiles were defined using expert judgement, considering the complexity of each activity and the expected workload required for its completion.

It should also be noted that, since this project is developed by a single student rather than a complete professional team, this schedule should not be read as a literal, day-by-day execution plan. Tasks, durations, and resource assignments are modeled as if carried out by a full team of differentiated professional profiles, in order to provide a realistic and comparable basis for budgeting and to preserve the professional context each role would represent. In practice, every task is carried out by the same person, frequently interleaved, paused, or resumed in ways this schedule does not capture. The planning therefore prioritizes representing the effort, structure, and professional context of the project over strict accuracy in its day-to-day execution.

ID	Task	Work	Profile	Start	End
0	**IRBoard development**	300 hrs		Thu 01/01/26	Fri 13/02/26
1	Project management	30 hrs		Thu 01/01/26	Fri 13/02/26
1.1	Design project schedule	2 hrs	Project manager	Thu 01/01/26	Thu 01/01/26
1.2	Generate budget	2 hrs	Project manager	Thu 01/01/26	Thu 01/01/26
1.3	Periodic project management	26 hrs		Fri 02/01/26	Fri 13/02/26
1.3.1	Periodic project management 1	2 hrs	Project manager	Fri 02/01/26	Fri 02/01/26
1.3.2	Periodic project management 2	2 hrs	Project manager	Mon 05/01/26	Mon 05/01/26
1.3.3	Periodic project management 3	2 hrs	Project manager	Fri 09/01/26	Fri 09/01/26
1.3.4	Periodic project management 4	2 hrs	Project manager	Mon 12/01/26	Mon 12/01/26
1.3.5	Periodic project management 5	2 hrs	Project manager	Fri 16/01/26	Fri 16/01/26
1.3.6	Periodic project management 6	2 hrs	Project manager	Mon 19/01/26	Mon 19/01/26
1.3.7	Periodic project management 7	2 hrs	Project manager	Fri 23/01/26	Fri 23/01/26
1.3.8	Periodic project management 8	2 hrs	Project manager	Mon 26/01/26	Mon 26/01/26
1.3.9	Periodic project management 9	2 hrs	Project manager	Fri 30/01/26	Fri 30/01/26
1.3.10	Periodic project management 10	2 hrs	Project manager	Mon 02/02/26	Mon 02/02/26
1.3.11	Periodic project management 11	2 hrs	Project manager	Fri 06/02/26	Fri 06/02/26
1.3.12	Periodic project management 12	2 hrs	Project manager	Mon 09/02/26	Mon 09/02/26
1.3.13	Periodic project management 13	2 hrs	Project manager	Fri 13/02/26	Fri 13/02/26
2	Analysis/Software Requirements	54 hrs		Thu 01/01/26	Fri 09/01/26
2.1	Determine project scope	1 hr	System Analyst	Thu 01/01/26	Thu 01/01/26
2.2	Review Regulatory Standards	4 hrs	Technology consultant	Thu 01/01/26	Thu 01/01/26
2.3	Identify required Documentation	2 hrs	System Analyst	Thu 01/01/26	Thu 01/01/26
2.4	Determine hand-ins for the project	1 hr	System Analyst	Thu 01/01/26	Thu 01/01/26
2.5	Adapt project template to typst	2 hrs	System Analyst	Fri 02/01/26	Fri 02/01/26

2.6	Analyze existing systems	6 hrs	Technology consultant	Fri 02/01/26	Fri 02/01/26
2.7	Draft preliminary software requirements	22 hrs	System Analyst	Mon 05/01/26	Wed 07/01/26
2.8	Model auxiliary diagrams	6 hrs	System Analyst	Wed 07/01/26	Thu 08/01/26
2.9	Review software requirements	2 hrs	System Analyst	Thu 08/01/26	Thu 08/01/26
2.10	Modify requirements with feedback	8 hrs	System Analyst	Thu 08/01/26	Fri 09/01/26
3	Analysis complete	0 hrs		Fri 09/01/26	Fri 09/01/26
4	Design	51 hrs		Fri 09/01/26	Fri 16/01/26
4.1	Design architecture	10 hrs	Software architect	Fri 09/01/26	Mon 12/01/26
4.2	Design brand identity	3 hrs	Junior software engineer	Fri 09/01/26	Mon 12/01/26
4.3	Design project management	8 hrs	Senior software engineer	Mon 12/01/26	Tue 13/01/26
4.4	Design stakeholder management	4 hrs	Senior software engineer	Tue 13/01/26	Tue 13/01/26
4.5	Design requirement management	6 hrs	Senior software engineer	Tue 13/01/26	Wed 14/01/26
4.6	Design user management	10 hrs	Senior software engineer	Wed 14/01/26	Thu 15/01/26
4.7	Design document management and diagram modelling	10 hrs	Senior software engineer	Thu 15/01/26	Fri 16/01/26
5	Design complete	0 hrs		Fri 16/01/26	Fri 16/01/26
6	Set up SonarQube for Quality Assurance	3 hrs	Senior software engineer	Fri 16/01/26	Mon 19/01/26
7	Development	105 hrs		Mon 19/01/26	Thu 05/02/26
7.1	Set up architecture	4 hrs	Software architect	Mon 19/01/26	Mon 19/01/26
7.2	Set up development enviroment	3 hrs	Software architect	Mon 19/01/26	Tue 20/01/26

7.3	Develop code	98 hrs		Tue 20/01/26	Thu 05/02/26
7.3.1	Develop project management module	22 hrs	Junior software engineer	Tue 20/01/26	Thu 22/01/26
7.3.2	Develop stakeholder management module	10 hrs	Junior software engineer	Thu 22/01/26	Mon 26/01/26
7.3.3	Develop requirement management module	25 hrs	Senior software engineer	Mon 26/01/26	Thu 29/01/26
7.3.4	Develop user management module	6 hrs	Junior software engineer	Thu 29/01/26	Thu 29/01/26
7.3.5	Develop document management	15 hrs	Senior software engineer	Fri 30/01/26	Mon 02/02/26
7.3.6	Develop modifying concurrency system	10 hrs	Senior software engineer	Mon 02/02/26	Wed 04/02/26
7.3.7	Develop search and filtering	10 hrs	Junior software engineer	Wed 04/02/26	Thu 05/02/26
8	Development complete	0 hrs		Thu 05/02/26	Thu 05/02/26
9	Testing	27 hrs		Fri 16/01/26	Thu 05/02/26
9.1	Test project management module	4 hrs	Junior software engineer	Mon 26/01/26	Mon 26/01/26
9.2	Test stakeholder management module	3 hrs	Junior software engineer	Mon 26/01/26	Mon 26/01/26
9.3	Test requirement management module	5 hrs	Junior software engineer	Fri 30/01/26	Fri 30/01/26
9.4	Test user management module	3 hrs	Junior software engineer	Fri 30/01/26	Fri 30/01/26
9.5	Test document management	3 hrs	Junior software engineer	Mon 02/02/26	Tue 03/02/26

9.6	Test modifying concurrency system	2 hrs	Junior software engineer	Thu 05/02/26	Thu 05/02/26
9.7	Test search and filtering	3 hrs	Junior software engineer	Thu 05/02/26	Thu 05/02/26
9.8	Usability and accessibility testing	2 hrs	Junior software engineer	Fri 16/01/26	Mon 19/01/26
9.9	Load testing	2 hrs	Senior software engineer	Mon 19/01/26	Tue 20/01/26
10	Testing complete	0 hrs		Thu 05/02/26	Thu 05/02/26
11	Documentation	30 hrs		Thu 01/01/26	Mon 09/02/26
11.1	Document declaration of originality, abstract and keywords	1 hr	Project manager	Thu 01/01/26	Thu 01/01/26
11.2	Document introduction	1 hr	Project manager	Thu 01/01/26	Thu 01/01/26
11.3	Document theoretical background	1 hr	Technology consultant	Thu 01/01/26	Thu 01/01/26
11.4	Document feasibility study and alternative analysis	2 hrs	Technology consultant	Thu 01/01/26	Thu 01/01/26
11.5	Document Initial project planning and management	2 hrs	Project manager	Fri 02/01/26	Fri 02/01/26
11.6	Document system analysis	3 hrs	System Analyst	Fri 02/01/26	Fri 02/01/26
11.7	Document system design	5 hrs	Software architect	Tue 13/01/26	Tue 13/01/26
11.8	Document system implementation	5 hrs	Senior software engineer	Thu 05/02/26	Thu 05/02/26
11.9	Document test plan execution	2 hrs	Senior software engineer	Fri 06/02/26	Fri 06/02/26
11.10	Document system manuals	5 hrs	Senior software engineer	Fri 06/02/26	Fri 06/02/26

11.11	Document final project closure	2 hrs	Project manager	Fri 06/02/26	Mon 09/02/26
11.12	Document conclusions and future work	1 hr	Project manager	Mon 09/02/26	Mon 09/02/26
12	Documentation complete	0 hrs		Mon 09/02/26	Mon 09/02/26

Table 6: Project initial planning

Which gives the following Gantt chart:

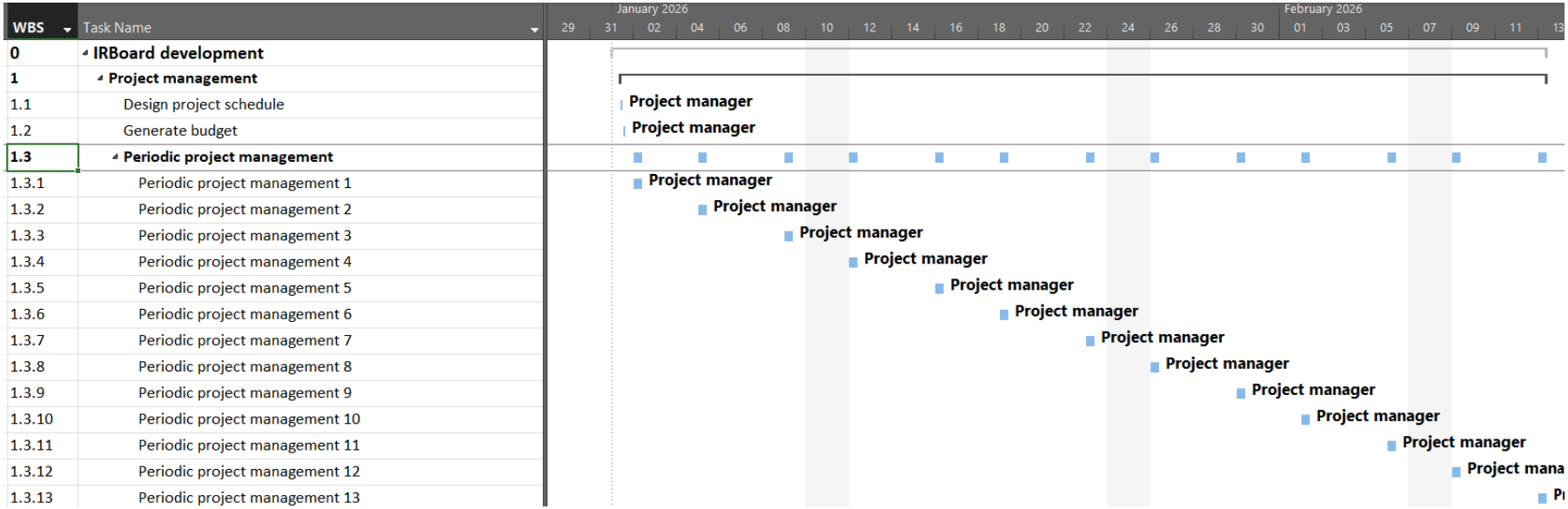

Figure 4: Project management Gantt chart

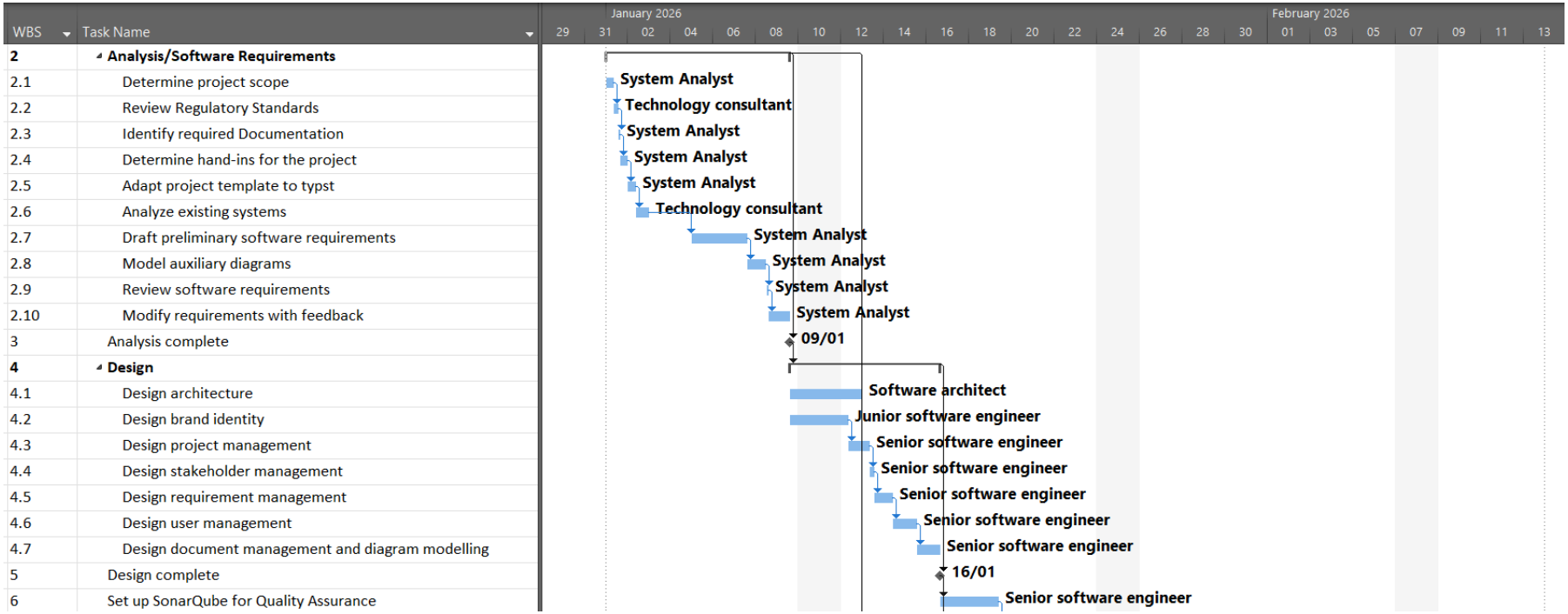

Figure 5: Analysis and design Gantt chart

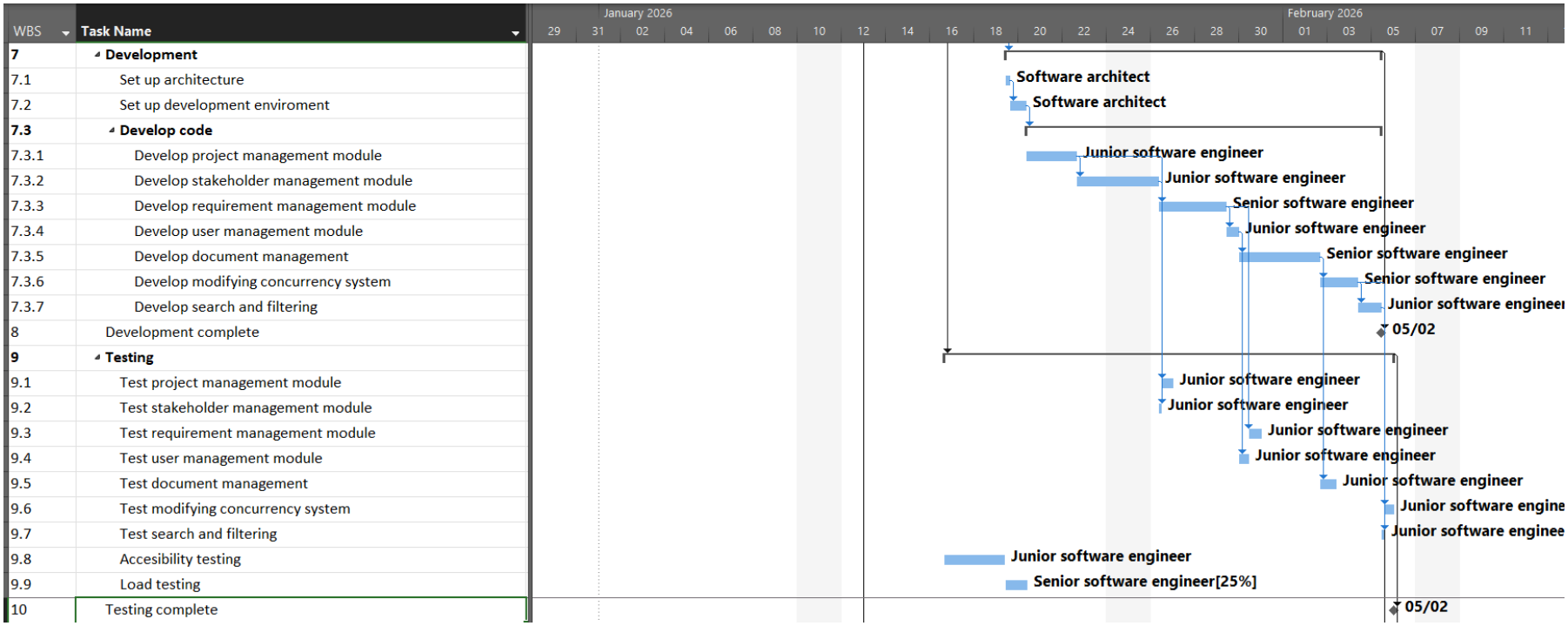

Figure 6: Development and testing Gantt chart

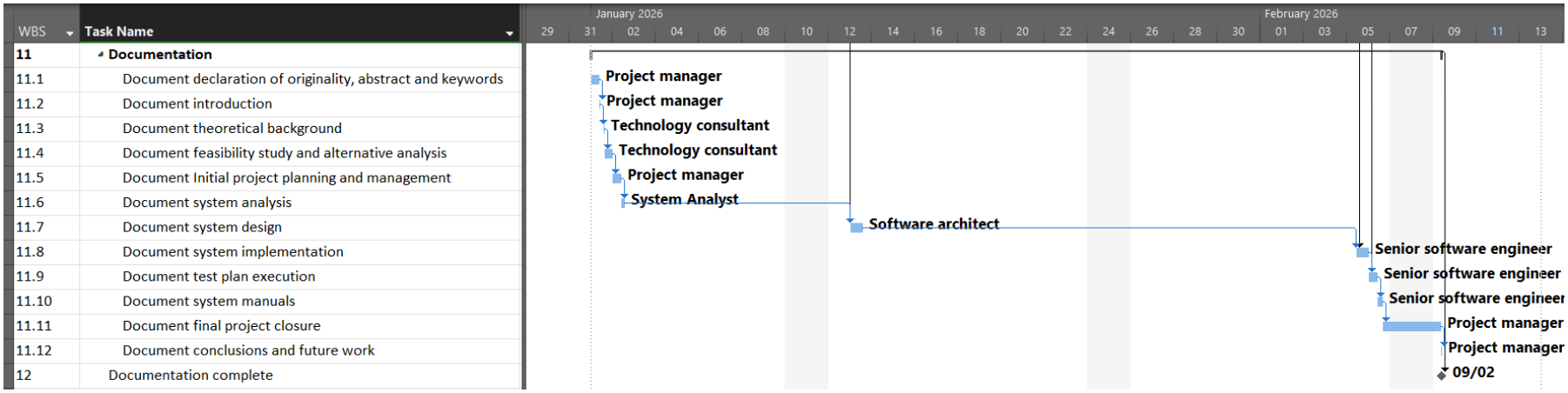

Figure 7: Documentation Gantt chart

4.5 Risk Analysis

Having defined the objectives pursued, the tasks required to achieve them, and the people involved in the project, this section addresses risk management for the risks identified during the planning phase. It presents the guidelines, techniques, and assessments carried out to counteract their possible effects on the project, following the proposal defined in the PMBOK. Accordingly, the phases of risk management planning, risk identification, and an analysis including prioritization and the corresponding response planning are carried out below.

4.5.1 Risk Management Planning

To correctly address the risks applicable to the project, each identified risk is classified according to different metrics in order to relate them and obtain a prioritization that facilitates their management. First, each identified risk can be assigned a category, in this case following the PMBOK classification shown in the figure below.

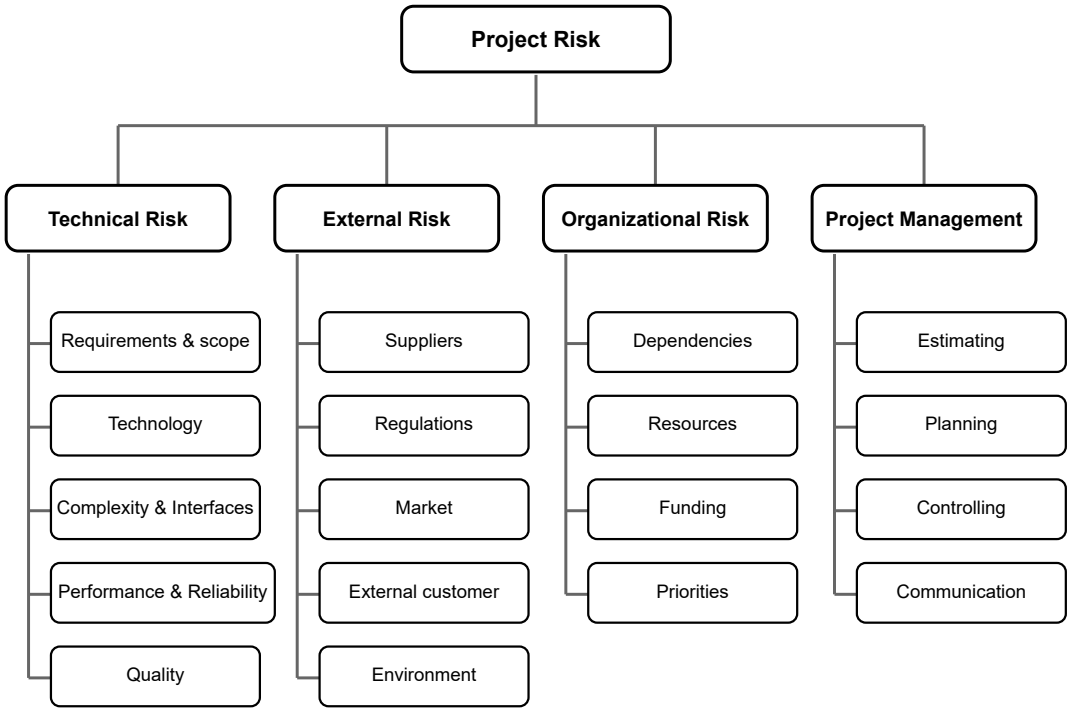

Figure 8: Risk classification according to PMBOK

Once each risk has been classified, two important metrics are defined for prioritization purposes. First, the probability of occurrence is defined, since some risks may be especially harmful to the project but, if their probability of occurrence is very low, their relevance to the project is also reduced. Second, the impact is defined, presented on a five-level scale according to its influence on different aspects of project development, namely cost, schedule, scope, and quality. These two metrics are jointly used to build a probability/impact matrix that provides a direct prioritization based on both variables.

Probability	Threats					Opportunities				
0.90	0.05	0.14	0.27	0.50	0.81	0.81	0.50	0.27	0.14	0.05
0.70	0.04	0.11	0.21	0.39	0.63	0.63	0.39	0.21	0.11	0.04
0.50	0.03	0.08	0.15	0.28	0.45	0.45	0.28	0.15	0.08	0.03
0.30	0.02	0.05	0.09	0.17	0.27	0.27	0.17	0.09	0.05	0.02
0.10	0.01	0.02	0.03	0.06	0.09	0.09	0.06	0.03	0.02	0.01
	0.05 Very low	**0.15** Low	**0.30** Moderate	**0.55** High	**0.90** Very high	**0.90** Very high	**0.55** High	**0.30** Moderate	**0.15** Low	**0.05** Very low
	Negative Impact					**Positive Impact**				

Table 7: Probability and impact risk matrix

As a last relevant characteristic for correct risk management, the response to be given to each risk is defined once the previous variables have been identified. In general terms, these responses are: avoidance of the exposure to the risk, transfer to a third party who assumes it, mitigation by keeping the risk factors under control, and acceptance by the responsible parties, who then live with the consequences should the risk materialize.

Given the limited budget and reduced team size of this project (a single developer), every response strategy considered below is designed to be absorbed within the existing schedule slack and assigned resources, without requiring additional budget or the creation of a contingency reserve.

4.5.2 Risk and opportunity Identification

Once the metrics and behaviors to adopt against risks and opportunities have been defined, research was carried out during the initial phases of the project regarding the most realistic situations that could materialize during its development. These are presented below.

4.5.2.1 Risk: Erroneous identification of requirements

This risk arises from a poor definition of requirements (given the little to no experience of the software engineer) during the phases intended for that purpose at the beginning of the project, or from needs expressed by the theoretical product owner (the project tutors, acting as the client's representatives) that are overlooked by the developer when building the system.

4.5.2.2 Risk: Modification of weekly working schedule

Due to the developer's current situation, who throughout the development of the entire project is simultaneously completing the Bachelor's Degree in Software Engineering and undertaking a professional internship at an external company, along with the possible academic needs that combining both activities may involve, or other factors such as illness, injury, holidays, or specific interests/needs of the rest

of the people involved, the calendar set out in the planning may be prone to weekly variation and irregularity.

4.5.2.3 Risk: Lack of experience with the Ory ecosystem and ReBAC implementation

The selected security architecture relies on components from the Ory ecosystem (Kratos, Oathkeeper, and Keto) to implement identity management and Relation-Based Access Control. As these components had not been used previously by the developer, their integration, configuration, and the correct modelling of relationship-based permissions may require a learning process longer than initially estimated.

4.5.2.4 Risk: Incompatibility of expected workflow with ory ecosystem

This risk arises from potential mismatches between the designed system workflows (particularly around authentication flows, authorization decisions, and ReBAC-driven permission resolution) and the actual behavior, constraints, or configuration complexity of the selected Ory ecosystem components, namely Ory Kratos, Ory Oathkeeper, and Ory Keto.

4.5.2.5 Opportunity: Acceleration and robustness gained through Ory ecosystem integration

This opportunity arises from the strategic adoption of the Ory ecosystem components, which can significantly reduce the implementation effort required for identity management, authentication, and authorization while improving the overall security robustness of the system. By leveraging Ory Kratos, Ory Oathkeeper, and Ory Keto, the project can benefit from production-grade security patterns without needing to implement complex security logic internally.

This also creates an opportunity to focus development effort on core domain functionalities (requirements lifecycle, traceability, collaboration, and workflow automation), while delegating security-critical concerns to specialized and widely adopted open-source solutions.

4.5.3 Risk Analysis

Below is the prioritized list of the previously identified risks and opportunities, ordered from highest to lowest impact, together with their corresponding probability and impact assessments, the selected management strategy, and the specific response actions to be applied.

As observed, the modification of the weekly working schedule represents the risk with the highest potential impact and therefore requires the closest monitoring, particularly regarding its possible effects on the project timeline and planned activities. Following this, special attention must be given to the correct identification of requirements and the prevention of inconsistencies between stakeholder expectations and the implemented solution, as these could propagate into later development phases.

The risks associated with the adoption of the Ory ecosystem, including the learning curve and possible workflow incompatibilities, represent moderate but manageable technical risks due to the complexity of integrating external security components.

Additionally, the opportunity derived from the use of open-source security components must be actively considered, as leveraging existing solutions can significantly reduce implementation effort, improve system robustness, and allow more resources to be focused on the core functionalities of the platform.

Risk: Modification of weekly working schedule			
Category: Project management: Schedule			
Probability: Very High			
Cost	**Schedule**	**Scope**	**Quality**
Medium	Critical	Medium	Low
0.81			
Strategy: Mitigation			
Response: Controlled redistribution of working hours between weeks to comply with the planned weekly workload, compensating shortfalls in subsequent weeks without altering the overall project deadline.			

Table 8: Evaluation of modification of weekly working schedule risk

Risk: Erroneous identification of requirements			
Category: Technical: Requirements			
Probability: Medium			
Cost	**Schedule**	**Scope**	**Quality**
Medium	High	High	Critical
0.45			
Strategy: Mitigation			
Response: Close involvement of and clarification with the project tutors during the initial identification process, together with continued review in periodic meetings to control possible misalignments before they propagate into design and development.			

Table 9: Evaluation of erroneous identification of requirements risk

Opportunity: Acceleration and robustness gained through Ory ecosystem integration			
Category: Technical: Security / Architecture			
Likelihood: High			
Benefit	**Schedule**	**Scope**	**Quality**
High	High	Medium	High
0.39			
Strategy: Exploitation			

Response: Maximize reuse of Ory ecosystem capabilities by aligning system design with native authentication and authorization flows, enabling faster implementation of secure access control, reducing custom security code, and improving long-term maintainability and auditability of the platform.

Table 10: Evaluation of acceleration opportunity

Risk: Lack of experience with the Ory ecosystem and ReBAC implementation			
Category: Technical: Complexity and Interfaces			
Probability: Medium			
Cost	**Schedule**	**Scope**	**Quality**
Medium	High	Low	Medium
0.28			
Strategy: Mitigation			
Response: Allocation of additional self-study hours within the existing schedule slack, reliance on official Ory documentation and community examples, and early prototyping of the most critical Kratos/Oathkeeper/Keto integration points to surface issues before they affect later phases.			

Table 11: Evaluation of lack of experience risk

Risk: Incompatibility of expected workflow with ory ecosystem			
Category: Technical: Architecture / Integration			
Probability: Medium (0.42)			
Cost	**Schedule**	**Scope**	**Quality**
Medium	High	Medium	High
0.28			
Strategy: Mitigation			
Response: Introduce an abstraction layer between the application domain model and the Ory ecosystem services, allowing internal workflows (requirements, projects, and permissions) to remain stable even if underlying authentication or authorization flows require adaptation. Where necessary, simplify the ReBAC model to align with Ory Keto capabilities and restrict overly complex relationship patterns that cannot be efficiently expressed in the external authorization layer.			

Table 12: Evaluation of incompatibility risk

4.6 Initial Budget

The budget is divided into three stages. The first stage describes the provider's financial reality, establishing the internal cost model. The second stage calculates the specific cost of executing the project for the theoretical provider. Finally, the third stage converts this cost into the final client offer, diluting the non-billable costs into each main task. In a situation where hardware is to be purchased, or where a price is standard or set by the market, these budget lines would be avoided for the dilution, as it is expected to match the market.

4.6.1 Provider finantial reality

The provider's financial reality reflects the internal cost structure of the organization responsible for delivering the project. It encompasses the personnel required to operate the company, their annual employment costs (including gross salary and employer contributions), and the corresponding hourly cost rates. The hourly rate excluding profit represents the minimum rate required to recover costs, while the hourly rate including profit incorporates the target commercial margin applied when invoicing clients.

To ensure that the salary assumptions are aligned with current market conditions, the gross annual salaries for each professional profile were obtained from Indeed, while the total employer cost was estimated using the Factorial employee cost calculator.

The complete spreadsheet is available here.

Resource	Num.	Annual employment cost	Hourly rate (without profit)	Hourly rate (with profit)
General manager	1	91.781,35 €	-	-
Project manager	1	53.039,40 €	49,78 €	55,62 €
Service coordinator	1	32.876,58 €	46,75 €	54,35 €
Systems analyst	1	46.530,79 €	42,16 €	46,91 €
Technology consultant	1	54.920,30 €	45,22 €	49,69 €
Software architect	1	58.034,65 €	49,15 €	54,22 €
Senior developer	2	104.323,46 €	43,85 €	48,31 €
Junior developer	4	135.362,32 €	38,55 €	43,98 €
Sales representative	1	35.118,49 €	-	-
TOTAL	**13**	**611.987,34 €**		

4.6.2 Initial provider cost breakdown

On this section of the budget, the complete cost structure is calculated and layed out, and the proportional amount to be diluted is calculated.

Category 1: IrBoard development costs breakdown										
I1	**I2**	**I3**	**Description**	**Resource**	**Quantity**	**Units**	**Price**	**Subtotal**	**Subtotal (2)**	**Total**
01			Project management							1.493,46 €
	001		Design project schedule	Project manager	2	Hours	49,78 €	99,56 €	99,56 €	
	002		Generate budget	Project manager	2	Hours	49,78 €	99,56 €	99,56 €	
	003		Periodic project management						1.294,34 €	
		01	Periodic project management 1	Project manager	2	Hours	49,78 €	99,56 €		
		02	Periodic project management 2	Project manager	2	Hours	49,78 €	99,56 €		
		03	Periodic project management 3	Project manager	2	Hours	49,78 €	99,56 €		
		04	Periodic project management 4	Project manager	2	Hours	49,78 €	99,56 €		
		05	Periodic project management 5	Project manager	2	Hours	49,78 €	99,56 €		
		06	Periodic project management 6	Project manager	2	Hours	49,78 €	99,56 €		
		07	Periodic project management 7	Project manager	2	Hours	49,78 €	99,56 €		

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
		08	Periodic project management 8	Project manager	2	Hours	49,78 €	99,56 €		
		09	Periodic project management 9	Project manager	2	Hours	49,78 €	99,56 €		
		10	Periodic project management 10	Project manager	2	Hours	49,78 €	99,56 €		
		11	Periodic project management 11	Project manager	2	Hours	49,78 €	99,56 €		
		12	Periodic project management 12	Project manager	2	Hours	49,78 €	99,56 €		
		13	Periodic project management 13	Project manager	2	Hours	49,78 €	99,56 €		
02			Analysis / Software Requirements							2.307,21 €
	001		Determine project scope	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €	
	002		Review Regulatory Standards	Technology consultant	4	Hours	45,22 €	180,88 €	180,88 €	
	003		Identify required Documentation	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	
	004		Determine hand-ins for the project	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €	
	005		Adapt project template to Typst	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	006		Analyze existing systems	Technology consultant	6	Hours	45,22 €	271,32 €	271,32 €	
	007		Draft preliminary software requirements	System Analyst	22	Hours	42,16 €	927,50 €	927,50 €	
	008		Model auxiliary diagrams	System Analyst	6	Hours	42,16 €	252,96 €	252,96 €	
	009		Review software requirements	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	
	010		Modify requirements with feedback	System Analyst	8	Hours	42,16 €	337,27 €	337,27 €	
03			Design							2.273,37 €
	001		Design architecture	Software architect	10	Hours	49,15 €	491,54 €	491,54 €	
	002		Design brand identity	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	003		Design project management	Senior software engineer	8	Hours	43,85 €	350,77 €	350,77 €	
	004		Design stakeholder management	Senior software engineer	4	Hours	43,85 €	175,39 €	175,39 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	005		Design requirement management	Senior software engineer	6	Hours	43,85 €	263,08 €	263,08 €	
	006		Design user management	Senior software engineer	10	Hours	43,85 €	438,47 €	438,47 €	
	007		Design document management	Senior software engineer	10	Hours	43,85 €	438,47 €	438,47 €	
04			Set up SonarQube for Quality Assurance	Senior software engineer	3	Hours	43,85 €	131,54 €	131,54 €	131,54 €
05			Development							4.386,90 €
	001		Set up architecture	Software architect	4	Hours	49,15 €	196,62 €	196,62 €	
	002		Set up development environment	Software architect	3	Hours	49,15 €	147,46 €	147,46 €	
	003		Develop code						4.042,82 €	
		01	Develop project management module	Junior software engineer	22	Hours	38,55 €	848,14 €		
		02	Develop stakeholder management module	Junior software engineer	10	Hours	38,55 €	385,52 €		

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
		03	Develop requirement management module	Senior software engineer	25	Hours	43,85 €	1.096,17 €		
		04	Develop user management module	Junior software engineer	6	Hours	38,55 €	231,31 €		
		05	Develop document management module	Senior software engineer	15	Hours	43,85 €	657,70 €		
		06	Develop modifying concurrency system	Senior software engineer	10	Hours	43,85 €	438,47 €		
		07	Develop search and filtering	Junior software engineer	10	Hours	38,55 €	385,52 €		
06			Testing							1.051,49 €
	001		Test project management module	Junior software engineer	4	Hours	38,55 €	154,21 €	154,21 €	
	002		Test stakeholder management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	003		Test requirement management module	Junior software engineer	5	Hours	38,55 €	192,76 €	192,76 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	004		Test user management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	005		Test document management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	006		Test modifying concurrency system	Junior software engineer	2	Hours	38,55 €	77,10 €	77,10 €	
	007		Test search and filtering	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	008		Usability and accessibility testing	Junior software engineer	2	Hours	38,55 €	77,10 €	77,10 €	
	009		Load testing	Senior software engineer	2	Hours	43,85 €	87,69 €	87,69 €	
07			Documentation							1.382,54 €
	001		Document declaration of originality, abstract and keywords	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	
	002		Document introduction	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	003		Document theoretical background	Technology consultant	1	Hours	45,22 €	45,22 €	45,22 €	
	004		Document feasibility study and alternative analysis	Technology consultant	2	Hours	90,44 €	90,44 €	90,44 €	
	005		Document initial project planning and management	Project manager	2	Hours	99,56 €	99,56 €	99,56 €	
	006		Document system analysis	System Analyst	3	Hours	126,48 €	126,48 €	126,48 €	
	007		Document system design	Software architect	5	Hours	245,77 €	245,77 €	245,77 €	
	008		Document system implementation	Senior software engineer	5	Hours	219,23 €	219,23 €	219,23 €	
	009		Document test plan execution	Senior software engineer	2	Hours	87,69 €	87,69 €	87,69 €	
	010		Document system manuals	Senior software engineer	5	Hours	219,23 €	219,23 €	219,23 €	
	011		Document final project closure	Project manager	2	Hours	99,56 €	99,56 €	99,56 €	
	012		Document conclusions and future work	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
Total										13.026,52 €

Table 13: Provider's budget category 1

Category 2: Other										
I1	**I2**	**I3**	**Description**	**Resource**	**Quantity**	**Units**	**Price**	**Subtotal**	**Subtotal (2)**	**Total**
01			Travel and transportation expenses						172,50 €	172,50 €
	001		Travel to client facilities (deployment/support)		150	km	0,35 €	52,50 €		
	002		Meals/daily allowance		6	meals	20,00 €	120,00 €		

Table 14: Provider's budget category 2

Which is the same as:

IrBoard costs		
Cat. Num	**Category**	**Total**
01	IrBoard development costs breakdown	13.026,52 €
02	Other	172,50 €
03	Profit (25%)	3.299,75 €
total		16.498,77 €

Table 15: Provider's budget summary

And allows us to compute the following metrics for the client's budget:

- Amount to be increased on the first category: 3.472,25 €

- Total billable hours: 300,00 hours
- Dilution increase per billable hour: 11,58 €

4.6.3 Client's budget

As stated previously, the client-facing budget includes only directly billable cost items. Budget lines with established or market-standard values cannot be artificially increased to absorb non-billable expenses such as profit margins, travel costs, or meal allowances. In this case, however, such adjustments are unnecessary because no hardware acquisition is required for the project.

Detailed client budget				
I1	**I2**	**Description**	**Cost**	**Total**
01		Project management		1.840,86 €
	001	Design project schedule	122,72 €	
	002	Generate budget	122,72 €	
	003	Periodic project management	1.595,42 €	
02		Analysis/Software Requirements		2.932,53 €
	001	Determine project scope	53,74 €	
	002	Review Regulatory Standards	227,20 €	
	003	Identify required Documentation	107,48 €	
	004	Determine hand-ins for the project	53,74 €	
	005	Adapt project template to typst	107,48 €	
	006	Analyze existing systems	340,80 €	
	007	Draft preliminary software requirements	1.182,26 €	
	008	Model auxiliary diagrams	322,44 €	
	009	Review software requirements	107,48 €	
	010	Modify requirements with feedback	429,91 €	
03		Design		2.863,95 €
	001	Design architecture	607,34 €	
	002	Design brand identity	150,40 €	
	003	Design project management	443,41 €	
	004	Design stakeholder management	221,71 €	
	005	Design requirement management	332,56 €	
	006	Design user management	554,27 €	
	007	Design document management	554,27 €	
04		Set up SonarQube for Quality Assurance	166,28 €	166,28 €
05		Development		5.602,80 €
	001	Set up architecture	242,94 €	
	002	Set up development enviroment	182,20 €	
	003	Develop code	5.177,66 €	
06		Testing		1.364,15 €

I1	I2	Description	Cost	Total
	001	Test project management module	200,53 €	
	002	Test stakeholder management module	150,40 €	
	003	Test requirement management module	250,66 €	
	004	Test user management module	150,40 €	
	005	Test document management	150,40 €	
	006	Test modifying concurrency system	100,26 €	
	007	Test search and filtering	150,40 €	
	008	Usability and accessibility testing	100,26 €	
	009	Load testing	110,85 €	
07		Documentation		1.729,94 €
	001	Document declaration of originality, abstract and keywords	61,36 €	
	002	Document introduction	61,36 €	
	003	Document theoretical background	56,80 €	
	004	Document feasibility study and alternative analysis	113,60 €	
	005	Document Initial project planning and management	122,72 €	
	006	Document system analysis	161,22 €	
	007	Document system design	303,67 €	
	008	Document system implementation	277,13 €	
	009	Document test plan execution	110,85 €	
	010	Document system manuals	277,13 €	
	011	Document final project closure	122,72 €	
	012	Document conclusions and future work	61,36 €	
Total				16.500,52 €

Table 16: Detailed client budget

Simplified client budget		
Cat. Num	**Category**	**Total**
01	IrBoard development costs breakdown	16.500,52 €
total		16.500,52 €

Table 17: Simplified client budget

Chapter 5. System Analysis

5.1 Users and Characteristics

In this project, instead of conventional system-wide roles, the approach I found fit best the nature of requirement management systems was a relation-based role system. Therefore, two sets of user permissions can be defined: system-level permissions and project-level permissions.

User	Description
Admin	Has access to project creation, deactivation, purge of removed projects... Still needs a project role to add to or modify a project, even if created by himself. Can link users as project manager to a project.
Basic	A basic user, the default. Has access to his profile management.

Table 18: List of permission levels on the system

A user is either an admin or not. Due to zero-trust, unless he is added as project manager to it he wont be able to modify any project, even if it was created by himself.

User	Description
ProjectManager	Access to add, edit and disable functionalities, requirements within those functionalities, documents, and can link users to that project.
RequirementEngineer	Linked to a functionality, has access to add, modify and disable requirements in that functionality, as well as with documents linked to the project.
Stakeholder	Linked to a functionality, has read-only access to the requirements of that functionality, and documents linked to the project.

Table 19: List of permissions within a project

These permissions overlap, in case a user is linked to a project in several ways. In that situation, due to this overlap, the most high level permission prevails.

For example, if a user is listed as requirement engineer and stakeholder in the same functionality, the user will be able to view (both permissions), and add, modify, disable and all other requirement engineer actions as well.

5.2 Requirements Analysis

Here are presented some diagrams that helped during the requirement edduction process.

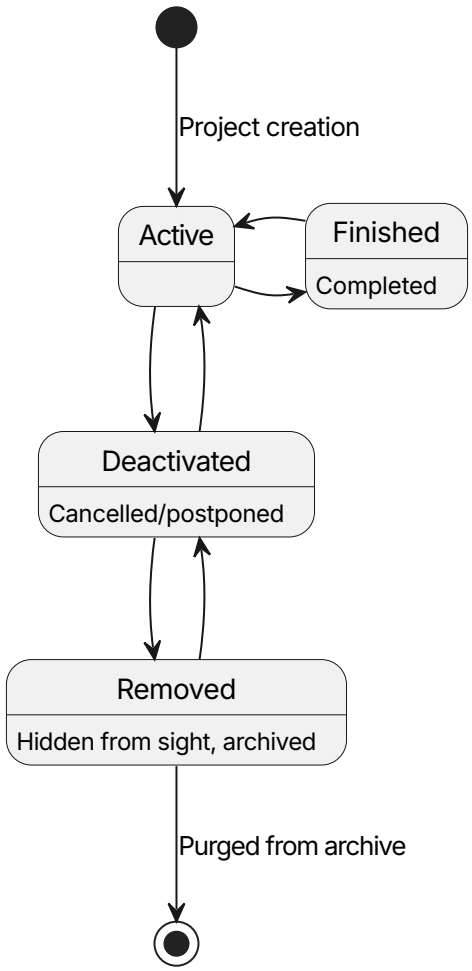

Figure 9: Lifecicle of a project entity

Active - A project entity that is currently in progress.

finished - A project entity that has been implemented. As the nature of a project is that one never ends, it does not represent an end state.

Deactivated - A project entity that has been cancelled, postponed etc; A project that is not finished but is not currently in use.

Removed - A project entity that has been archived for removal, placed in the trash bin in case it is needed as last resort.

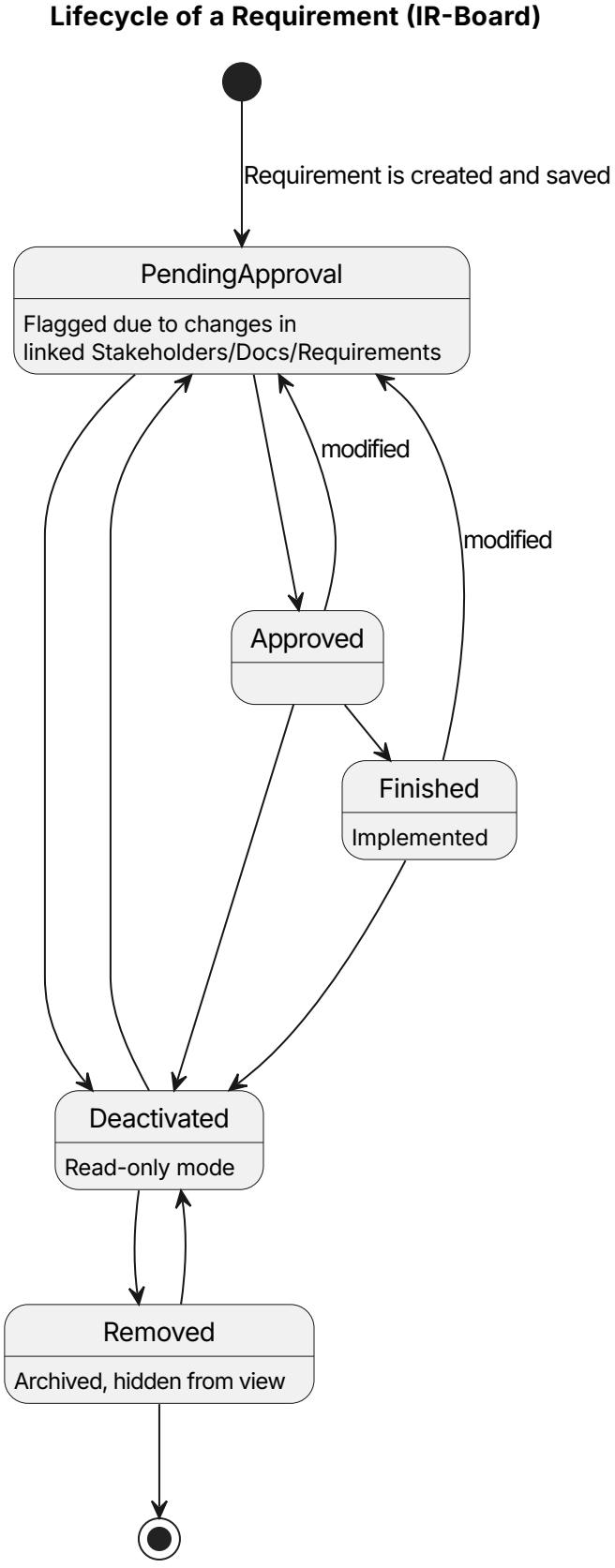

Figure 10: Requirement entity's state diagram

PendingApproval - A requirement entity that has not been validated by a stakeholder as it currently is. It's a complex state to ease development, as the pending

review can be seen as an extension of itself, and therefore can be a simple boolean flag.

PendingReview - A requirement entity that needs attention and possible modification, due to a change on a linked entity. Expected to be purely a flag.

Approved - A requirement entity that has been validated by the appropiate stakeholders with the project manager outside the system.

Deactivated - A requirement entity has been deactivated for some reason, be cancelled or an error, and does not count towards the metrics of the project.

Removed - A requirement entity that has been deemed innecessary to the project. It is hidden from view, archived.

5.3 signup flows

TODO

5.4 System Analysis

5.4.1 Class Analysis

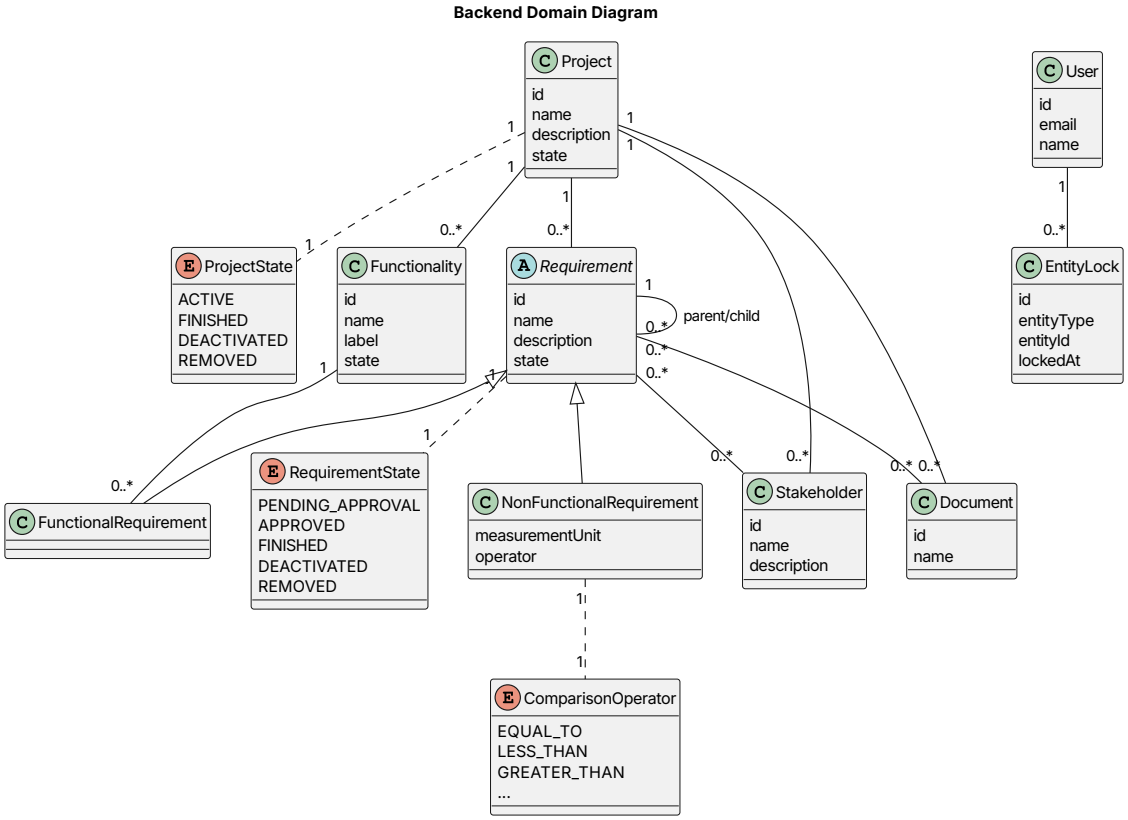

Figure 11: Analysis domain class diagram

The backend domain model defines the main entities involved in requirements management and their relationships. The model is centered around the **Project** entity, which acts as the main container for functionalities, requirements, stakeholders, and documents.

Project - Represents a software project and its lifecycle. A project groups the main elements required for requirements engineering, including functionalities, requirements, stakeholders, and documentation.

Functionality - Represents a system capability or feature within a project. Functional requirements are associated with functionalities to define the expected behavior of the system.

Requirement - Abstract entity representing a project requirement, as most of the core logic for one is indistinct from whether it is functional or non functional.

FunctionalRequirement - Represents a requirement describing what the system must do. It is linked to a specific functionality.

NonFunctionalRequirement - Represents a requirement describing measurable quality constraints. It uses a measurement unit and a comparison operator to define conditions such as performance limits.

Stakeholder - Represents an actor interested in the project. Stakeholders can be associated with projects and requirements to maintain traceability.

Document - Represents documentation artifacts related to projects or requirements, supporting requirement traceability.

User - Represents an authenticated user. Users are not directly linked to domain entities, as authorization relationships are managed externally through ReBAC (Relationship-Based Access Control).

EntityLock - Represents a temporary lock over an entity to prevent conflicting modifications during collaborative work.

ProjectState - Defines the possible lifecycle states of a project: active, finished, deactivated, or removed.

RequirementState - Defines the lifecycle states of a requirement: pending approval, approved, finished, deactivated, or removed.

ComparisonOperator - Defines the comparison logic used by non-functional requirements, such as equality or numerical comparisons.

5.4.2 Data Modeling

The domain model will be implemented using **JPA (Java Persistence API)**, where each domain entity will be mapped to a relational database table. The relationships defined in the class diagram are translated into JPA associations such as one-to-many, many-to-one, and many-to-many mappings.

5.4.2.1 Entity identifiers

The system uses three distinct identifier strategies depending on the purpose of each entity. Each persistent entity contains an internal numeric primary key (`id : bigint`) managed by the database and not intended to be exposed externally. Certain entities also carry a human-readable `entity_identifier`, generated dynamically from contextual information such as the containing project or functionality. Finally, the `Requirement` entity exposes an `order_value` used to support floating-point

ordering, allowing requirements to be reordered efficiently without renumbering the entire list.

5.4.2.2 Inheritance mapping

The abstract `Requirement` entity uses JPA's single-table inheritance strategy: both `FunctionalRequirement` and `NonFunctionalRequirement` are stored in the same `requirement` table, distinguished by a `requirement_type` discriminator column. Fields specific to non-functional requirements (`measurement_unit`, `operator`, `actual_value`, `target_value`, `threshold_value`) are nullable for functional requirement rows.

5.4.2.3 Relationship representation

Parent–child requirement nesting is modelled as a self-referential foreign key (`parent_id`) on the `requirement` table. The many-to-many associations between requirements and stakeholders, between requirements and documents, and the peer cross-linking between requirements themselves are each represented as dedicated join tables (`stakeholder_observer_requirement`, `document_observer_requirement`, and `requirement_observing` respectively). This separation makes the observer relationships explicit at the schema level and enables efficient traversal in both directions.

5.4.2.4 Enumerated state fields

Lifecycle states such as `ProjectState` and `RequirementState`, as well as the `ComparisonOperator` and `requirement_type` discriminator, are stored as database enumerations, ensuring that only controlled values can be persisted and that state transitions are enforced at the application layer before they reach the database.

5.4.2.5 Concurrency control record

The `EntityLock` entity is not part of the business domain but acts as an infrastructure record. It identifies the entity being locked by a (`entity_id`, `entity_type`) pair rather than a typed foreign key, allowing a single table to cover locks on any entity type without schema changes. A lock is always associated with the user who holds it and the project it belongs to.

5.4.3 Process Modeling

A crucial process for the system is the updates triggered by modifications between observed and observer entities. To illustrate the flows depicted on the system, the following diagram is provided:

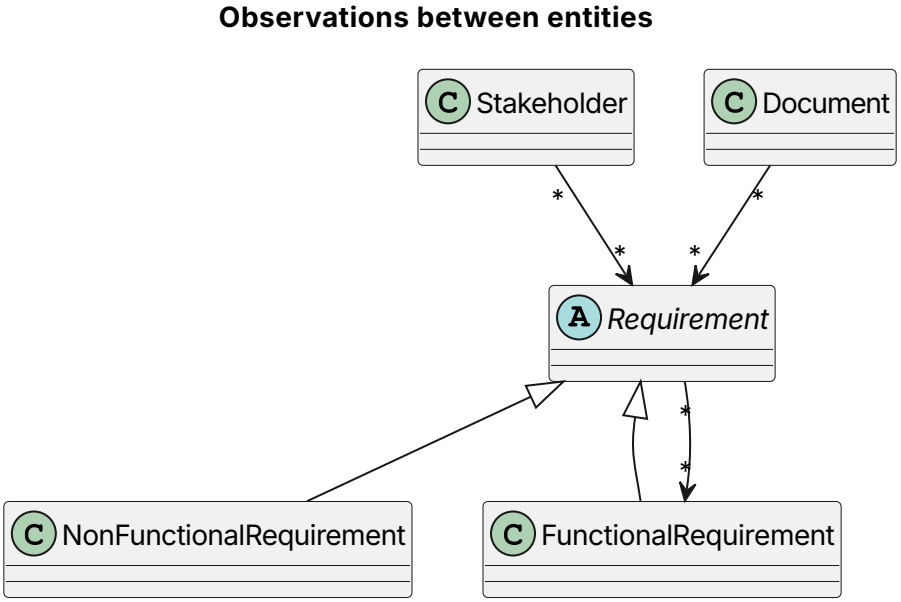

Figure 12: Possible observation processes between entities

Linking Arrows - The arrows in this diagram refer to linked entities in a modified observer pattern, to allow to search for linked elements from both sides of the relation. The direction of the arrow expresses the flow from a observed element to the observer element, for example, a modification on a Stakeholder element would trigger an update() call to all requirements observing it.

5.4.4 User Interface Definition

The user interface of IR-Board is designed around a minimalist philosophy, prioritizing clarity and reducing cognitive overhead for users managing complex requirement structures across multiple projects. Rather than exposing all available functionality at once, the interface reveals contextual actions progressively as the user navigates deeper into the entity hierarchy. The design language relies on a neutral color palette with indigo accents for interactive and active elements, using iconography and spatial layout to communicate structure without verbose labels.

Before authentication, the user only encounters the **Login** and **Registration** pages, which are simple standalone forms. Once authenticated, the persistent NavBar becomes the structural backbone of the interface, present across all views and adapting its visible links depending on whether the user is currently inside a project context or not: at the top level it shows home and administrative links, and when navigating within a project it additionally exposes the project-scoped section links for stakeholders, non-functional requirements, and documents.

5.4.4.1 User Interface Description

The following sketches represent the preliminary interface models defined during the analysis phase, prior to implementation. They establish the intended structure, layout, and content of each view, and were modified live with the tutor’s feedback.

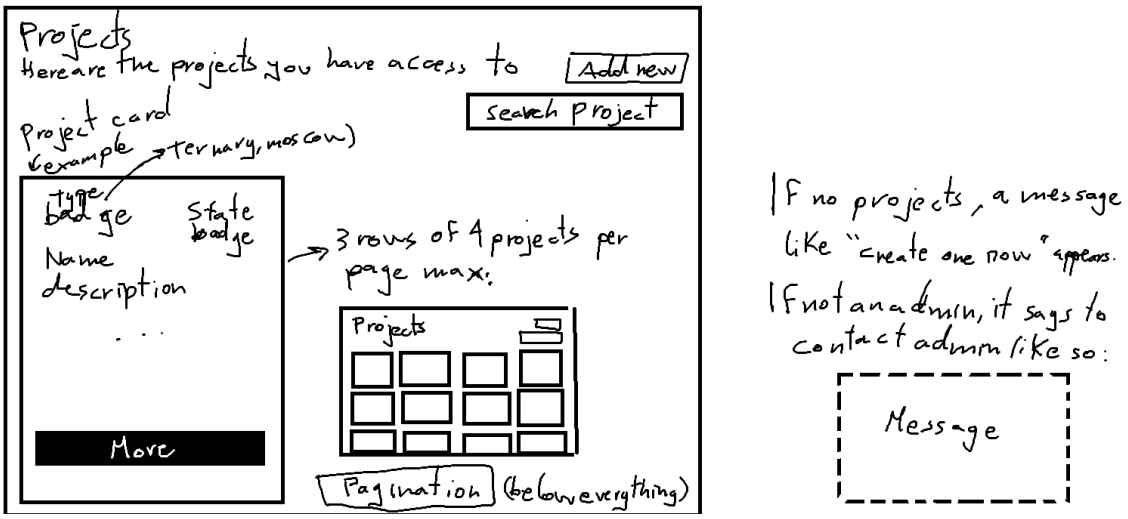

Figure 13: Home page sketch

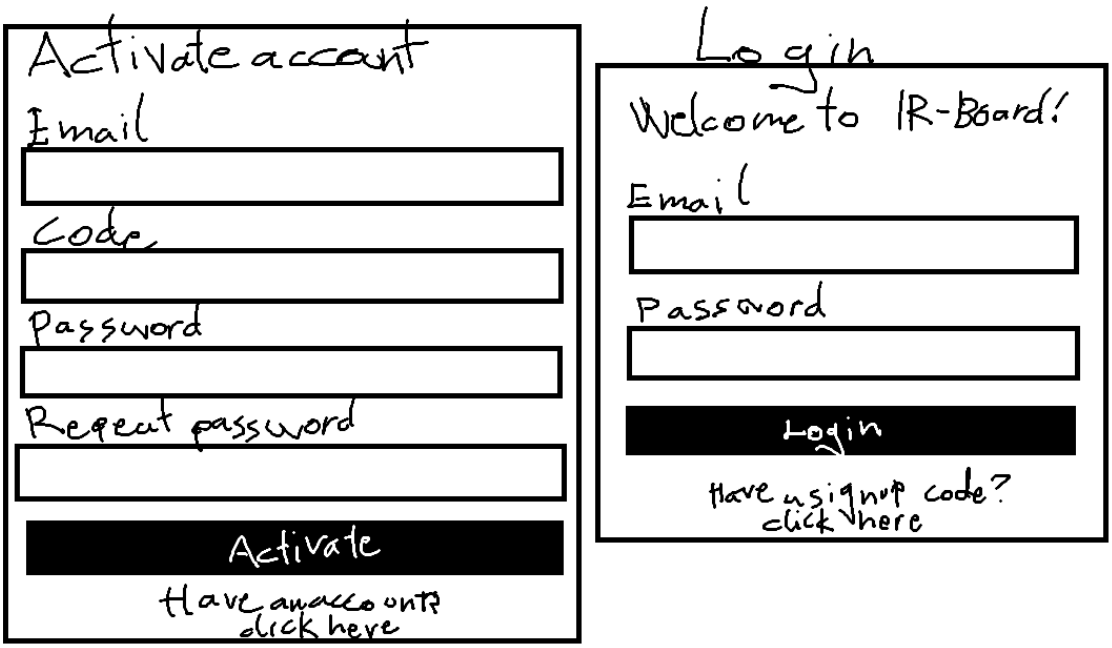

Figure 14: Login and signup page sketch

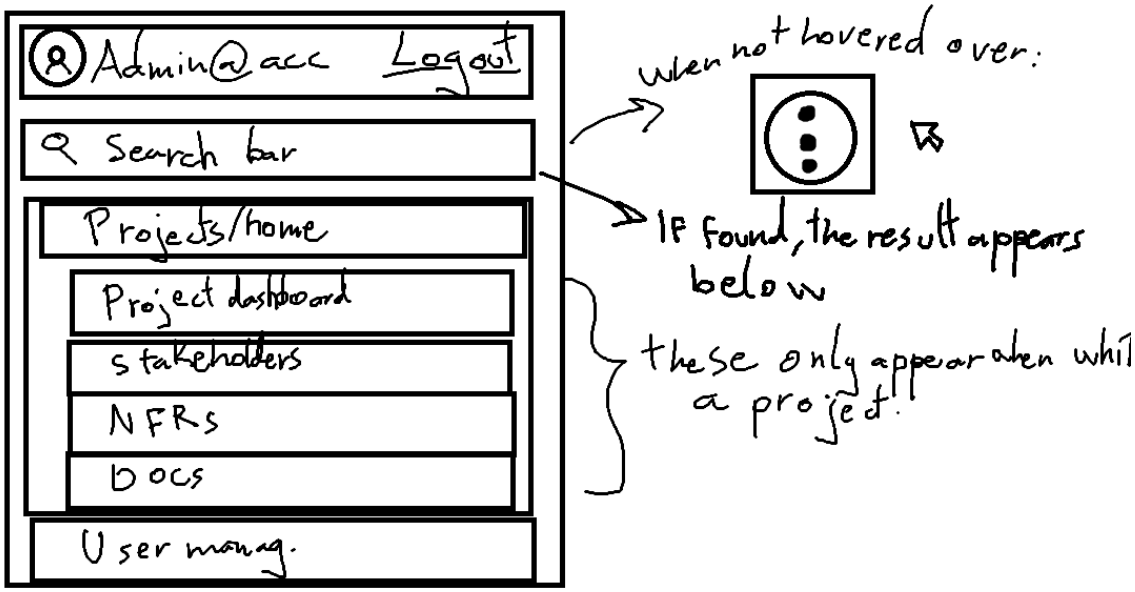

Figure 15: Navigation bar component sketch

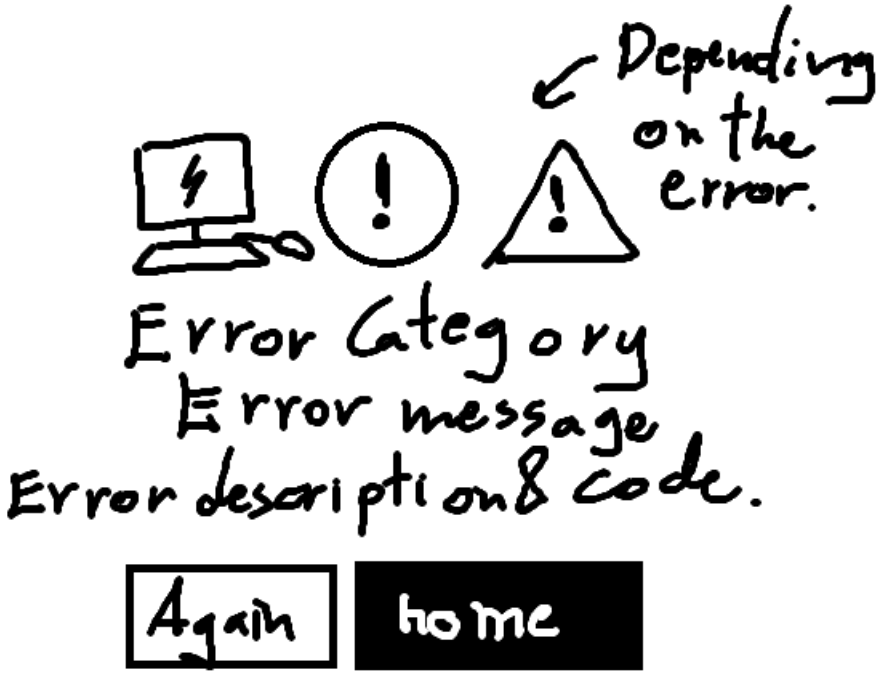

Figure 16: Error page sketch

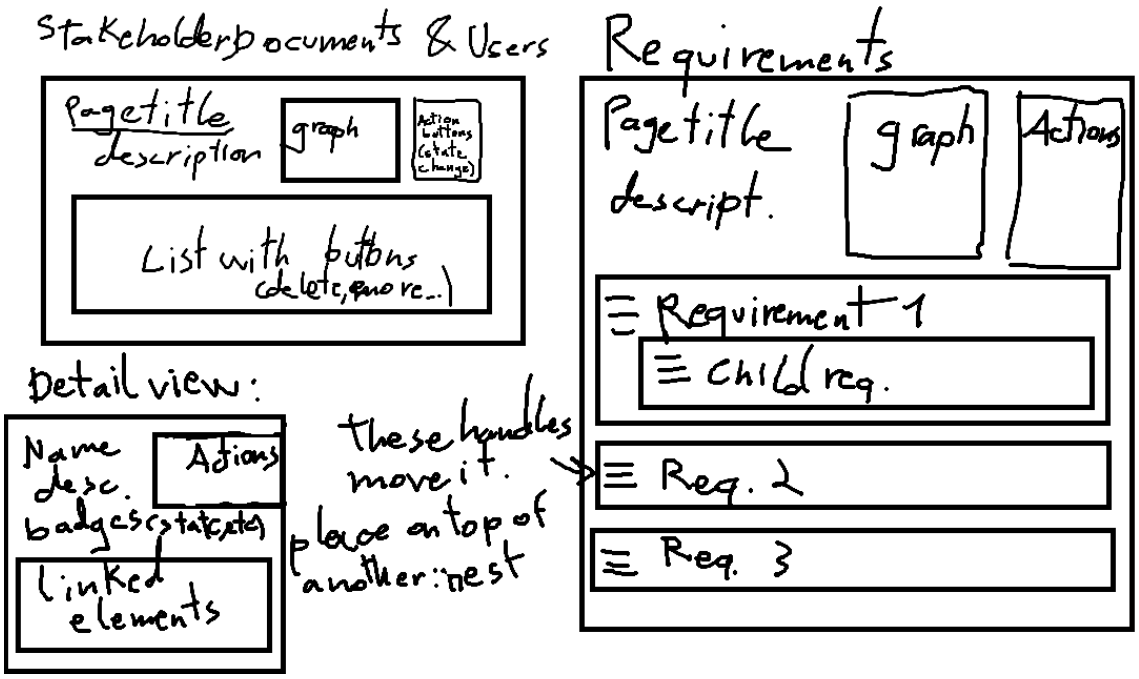

Figure 17: Project element list views sketch

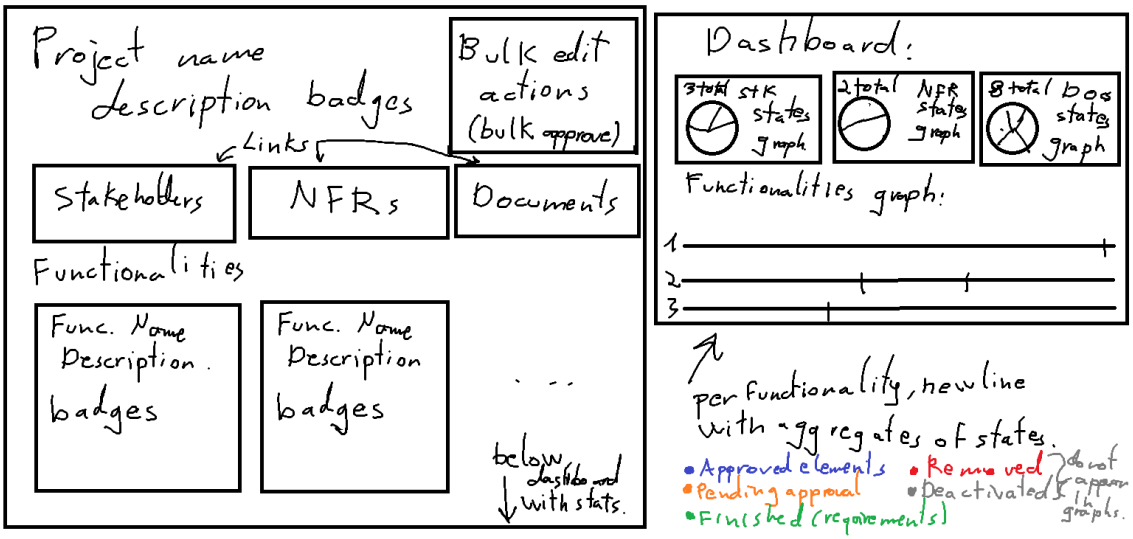

Figure 18: Project view and its dashboard sketch

5.4.4.2 Navegability Diagram

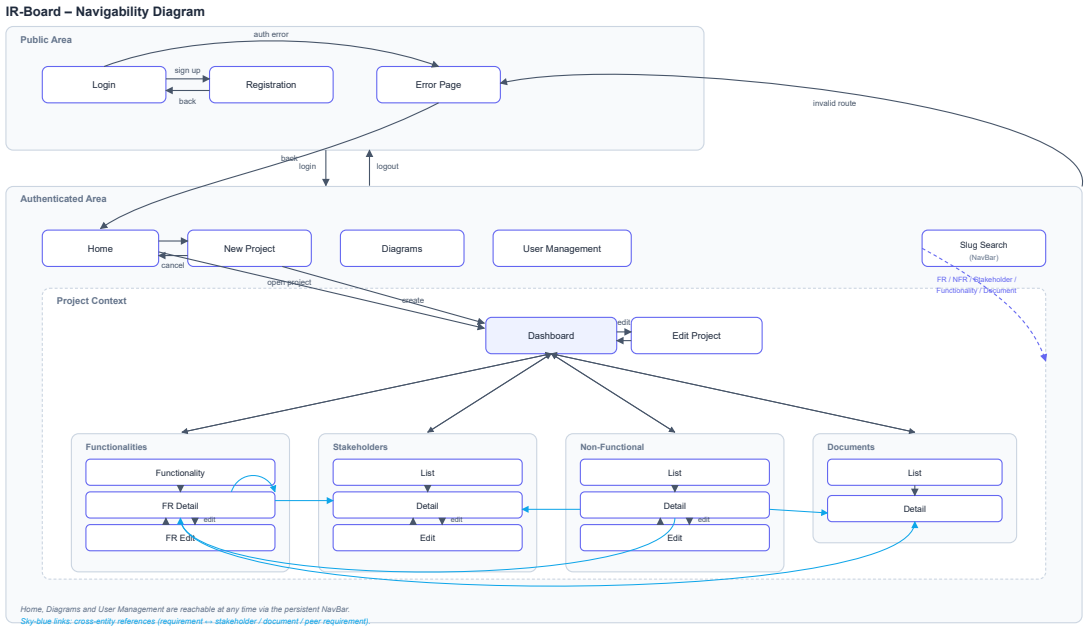

Figure 19: Navigability diagram

The navigability diagram models the application as a state machine, where each state represents a distinct page or view, and each transition represents a user-triggered navigation action. States are grouped into two top-level regions according to the access control boundary enforced by `ProtectedRoute`: a **Public Area**, accessible without an authenticated session, and an **Authenticated Area**, only reachable once a valid session has been established.

The Public Area groups the **Login**, **Registration**, and **Error Page** views. A successful authentication transitions the user into the Authenticated Area, while logging out returns them to the Public Area. Any unrecoverable error, whether an authentication failure or an attempt to access an invalid or unauthorized route, redirects the user to the Error Page, from which they can return to the application's entry point.

Within the Authenticated Area, **Home**, **New Project**, **Diagrams**, and **User Management** are top-level views reachable directly from the persistent NavBar described in the User Interface Description, independently of where the user is currently located in the application; this is noted explicitly in the diagram rather than drawn as individual transitions, to avoid cluttering the layout with redundant arrows. **User Management** is additionally constrained by the `adminOnly` route guard, and is therefore only reachable for users with administrative privileges.

Selecting or creating a project transitions the user into the **Project Context**, a composite region scoped to a single project and corresponding to the `ProjectLockWrapper`, `ProjectProviderWrapper`, and `FunctionalitiesProviderWrapper` route guards described in the Class Design section. Within this context, the **Project Dashboard** acts as the central hub, from which a project manager can reach the **Edit Project** view or navigate, through the project-scoped NavBar links, into any of the project's main entity collections: **Functionalities**, **Stakeholders**, **Non-Functional Requirements**, and **Documents**. Each of these collections follows the same

internal pattern, modelled as a nested composite state: a list view transitions into a detail view upon selecting an entity, and the detail view transitions into an edit view when the user requests a modification, returning to the detail view once the change is saved or cancelled.

A dedicated **Slug Search** feature, also part of the persistent NavBar, is modelled as a shortcut transition that bypasses the regular hierarchical navigation entirely: given a valid entity slug, it allows the user to jump directly into the corresponding functional requirement, non-functional requirement, stakeholder, functionality, or document detail view from anywhere within the Authenticated Area, reflecting the identifier-based traceability mechanism discussed in the Theoretical Background.

Finally, the sky-blue links connecting the **Functional Requirement Detail**, **Non-Functional Requirement Detail**, **Stakeholder Detail**, and **Document Detail** views represent the horizontal traceability relationships described in the Traceability section: from a requirement's detail view, a user may navigate directly to a linked stakeholder, a linked document, or a related peer requirement, including other functional requirements observed by the same requirement, modelled here as a self-transition. Unlike the structural transitions described above, these links do not follow the entity hierarchy; they instead reflect the observer relationships maintained at the data layer, as illustrated earlier in the Process Modeling section's observation flow diagram.

5.5 Requirements Specification

5.5.1 Functional Requirements

5.5.1.1 project management

- PM.1. The system must allow an admin to create a project.
  - PM.1.1. The system must require a Project name to generate a project correctly.
  - PM.1.2. The system must require a description to generate a project correctly.
  - PM.1.3. The system must require a user to be an admin of the system to generate a project correctly.
  - PM.1.4. The system must allow to optionally change the priority style set for the project, either:
    - PM.1.4.1. Ternary (High, Medium, Low) Predefined.
    - PM.1.4.2. MOSCOW (Must, Should, Could or Won't have).
- PM.2. The system must allow an admin to deactivate a project.
  - PM.2.1. The system must ask for confirmation before deactivating.
  - PM.2.2. The system must put the project on read only mode.
- PM.3. The system must allow an admin to reactivate a deactivated project.
- PM.4. The system must allow an admin to modify an active project.
- PM.5. The system must allow to link users with a project.
  - PM.5.1. The system must allow an admin to link users to a project as project manager.

PM.5.2. The system must allow an admin or project manager linked to the project to link users to a functionality on said project as a stakeholder user.
PM.5.3. The system must allow a project manager to link users to one or more functionalities of a project as requirement engineers.
PM.6. The system must allow access to the project description/dashboard to users linked to it or a functionality of it.
PM.6.1. The system must show the total split of requirements by their states (pie chart).
PM.6.2. The system must not take into account deactivated requirements toward any metric.
PM.6.3. The system must show the different functionalities of the project.
PM.7. The system must allow a project manager to mark as approved all elements in a project.
PM.8. The system must allow a project manager to add a functionality to a project.
PM.8.1. A functionality needs a name and unique set of letters for its dynamic identifier.
PM.8.2. The system must automatically attempt to get the letters for the dynamic identifier from the name.
PM.8.2.1. The system must take the first letter from every word in the name.
PM.8.3. The system must deny adding any functionality with an identifier matching another on the same project.
PM.8.4. The system must allow to use a custom functionality identifier.
PM.8.5. The system must automatically link the project manager to the new functionality.
PM.9. The system must allow a project manager to modify a functionality.
PM.10. The system must allow a project manager to deactivate a functionality.
PM.10.1. The system must ask for confirmation before deactivating.
PM.10.2. The system must put the functionality (elements contained by it) on read only.
PM.11. The system must allow a project manager to reactivate a functionality.
PM.12. The system must allow a project manager to mark as approved all elements in a functionality.

5.5.1.2 Stakeholders management

SM.1. The system must allow any user linked with the project access to view its stakeholders.
SM.1.1. The system must show stakeholders that are not removed.
SM.1.2. The system must show if a stakeholder is flagged as pending review.
SM.1.3. The system must show the identifier, the name and part of the description.
SM.1.4. The system must allow to view the details of a stakeholder.
SM.1.4.1. The system must show all atributes of a stakeholder.
SM.1.4.2. The system must show all requirements linked to it.
SM.2. The system must allow a project manager to view its removed stakeholders.

SM.3. The system must allow a requirement engineer or a project manager to add a new stakeholder to a project.
- SM.3.1. The system must only allow a stakeholder to be added to a project the user is linked to.
- SM.3.2. The system must require a name to generate a stakeholder.
- SM.3.3. The system must require a description to generate a stakeholder.
- SM.3.4. The system must generate the identifier for the stakeholder.

SM.4. The system must allow a project manager or requirement engineer to link a stakeholder to one or more requirements on the same project.
- SM.4.1. The system must only allow the user to link the stakeholder to a requirement on a functionality they are linked to.

SM.5. The system must allow a project manager or requirement engineer to unlink a stakeholder from one or more requirements.
- SM.5.1. The system must only allow the user to unlink a stakeholder from a requirement of a functionality they are linked to.

SM.6. The system must allow a requirement engineer or a project manager to deactivate a stakeholder from a project the user is linked to.
- SM.6.1. The system must flag all entities linked as pending review.
- SM.6.2. The system must put the stakeholder on read only mode.

SM.7. The system must allow a requirement engineer or a project manager to enable a disabled stakeholder from a project the user is linked to.
- SM.7.1. The system must flag all entities linked as pending review.
- SM.7.2. The system must put the stakeholder on read only mode.

SM.8. The system must allow a project manager to remove a disabled stakeholder from a project the user is linked to.
- SM.8.1. The system must flag all entities linked as pending review.
- SM.8.2. The system must unlink any requirements linked to the stakeholder

SM.9. The system must allow a project manager to delete permanently a removed stakeholder.

SM.10. The system must allow a project manager or requirement engineer to modify a stakeholder that's not deactivated or removed.
- SM.10.1. The system must flag as pending review linked entities upon saving.

5.5.1.3 Requirement management

RM.1. The system must allow users linked to a project or functionality of a project access to its requirements.
- RM.1.1. The system must only allow users linked to the functionality of a functional requirement access to it.
- RM.1.2. The system must allow to collapse and expand requirements with children.
- RM.1.3. The system must show the dynamic identifier, the name, state and part of the description.
- RM.1.4. The system must allow to view the details of a requirement.
  - RM.1.4.1. The system must show the internal unique identifier.
  - RM.1.4.2. The system must show all atributes of a requirement.
  - RM.1.4.3. The system must show all stakeholders linked to it.

RM.1.4.4. The system must show all requirements cross-linked with it.
RM.1.4.5. The system must show all documents linked to it.

RM.2. The system must allow a requirement engineer or a project manager to add a requirement to a project the user is linked to.
RM.2.1. The system must only allow a functional requirement to be added to a functionality the user is linked to.
RM.2.2. The system must allow the user to generate a requirement as a child of another requirement (nesting).
RM.2.3. The system must assign automatically the dynamic identifier.
RM.2.3.1. The identifier must be based on its relation to other requirements.
RM.2.3.2. The identifier must represent if it is a functional or non functional requirement (FR or NFR).
RM.2.3.3. The identifier must represent the folder/component that holds the requirement (user management → UM).
RM.2.4. The system must assign automatically an internal unique slug.
RM.2.4.1. The identifier must represent the project that will hold the requirement.
RM.2.4.2. The identifier must represent whether the requirement is functional or non functional.
RM.2.4.3. The identifier must have a random element to ensure a low colision rate.
RM.2.5. The system must ask for the following data for a functional requirement:
RM.2.5.1. The system must require a name.
RM.2.5.2. The system must require a description.
RM.2.5.3. The system must require a priority following the one set on the project creation.
RM.2.5.4. The system must allow to set a stability, which is optional.
RM.2.5.5. The system must allow to set a stakeholder as origin, which is optional.
RM.2.6. The system must ask for the following data for a non-functional requirement:
RM.2.6.1. The system must require a name.
RM.2.6.2. The system must require a description.
RM.2.6.3. The system must allow to set a measurement unit.
RM.2.6.4. The system must allow to set a comparison operator.
RM.2.6.4.1. equal to, less than or greater than.
RM.2.6.5. The system must allow to set a threshold value.
RM.2.6.5.1. This value represents the minimum value to mark the requirement as passed.
RM.2.6.6. The system must allow to set a target value.
RM.2.6.6.1. This value represents the optimal value desired by the team.

RM.2.6.7. The system must allow to set an actual value.
RM.2.6.7.1. This value represents the current status of the measurement.
RM.2.7. The system must automatically set the new requirement as pending approval.
RM.3. The system must allow a project manager or requirement engineer to link a requirement on a functionality they are linked to, to another entity.
RM.3.1. The system must allow to link a requirement with a stakeholder of the same project.
RM.3.2. The system must allow to un-link a requirement with a stakeholder.
RM.3.3. The system must allow to link a requirement with one or more requirements of functionalities of the same project the user is linked to.
RM.3.4. The system must allow to un-link a requirement with other requirements of functionalities of the same project the user is linked to.
RM.3.5. The system must allow to link a requirement with one or more documents of the same project.
RM.3.6. The system must allow to un-link a requirement with one or more documents of the same project.
RM.4. The system must allow a requirement engineer or a project manager to deactivate a requirement pending approval on a functionality they are linked to.
RM.4.1. The system must flag any requirements linked to the deactivated requirement as pending review.
RM.4.2. The system must put the requirement on read only.
RM.5. The system must allow a project manager or requirement engineer to set a deactivated requirement as removed.
RM.5.1. The system must hide from view a removed requirement, effectively archiving it.
RM.6. The system must allow a project manager to permanently delete a removed requirement.
RM.6.1. The system must hide from view a removed requirement, effectively archiving it.
RM.7. The system must allow a requirement engineer or a project manager to reactivate a requirement on a functionality they are linked to.
RM.7.1. The system must automatically flag as pending review the reactivated requirement.
RM.8. The system must allow a requirement engineer or a project manager to modify a requirement on a project.
RM.8.1. The system must only allow a project manager or requirement engineer to modify functional requiremets of a functionality the user is linked to.
RM.8.2. The system must flag linked requirements as pending review upon saving with changes.
RM.9. The system must allow a project manager to mark as approved one or more requirements.

RM.9.1. The system must only allow to mark as approved a requirement that is pending approval, not pending review nor deactivated.

RM.10. The system must allow a project manager to mark as finished a requirement.

RM.10.1. The system must set the requirement as pending approval if it is modified.

RM.11. The system must allow a project manager or requirement engineer linked to a functionality of the project to change the position of a requirement.

RM.11.1. The system must allow reordering of functional requirements to users linked to the same functionality.

RM.11.2. The system must update the dynamic identifier automatically.

RM.11.3. The system must set the order of requirements using a floating point order value.

5.5.1.4 User management

UM.1. The system must allow an admin to invite new users to the system.

UM.1.1. The system must provide different levels of authorisation, based on the relationship between its elements.

UM.1.1.1. The system must have the levels: Admin, project manager, requirement engineer and stakeholder user.

UM.1.2. The system must ask the admin to set the name, surname, and email of the invited user.

UM.1.2.1. The system must generate an signup code as a temporal password.

UM.1.2.2. The system must automatically send an invitation with the signup code to the email of the invited user.

UM.2. The system must allow an admin to view the name and surname of a user from the system.

UM.3. The system must allow an admin to modify the name and surname of a user from the system.

UM.4. The system must allow an admin to view the current permissions of a user from the system.

UM.5. The system must allow an admin to generate a new invite with a signup code for a user.

UM.6. The system must allow any user with valid credentials to sign in to the system.

UM.6.1. The system must prompt any user signing in with a signup code to set a permanent password.

UM.6.1.1. The system must ensure the password is between 15 and 64 characters long.

UM.6.1.2. The system must make use of a random salt specific of each user.

UM.6.1.3. The system must remove any password or signup code of the user upon seting a permanent password.

UM.6.2. The system must temporally block the user after 3 consecutive failed attempts.

UM.7. The system must allow an admin to remove a user from the system.

UM.7.1. The system must clean the removed user's ReBAC permissions.

5.5.1.5 Document management and modelling

DMM.1. The system must allow users linked to a project access to documents of that project.
- DMM.1.1. The system must show documents not removed.
- DMM.1.2. The system must show entities linked to the document.

DMM.2. The system must allow a project manager to access removed documents of that project.
- DMM.2.1. The system must show entities linked to the document.

DMM.3. The system must allow a project manager or a requirement engineer to add document to a project.
- DMM.3.1. The user must be linked to the project.

DMM.4. The system must allow a document to be linked to one or more requirements of the same project.
- DMM.4.1. The system must flag those requirements linked to it as pending a review if the document is altered.

DMM.5. The system must allow a project manager or requirement engineer to update a document.
- DMM.5.1. The user must be linked to the project the document is on.
- DMM.5.2. The system must flag as pending a review any requirements linked to the document.

DMM.6. The system must allow a project manager or requirement engineer to disable a document.
- DMM.6.1. The system must flag as pending a review any requirements linked to the document.

DMM.7. The system must allow a project manager or requirement engineer to enable a disabled document.
- DMM.7.1. The system must flag as pending a review any requirements linked to the document.

DMM.8. The system must allow a project manager or requirement engineer to remove a disabled document.
- DMM.8.1. The system must flag as pending a review any requirements linked to the document.
- DMM.8.2. The system must unlink any requirements linked to the document.

DMM.9. The system must allow a project manager to delete permanently a removed document.

5.5.1.6 Concurrency

C.1. The system must block other users from modifying an entity that another user is already modifying.
- C.1.1. The system must release automatically the entity if the user modifying it saves and exits (stops modifying).
- C.1.2. The system must release automatically the entity after a predetermined timeout period.
- C.1.3. The system must release automatically the entity if the user editing it modifies another entity.

C.1.4. The system must only accept changes to the entity from the user who holds the entity.

C.2. The system must display for other users who is modifying the entity.

5.5.1.7 Search and filtering

SF.1. The system must allow searching a project entity by its internal unique identifier.

SF.1.1. The system must search lexically.

SF.1.2. The system must allow the user to see the details of the found entity.

SF.1.2.1. only if an exact match occurs,

SF.1.2.2. only if the user has access to it.

SF.2. The system must allow users to filter entities they have access to.

SF.2.1. The system must allow filtering out deactivated requirements.

SF.2.2. The system must allow filtering requirements based on priority.

SF.2.3. The system must allow filtering requirements based on state.

SF.2.4. Any filter must be reversible; ascending or descending order.

5.5.2 Usability Requirements

UR.1. The system must be usable by professionals with software development or requirements engineering backgrounds without prior training on the platform.

UR.1.1. Usability testing must be performed to ensure any issues identified during use are addressed.

UR.2. All destructive or irreversible actions must require explicit confirmation before execution.

UR.3. State labels and lifecycle transitions must be clearly communicated to users regardless of their role within a project.

5.5.3 Performance Requirements

TODO check values with load testing

PR.1. The system must remain responsive under the concurrent usage conditions expected of a small software development team of up to 20 simultaneous authenticated users.

PR.1.1. Under this load, the p95 response latency for any API request must not exceed 500ms.

PR.1.2. Under this load, the p95 response latency for the slug-based search endpoint must not exceed 300ms.

PR.2. Entity locking timeouts must not exceed 1 hour per single lock before it is automatically released, to avoid blocking collaborative workflows for extended periods.

PR.3. The above thresholds are to be validated during load testing as described in the Test Plan Analysis.

5.5.4 Logical Database Requirements

DBR.1. All primary entities must be uniquely identifiable through an internal numeric identifier managed by the database.

DBR.2. Entities that support external reference must additionally carry a unique semantic identifier called entity slug, and in the case of requirements, a human-readable dynamic identifier as well.
DBR.3. Lifecycle states must be stored as controlled enumeration values, enforced at the application layer before persistence.
DBR.4. Traceability relationships between requirements, stakeholders, and documents must be represented as explicit join tables rather than embedded references, to support bidirectional traversal and impact analysis.
DBR.5. No entity may be permanently deleted without passing through the defined deactivation and removal states first.
DBR.5.1. In case of parent removal, the appropiate states must have been traversed at least on the parent.

5.5.5 Design Constraints

DC.1. The system must be deployable as a containerized stack using Docker Compose, without requiring cloud-specific infrastructure.
DC.2. The security architecture must follow Zero-Trust principles, delegating identity management, session handling, and authorization enforcement to the Ory ecosystem rather than implementing these concerns within the business application.
DC.3. The frontend must communicate with backend services exclusively through the API gateway; no direct internal service access is permitted from the client.
DC.4. The backend must expose a RESTful API and must not embed frontend concerns.

5.5.6 System Attributes

SA. Security:
SA.1. All requests to protected resources must pass through Oathkeeper for session validation and permission verification before reaching internal services.
SA.2. Passwords must be salted per user and must meet the length requirements defined in the user management requirements.
SA.3. The system must apply rate limiting on authentication endpoints to mitigate brute-force attacks.
SB. Maintainability:
SB.1. The codebase must meet the minimum coverage threshold enforced by SonarQube, that is, >80%.
SB.2. The codebase must not accumulate critical code smells or reliability issues as defined by the configured quality gate.
SC. Traceability:
SC.1. Every requirement must carry a stable, unique identifier throughout its lifecycle.
SC.2. Changes to linked entities must propagate review flags to observing requirements automatically.
SD. Extensibility:

SD.1. Infrastructure components such as the identity provider, authorization server, mail server, and object storage backend must be replaceable without requiring changes to the core business logic, as they are accessed through abstraction layers rather than direct dependencies.

5.5.7 Supporting Information

The system is intended to operate in a containerized environment during both development and demonstration. The deployment configuration is defined in the Docker Compose file described in the Alternatives Analysis and Scope sections. All development-time components, including the local mail server and object storage, are replaceable with production-grade alternatives when deploying outside a development context, as noted in the assumptions and constraints. The observability stack provides operational visibility through Grafana, Loki, and Prometheus, as described in the System Architecture section.

5.6 Test Plan Analysis

To ensure the reliability, maintainability, and performance of the IR-Board system, a multi-dimensional testing strategy has been defined. This plan covers the entire development lifecycle, from code quality to system behavior under stress.

5.6.1 Code maintainability and unit testing with SonarQube

The project utilizes SonarQube as a Static Application Security Testing (SAST) tool. This analysis is integrated into the development workflow to ensure the following:

- Maintainability: Identification of "code smells" and technical debt that could hinder future scalability.
- Code Coverage Validation: Monitoring the percentage of the source code executed during automated tests. This ensures that critical business logic is thoroughly verified, maintaining a high safety net against regressions and establishing a minimum threshold of tested code before deployment.
- Reliability: Detection of potential bugs and logic errors through automated pattern matching.
- Security: Scanning for common vulnerabilities and ensuring compliance with industry standards (e.g., OWASP Top 10).

5.6.2 Integration and acceptance tests

Integration testing validates that independently developed components behave correctly when combined, targeting the boundaries between layers and services rather than the internal correctness of individual units.

Acceptance testing is treated as a natural extension of integration testing at the boundary between the system and its functional requirements. Each integration test scenario is traceable to one or more functional requirements defined in the Requirements Specification, providing a structured record of which requirements have been verified at the integration level.

5.6.3 End-to-end testing

End-to-end (E2E) testing validates the system as a whole, exercising the full deployed stack from the browser through Traefik, Oathkeeper, Kratos, Keto, the

backend, and the database, exactly as a real user would interact with it. The selected tool for this layer is **Playwright**, which drives a real browser instance and provides reliable cross-browser support, network interception capabilities, and a robust selector model suited to the component structure of the React frontend.

E2E test scenarios are organized around the main user journeys identified in the requirements specification, covering the complete lifecycle of the system's primary entities. Representative scenarios include the full project creation and configuration flow, the creation and management of functional and non-functional requirements, the approval and state transition workflows, the creation and management of stakeholders and documents...

Each scenario is run against a dedicated deployment of the full Docker Compose stack in a clean state, with the initial administrator account seeded by the same `initial-setup-admin` service used in regular deployment. This ensures that E2E tests validate not only the application logic but also the correctness of the infrastructure configuration, including routing rules, CORS policies, authorization rules defined in Keto, and session handling by Kratos.

5.6.4 Load testing

For the load testing phase, the primary focus will be on stressing the critical entry points of the system, specifically the traffic flow passing through Traefik and Oathkeeper toward the Spring Boot backend. The goal is to simulate bursts of concurrent users to identify the exact point where identity validation latency begins to degrade the user experience or if Kratos' session management can handle the expected volume. This process goes beyond checking for server crashes; it involves using the Grafana stack to monitor how container resources scale and ensuring the internal network routing maintains stability under heavy pressure.

5.6.5 Usability testing

Usability testing validates that the system can be used efficiently and without unnecessary friction by the professional profiles who constitute its target audience. Their objective is to identify interaction patterns, navigation structures, or interface elements that cause confusion, hesitation, or error in real users performing realistic tasks.

Test participants are selected from software-related professional profiles, including individuals with experience in software development, requirements engineering, or project management, as these represent the system's primary user groups as defined in the stakeholder list. Participants who have no prior familiarity with IR-Board are preferred, to avoid familiarity bias distorting the results.

Each session follows a task-based protocol: participants are given a set of realistic tasks to complete without assistance, such as creating a project, adding a functionality, writing a functional requirement with a given set of attributes, linking it to a stakeholder, and navigating to that stakeholder's detail view from the requirement. Observers record task completion rates, time-on-task, navigation errors, and points of visible hesitation, while participants are encouraged to verbalize their thought process using a think-aloud protocol.

Results are analyzed to identify recurring friction points and prioritized by frequency and severity of impact. Any findings that point to structural navigability problems are cross-referenced against the navigability diagram, while findings related to form behavior or state visibility are cross-referenced against the functional requirements specification, to determine whether a deficiency represents an implementation gap or a design decision that should be revisited in future work.

Chapter 6. System Design

6.1 System Architecture

6.1.1 General architecture desing

The system architecture follows a Microservices approach based on the Zero Trust security model. This ensures flexibility and scalability while maintaining a high level of isolation between business logic and infrastructure concerns. To guarantee a professional security standard while maintaining a manageable project scope, core identity and access management responsibilities have been delegated to the Ory Open Source ecosystem.

The figure below shows the main flow of the application represented by solid arrows, and secondary messaging between microservices represented by a dotted arrows.

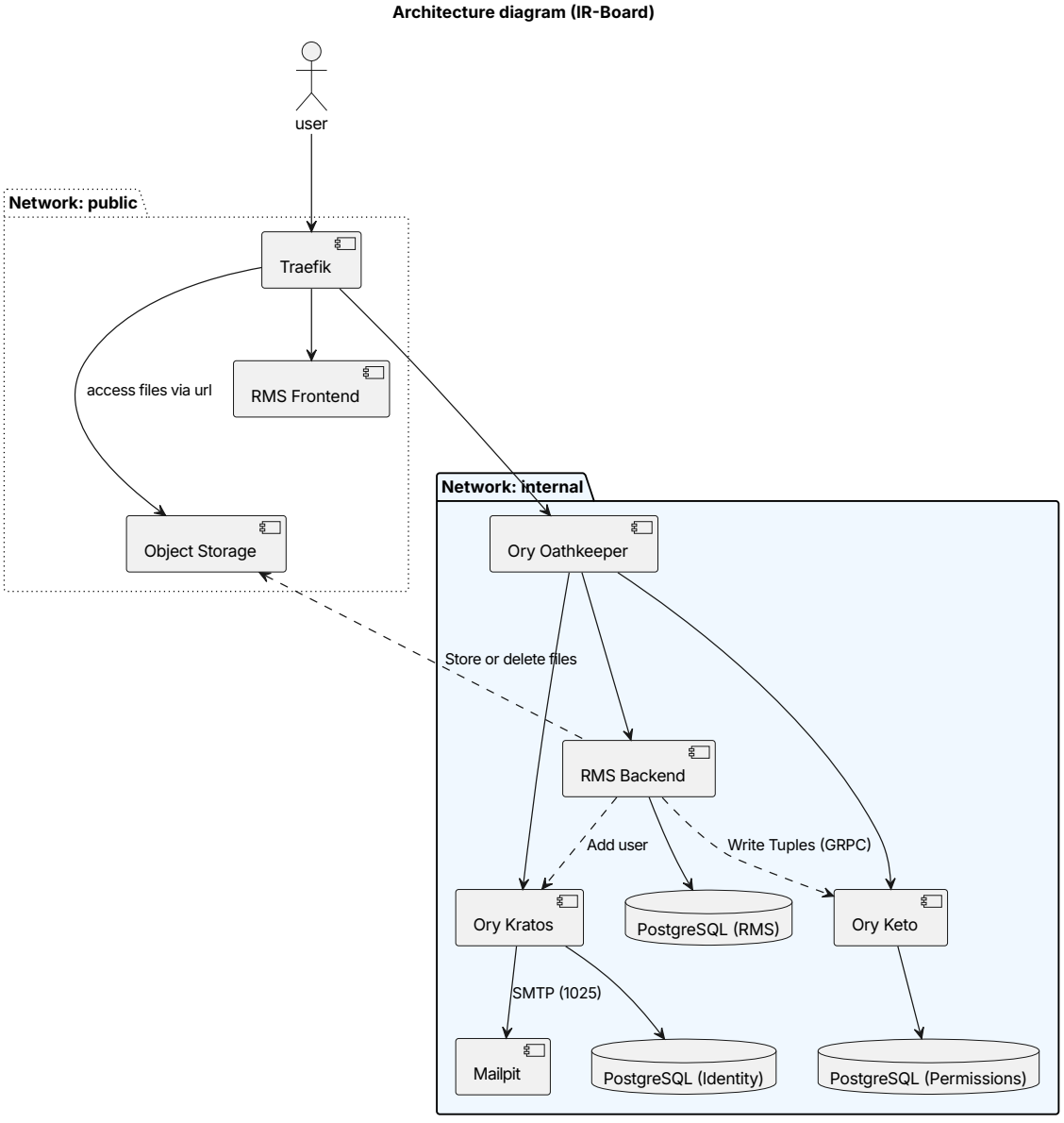

Figure 20: Architecture C2 container diagram

Traefik - Acts as the system's entry point and TLS Termination Proxy. It handles dynamic routing and load balancing, effectively hiding the internal network topology and eliminating the need to expose multiple ports to the public internet.

RMS Frontend - Built with React and TypeScript, served as static content. It executes within the user's browser and communicates with the backend services through the API Gateway.

Ory Oathkeeper - A policy-enforcement engine that acts as a gatekeeper between the public and internal networks. It intercepted every request to validate session integrity (via Kratos) and fine-grained permissions (via Keto) before allowing traffic to reach the internal services.

Ory Keto - A relationship-based access control (ReBAC) server inspired by Google's Zanzibar. It manages permission tuples, allowing the system to verify complex authorization rules (e.g., checking if a user is linked to a specific project).

Ory Kratos - Manages the full identity lifecycle, including user registration, multi-factor authentication, and session management, ensuring that sensitive credentials are handled by a specialized security component.

RMS Backend - The core service developed using Spring Boot, containing the domain-specific business logic and data persistence. It interacts with keto both to write Relation-Based Access Control (ReBAC) tuples and to filter by permissions.

Mailpit - A simple email server that receives all messages sent by Ory Kratos. Acts as a placeholder for development instead of a real email server, to ensure the signup works.

Aditionally, the following containers are present on the deployment:

Draw.io - A self-hosted instance of the draw.io diagramming tool, embedded in the frontend to allow users to create and edit diagrams (flowcharts, use case diagrams, and similar) directly within the platform. It is served on its own subdomain and configured to allow cross-origin embedding from the main application domain.

Grafana - The observability dashboard, accessible on a dedicated subdomain through Traefik. It aggregates logs from Loki and metrics from Prometheus to provide visibility into infrastructure health and application behaviour.

Loki - A log aggregation system operating on the internal network. It receives container logs forwarded by Promtail and exposes them to Grafana for querying.

Promtail - A log collection agent that reads container stdout/stderr logs from the Docker socket and forwards them to Loki. It runs with root privileges solely to access the Docker daemon, a trade-off acknowledged as a hardening concern outside the scope of this project.

Prometheus - A metrics collection server that scrapes operational metrics from instrumented services, making them available to Grafana for time-series visualisation and alerting.

These are not added to the diagram, given their additional nature and non-essential uses, as to ensure clear readability.

6.1.2 Backend system design

The backend follows an architecture that blends **hexagonal architecture** (ports and adapters, as described by Alistair Cockburn [4]) with the explicit layering conventions of **Clean Architecture** [5] (Robert C. Martin). In pure hexagonal architecture the only structural distinction is between the inside of the hexagon (domain and application logic) and the outside (adapters to external systems). Clean Architecture refines this by naming three explicit concentric layers (domain, application, and infrastructure) and formalizing the dependency rule: outer layers depend on inner layers, and the domain at the center has no knowledge of any external technology or framework. The result is a structure that is hexagonal in its port-and-adapter organization and clean-architecture in its explicit package decomposition.

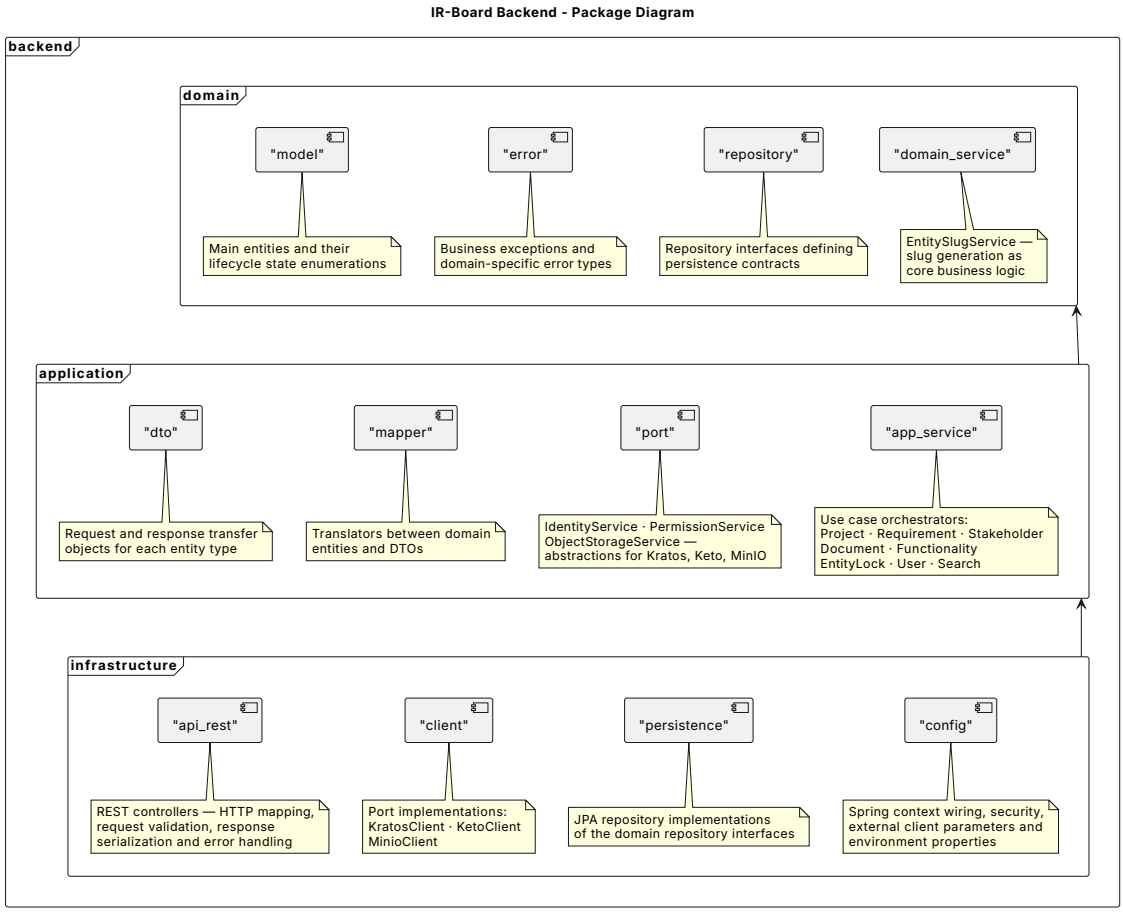

Figure 21: Backend package diagram

The **domain layer** forms the innermost core and contains only pure business logic with no framework dependencies. It defines the main entities and their lifecycle rules, the repository interfaces through which persistence is accessed, the enumerated types and value objects used across the domain, and a single `EntitySlugService` responsible for generating the structured, semantically meaningful slugs assigned to each entity. Slug generation is treated as core business logic rather than an infrastructure concern because the slug format encodes domain knowledge, specifically the project, the entity type, and a collision-resistant random component, making it inseparable from the domain model.

The **application layer** sits above the domain and orchestrates use cases by coordinating domain entities, repositories, and external service ports. It contains the service classes that implement the system's operations, the Data Transfer Objects (DTOs) used to communicate with the outside world, the mappers that translate between domain entities and DTOs, and three port interfaces that represent the external dependencies the application requires: `IdentityService`, `PermissionService`, and `ObjectStorageService`, abstracting Ory Kratos, Ory Keto, and MinIO respectively. These interfaces are the ports in the hexagonal sense: they define what the application needs from the outside world without prescribing how those needs are fulfilled. By keeping the port definitions inside the application layer, the business logic remains fully decoupled from the specific technologies chosen to fulfil each role.

The **infrastructure layer** is the outermost layer and contains all technology-specific implementations, which correspond to the adapters in hexagonal terminology. It is subdivided into four main areas: the REST API controllers that expose the system's endpoints and handle HTTP concerns; the port implementations, namely the Kratos, Keto, and MinIO clients that adapt the external service APIs to the interfaces defined in the application layer; the persistence package, which contains the Spring Data JPA repository implementations of the domain repository interfaces; and the configuration package, which wires together the Spring context, security settings, external client parameters, and environment-specific properties.

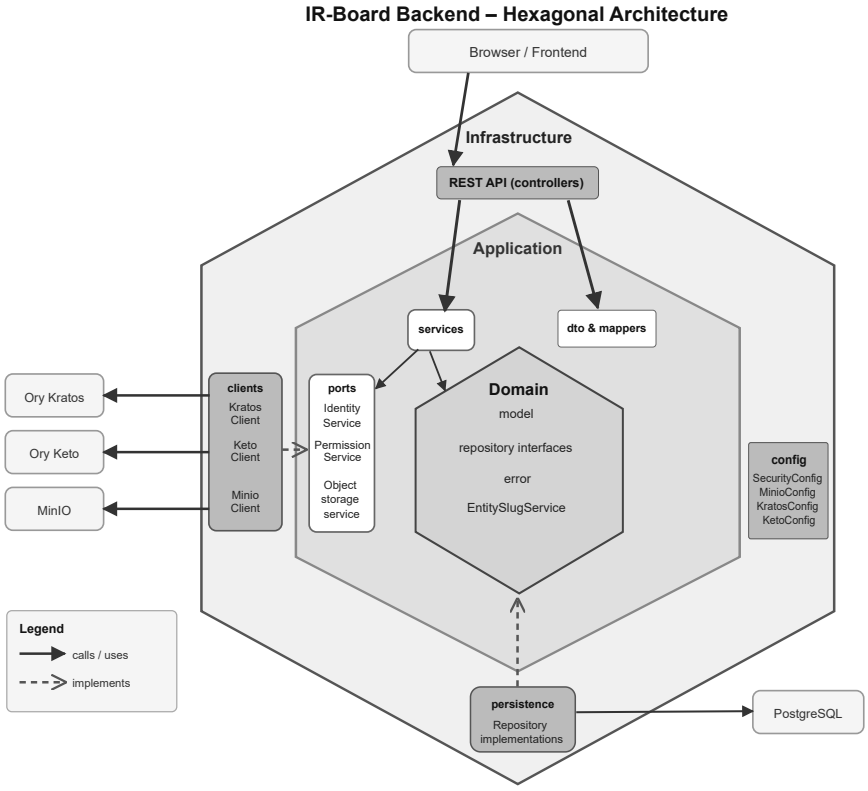

Figure 22: Backend hexagonal architecture diagram

This separation provides several practical benefits for IR-Board. The domain and application layers can be tested in complete isolation from infrastructure concerns, with the Ory ecosystem components replaced by test mocks injected through the port interfaces. The port abstraction also means that any of the three external services can be replaced (for example substituting Kratos with a different identity provider, or MinIO with an alternative S3-compatible store) without touching any business logic. Finally, the clear boundary between the REST layer and the application services ensures that HTTP-level concerns such as request mapping, response serialization, and error translation do not leak into the domain model.

6.1.3 Frontend system design

The frontend is structured as a Single Page Application (SPA) built with React and TypeScript, served as static content and communicating with the backend exclusively through the API gateway. Under this model, the browser loads the application once and subsequent navigation is handled entirely client-side by React Router, which maps URL paths to page components without triggering full page reloads. This approach is well suited to the requirements management domain, where users frequently navigate between deeply nested entities (from project dashboard to functionality to individual requirement and back) and where preserving in-memory state across those transitions reduces unnecessary network requests and improves perceived responsiveness.

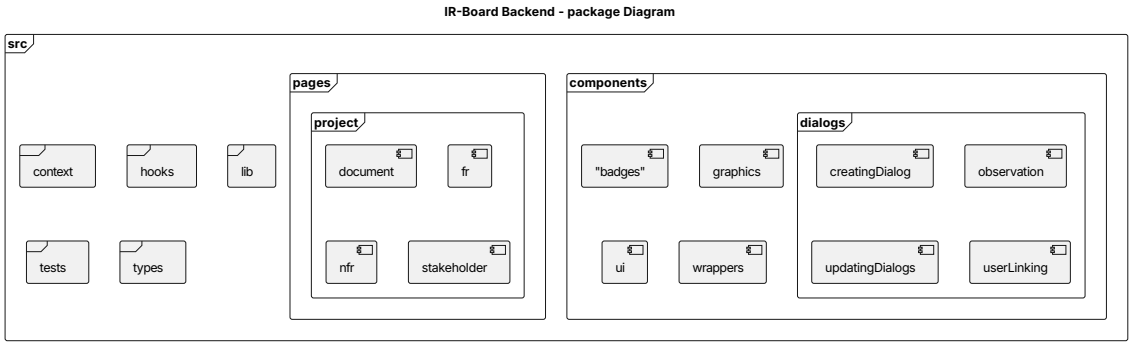

Figure 23: Frontend package diagram

The src directory is organized into a set of purpose-delimited packages that reflect the layered responsibilities of the application. The `pages` folder contains one top-level component per route, each responsible for orchestrating the data-fetching, permission checks, and layout for its view. Shared visual building blocks live under `components`, which is further subdivided by concern: `badges` contains the state and role indicator chips used across entity detail views; `dialogs` groups the modal forms for entity creation, update, observation linking, and user permission assignment; `graphics` holds the pie chart components used by the project and functionality dashboards to visualize requirement state distribution; `ui` contains the unstyled primitives sourced from the shadcn/ui component library; and `wrappers` provides the route guard and context provider components (such as `ProtectedRoute`, `ProjectLockWrapper`, and `FunctionalitiesProviderWrapper`) that enforce authentication and project-scoped state before their child routes are rendered.

Cross-cutting application state is managed through the `contexts` package, which exposes React contexts for the authenticated session and the currently active project and its functionalities, making this information available throughout the component tree without prop drilling. Stateful data-fetching logic is encapsulated in the `hooks` package, where custom hooks abstract the fetch lifecycle, loading and error states, and cache invalidation patterns away from the page components that consume them. The `lib` package groups non-React utilities: global configuration constants, graph utility functions used by the dashboard charts, the Kratos SDK client instance, and general-purpose helper functions shared across the application. Domain types are maintained in a dedicated `types` folder, which contains TypeScript interfaces that mirror the DTOs received from the backend, providing end-to-end type safety from the API response through to the component props without duplicating the backend's domain model. Finally, the `tests` folder mirrors the `src` directory structure exactly, placing each test file adjacent in hierarchy to the module it covers, and adds an `e2e` subfolder at the root of the test tree for the Playwright end-to-end scenarios that exercise the fully deployed stack.

6.2 Real Use Case Design

TODO

6.3 Class Design

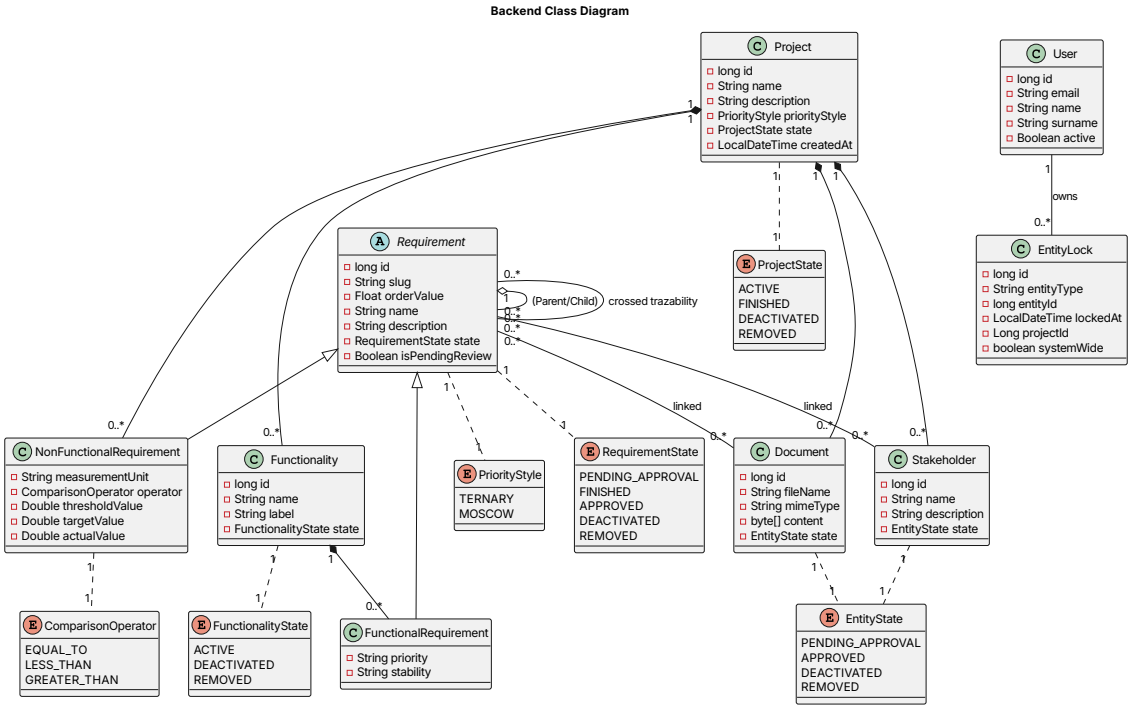

Figure 24: Domain class diagram

User - The relationships between User and Project and Functionality, as they are purely access control related, are delgated to ory Keto or whatever security ReBAC system used. The boolean value isActive is also delegated to the ReBAC system, as it represents a user-to-system relationship.

EntityLock - This class models the concurrent mutex operations of the system. Whenever someone attempts to modify an entity, the entity lock service checks

whether one entity lock object already exists and whether it is expired if it is present. The objects are linked to the user and have information of the entity locked to ensure it can correctly be retrieved and cancelled if the user requests another lock on a different entity.

Project - The base container object, TODO add all remaining classes

6.4 Database Design

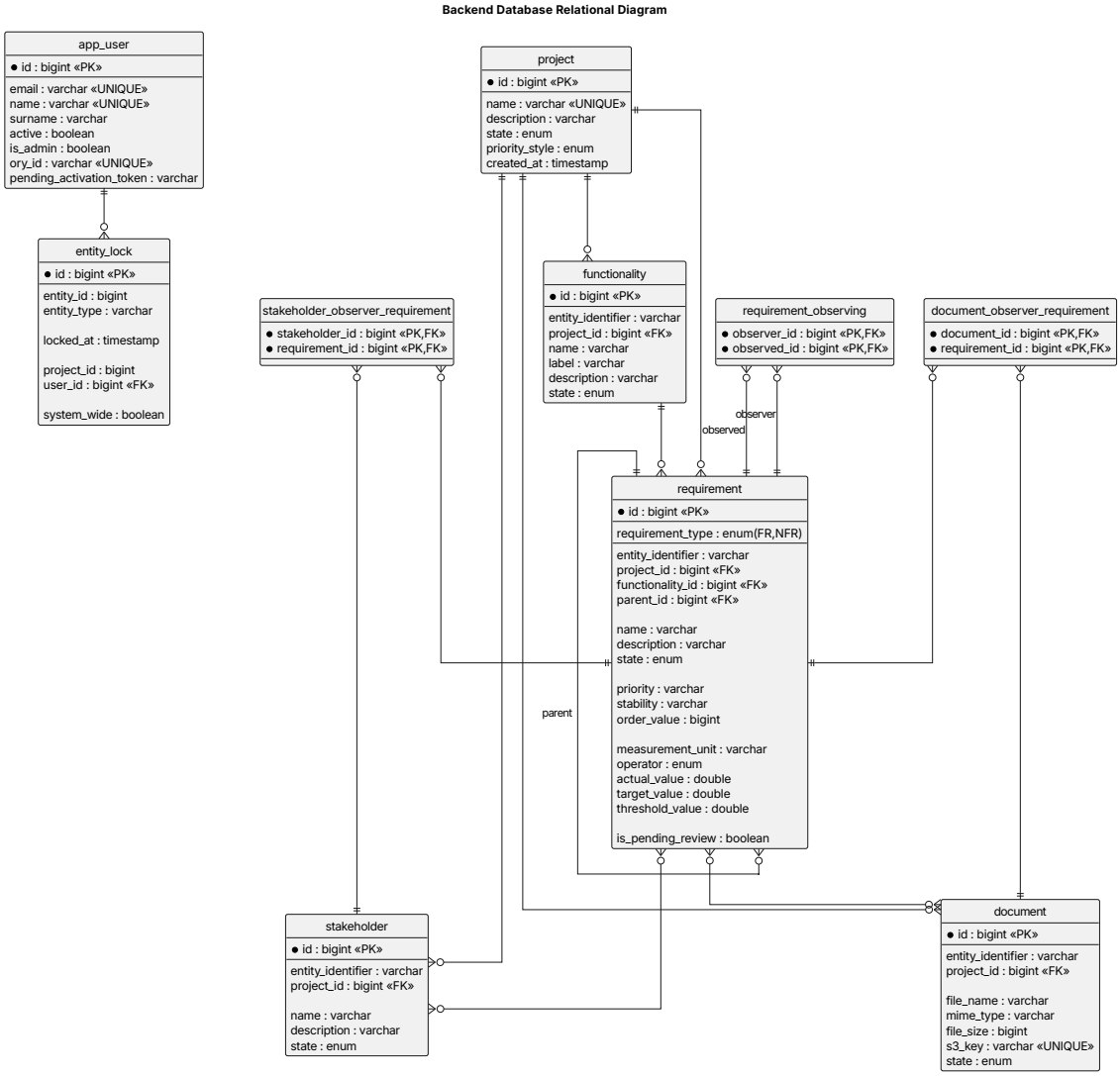

Figure 25: Relational database schema

The relational schema is a direct projection of the domain model onto PostgreSQL, with several design decisions worth highlighting.

Central aggregation around `project`. Every primary entity () `functionality`, `requirement`, `stakeholder`, and `document`) carries a `project_id` foreign key. This makes the project the natural access-control boundary and simplifies bulk operations such as project export or archival.

Flat requirement table with discriminator. Rather than splitting functional and non-functional requirements into separate tables, a single `requirement` table is used with a `requirement_type` enum column acting as a discriminator. Fields exclusive to non-

functional requirements (`measurement_unit`, `operator`, `actual_value`, `target_value`, `threshold_value`) are nullable. This avoids a join on every requirement query while keeping the schema compact.

Self-referential nesting. The `parent_id` column on `requirement` implements hierarchical nesting without a separate closure table. The dynamic identifier and floating-point `order_value` are recalculated at the application layer whenever requirements are reordered, so the database stores only the raw float rather than a sequence integer, avoiding full-table renumbering.

Explicit observer join tables. The many-to-many relationships between requirements and their observers (stakeholders, documents, peer requirements) are represented as three dedicated join tables (`stakeholder_observer_requirement`, `document_observer_requirement`, `requirement_observing`). Keeping these separate makes the observer pattern explicit at the schema level and allows indexed traversal from either side of each relationship.

Type-agnostic entity lock. The `entity_lock` table uses a (`entity_id`, `entity_type`) pair rather than typed foreign keys, so a single table can hold locks on any entity type. A `system_wide` flag distinguishes locks that span the entire system from those scoped to a project.

Deferred integration for access control. The `app_user` table stores only identity and administrative attributes. No schema-level relationship links users to projects or functionalities, because those associations are managed entirely by Ory Keto as ReBAC tuples. The only structural link is through `entity_lock`, where a `user_id` foreign key records who holds each lock.

6.5 User Interface Design

Following validation of the wireframes and navigability diagram presented in the User Interface Definition, the final interface was implemented maintaining the structural decisions established during analysis while refining visual details, interaction patterns, and component behavior based on feedback received during the design phase. The sections below describe each view as implemented, and the screenshots shown represent the final deployed interface rather than preliminary models.

6.5.1 Navigation Bar

The most structurally significant interface element is the persistent navigation bar, present throughout the entire authenticated area. Unlike conventional top-mounted navigation bars, the IR-Board NavBar is implemented as a collapsible panel fixed to the top-left corner of the screen, expanding on hover and collapsing back to a compact icon state when not in use, keeping navigation accessible at all times without consuming permanent screen real estate.

In its collapsed state the NavBar presents only a neutral icon. Upon hovering it expands to show the authenticated user's name and email, a logout button, a slug-based entity search field, and a set of contextual navigation links. When inside a project context, a secondary set of project-scoped links appears beneath the top-level links, providing direct access to the project's stakeholders, non-functional

requirements, and documents. Links that are not yet operational are rendered in a visually disabled state to communicate their planned but incomplete status.

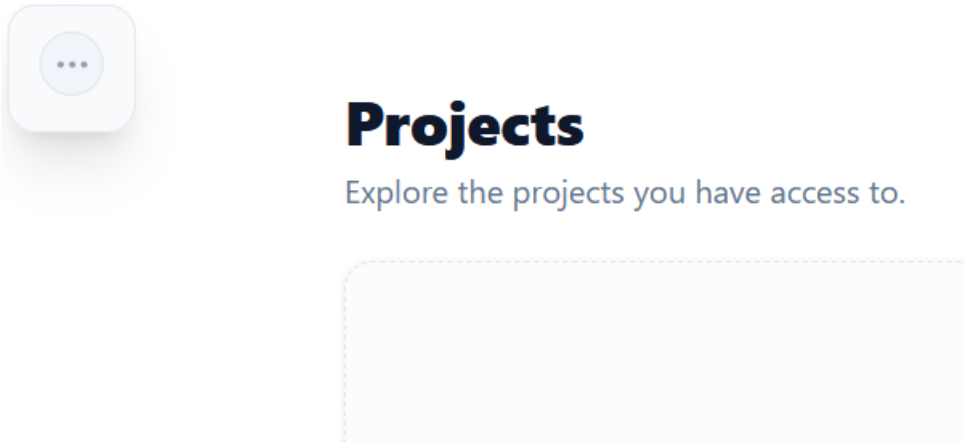

Figure 26: Closed navigation bar

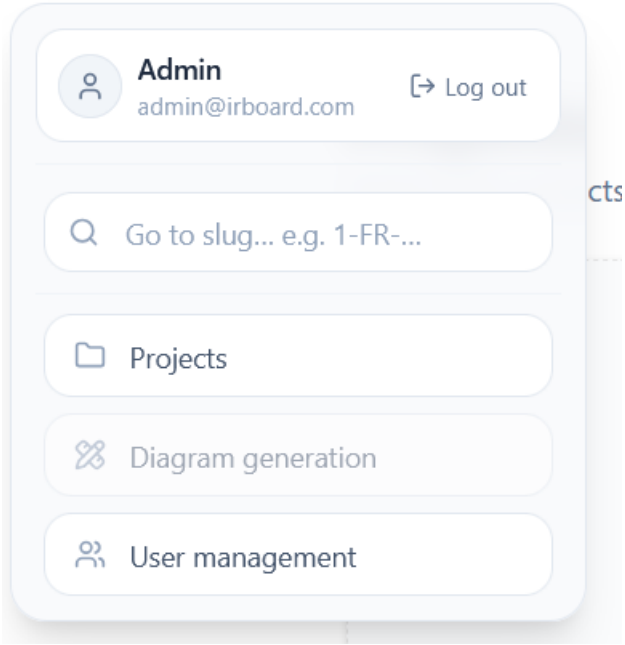

Figure 27: Opened navigation bar

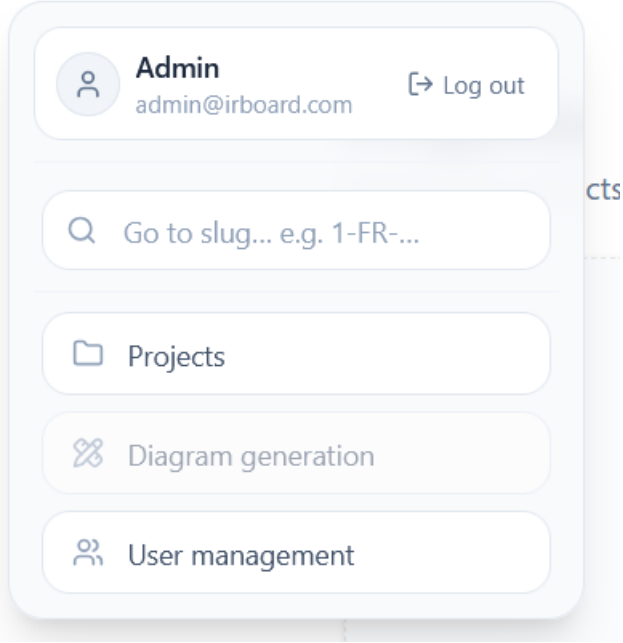

Figure 28: Navigation bar within a project's context

6.5.2 Public Area

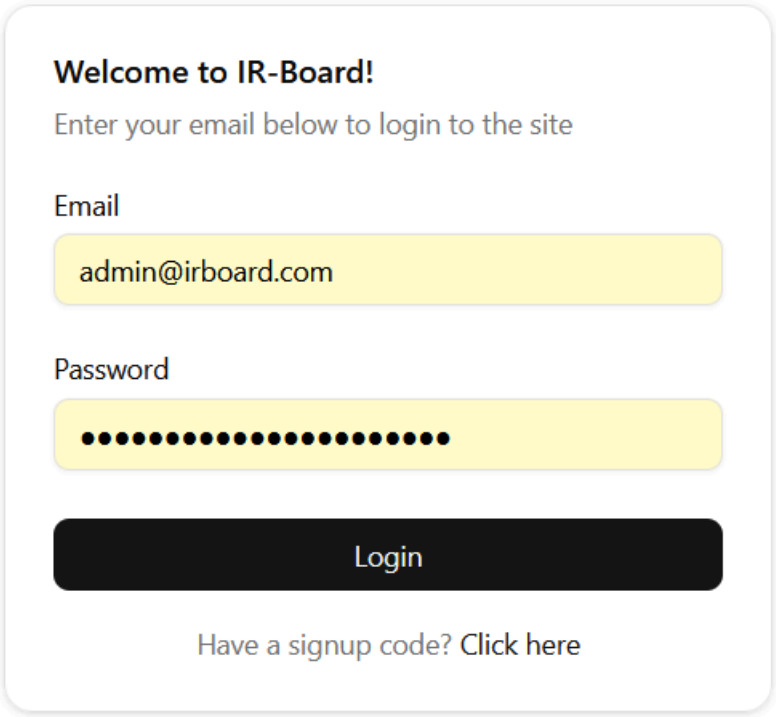

Figure 29: User login page

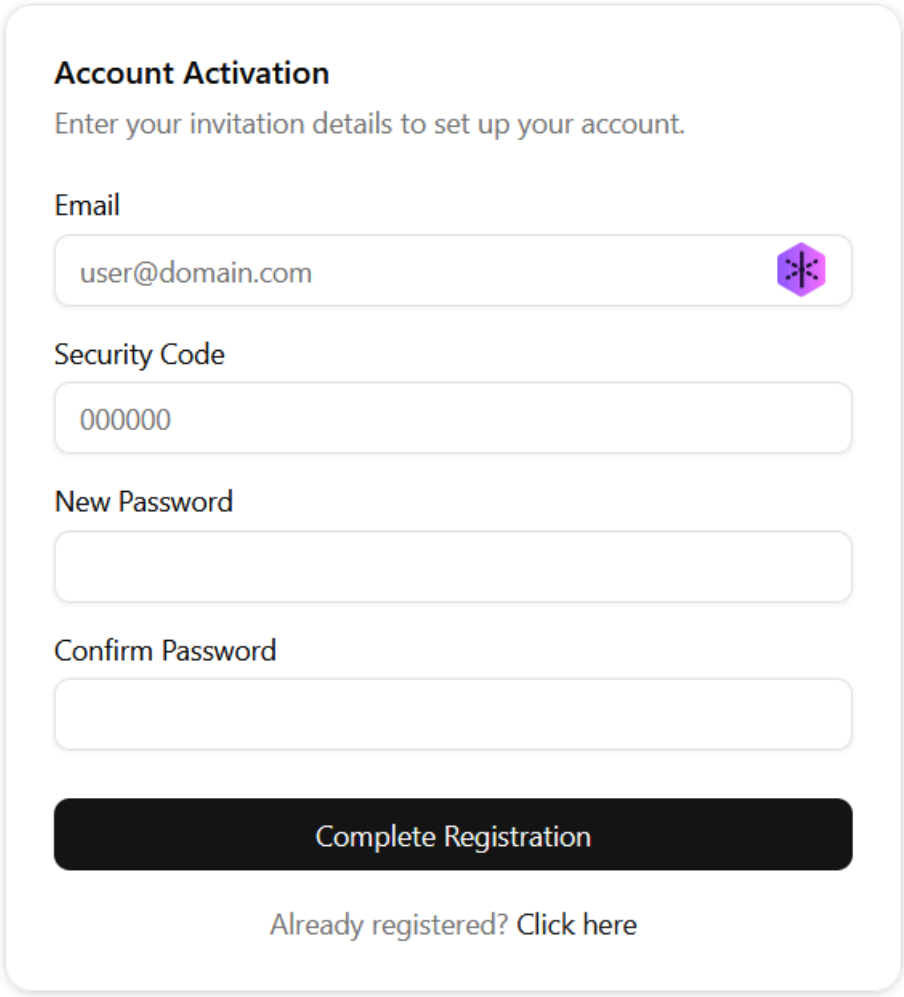

Figure 30: User account activation page

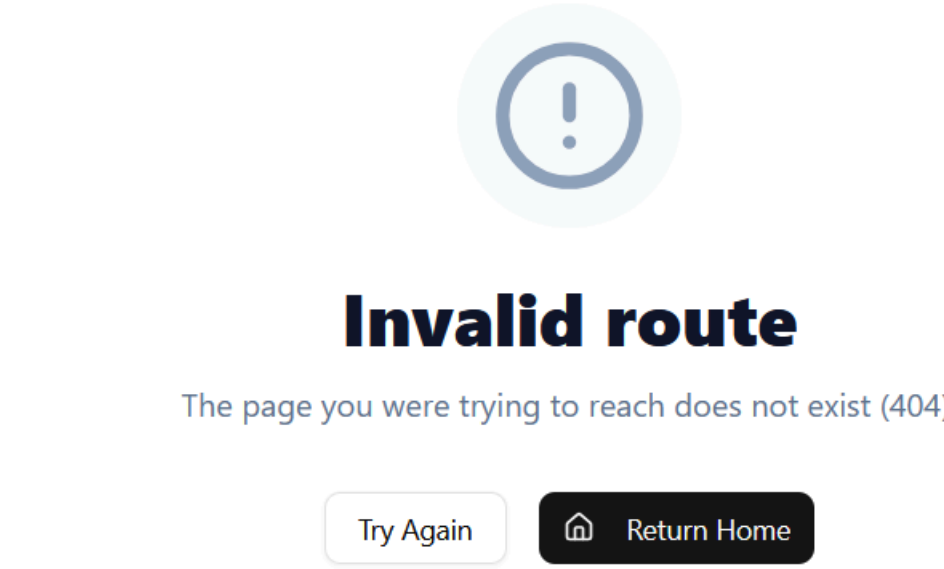

Figure 31: Error page for invalid url

6.5.3 Home View

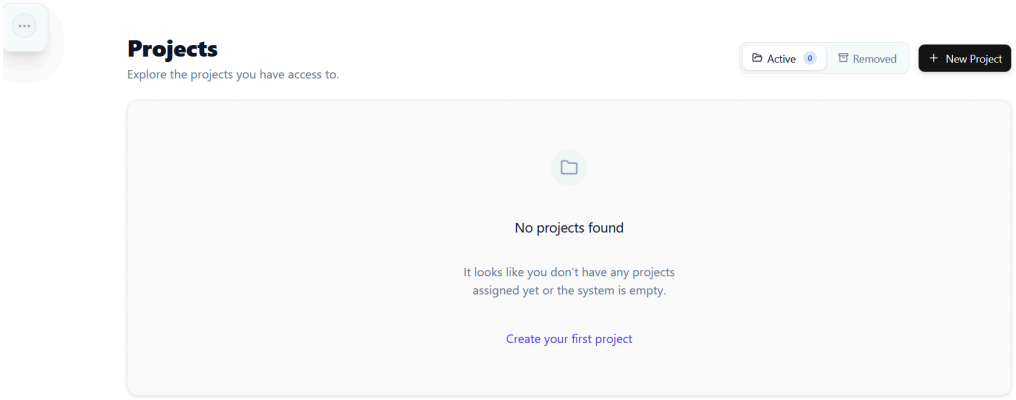

Figure 32: Empty homepage

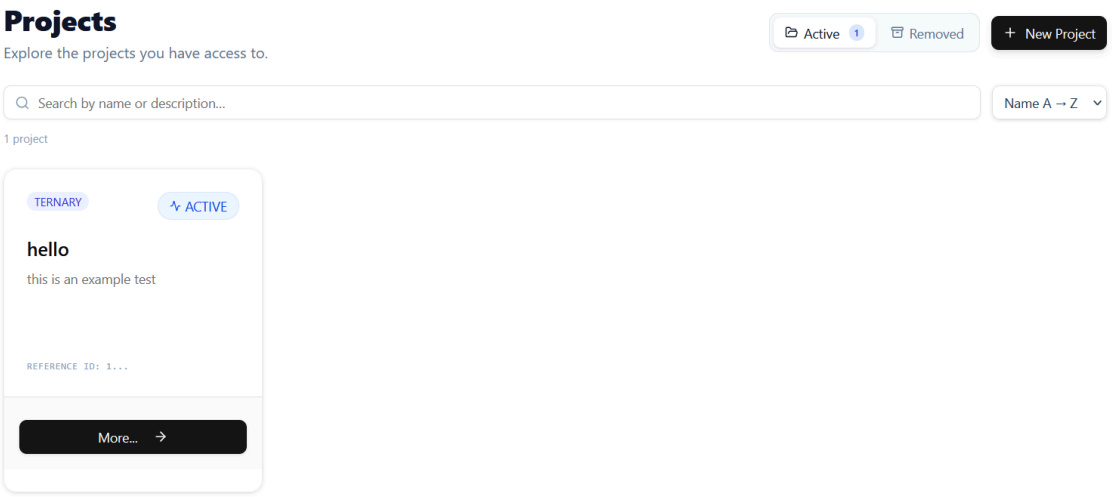

Figure 33: Homepage with a single project

Logically, upon exceeding the maximum amount of projects per page, a basic pagination menu appears on the bottom of the page.

6.5.4 Project Dashboard

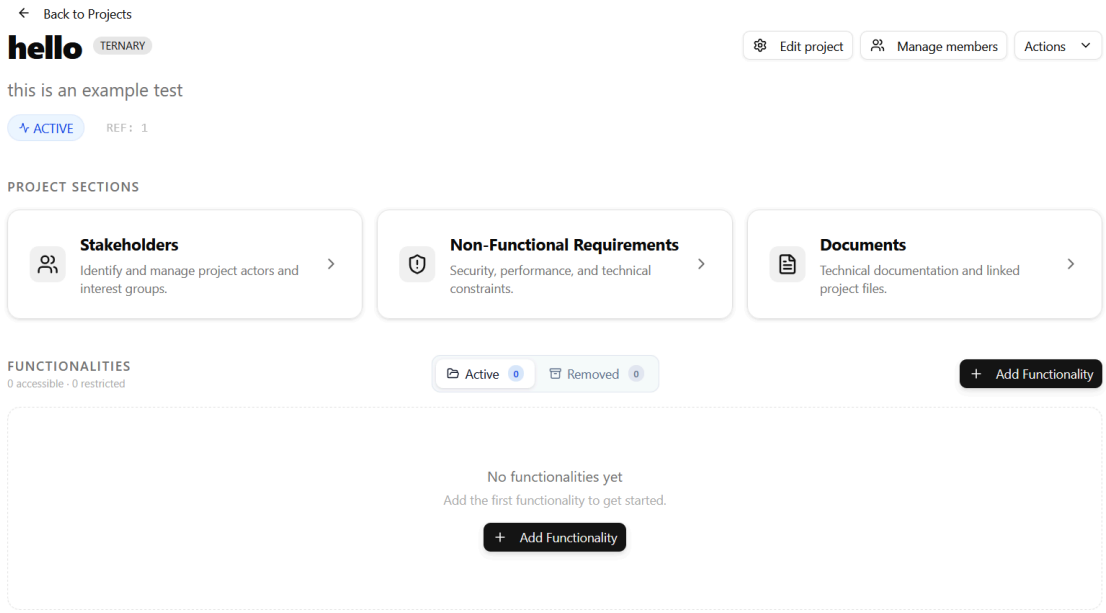

Figure 34: Empty project page

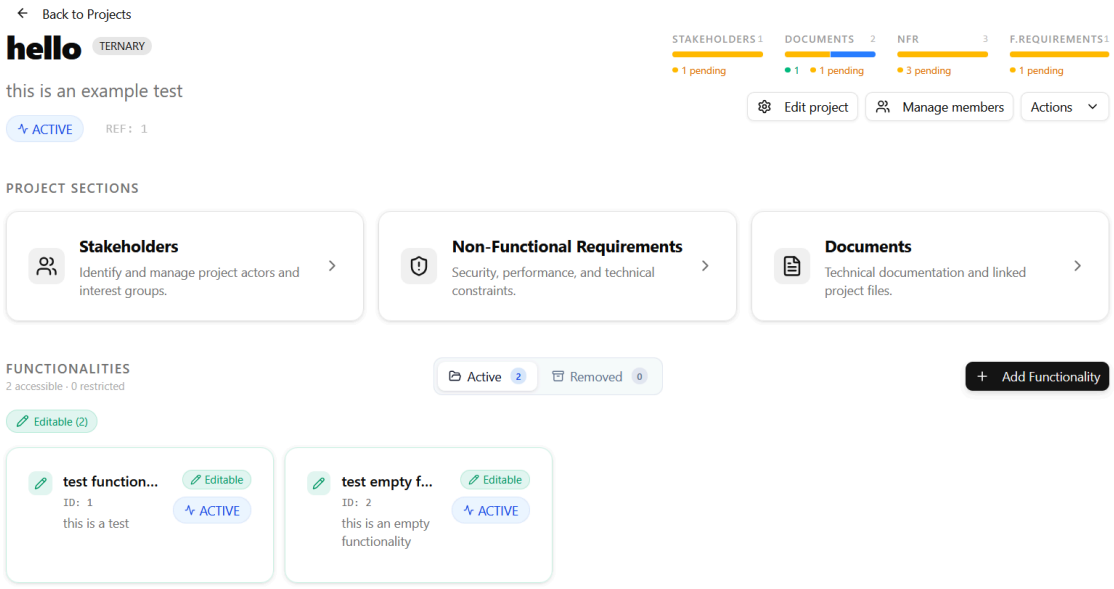

Figure 35: Example project page

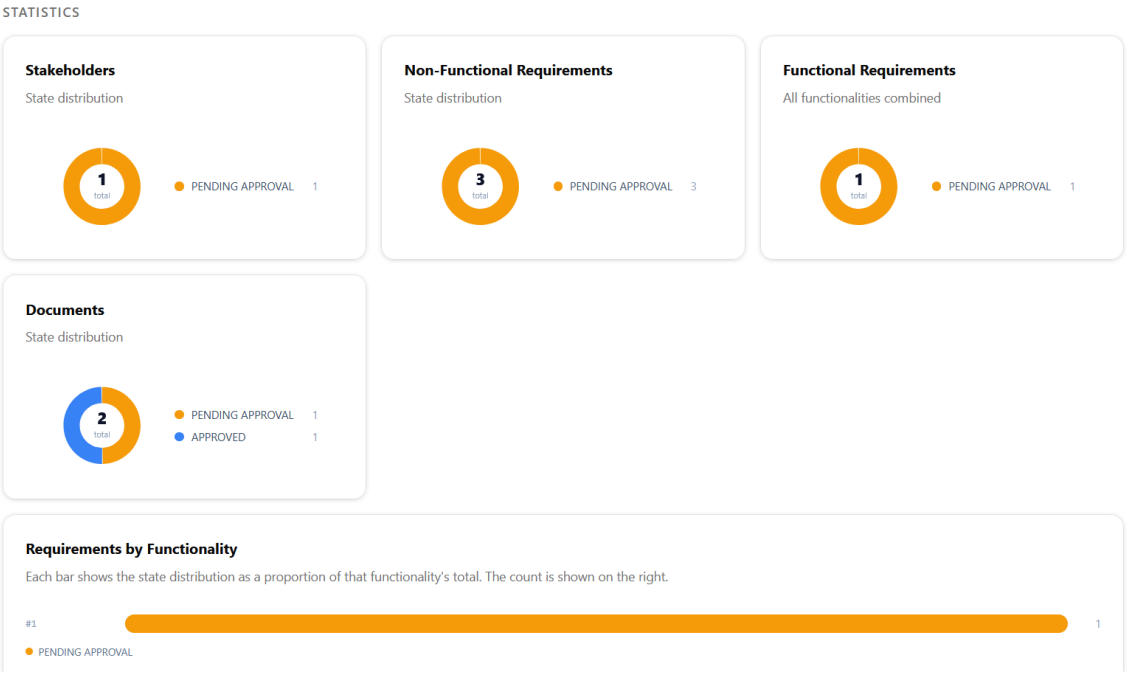

Figure 36: Project's dashboard

6.5.5 Functionality View

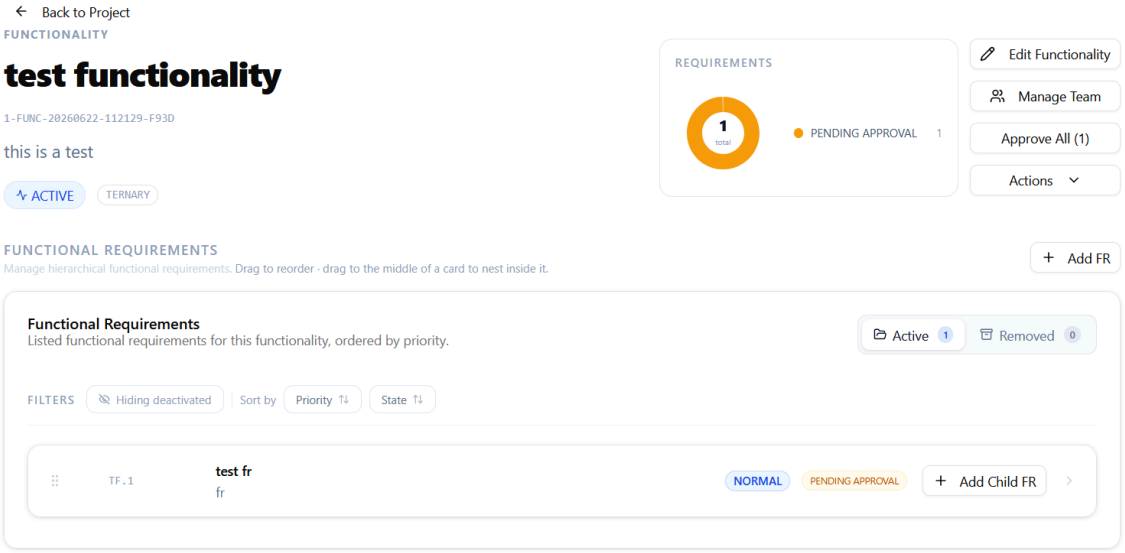

Figure 37: Functionality view

6.5.6 Project entity views

The pages for documents and stakeholders follow the same table design, therefore their main distinction is the update dialog for documents.

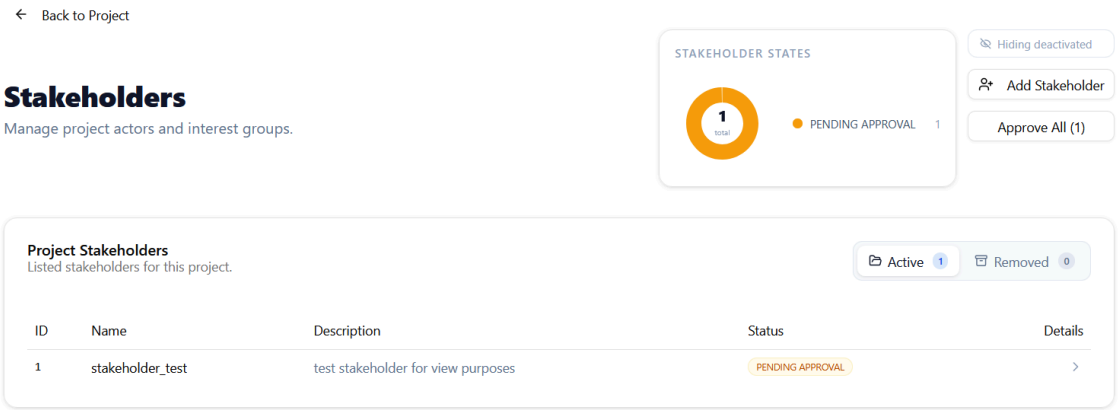

Figure 38: Stakeholders view

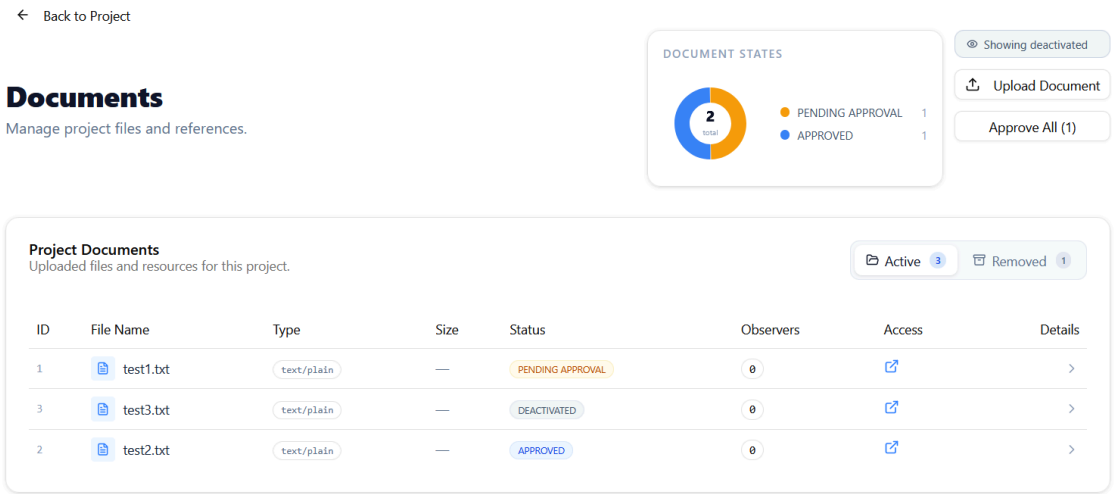

Figure 39: Documents view

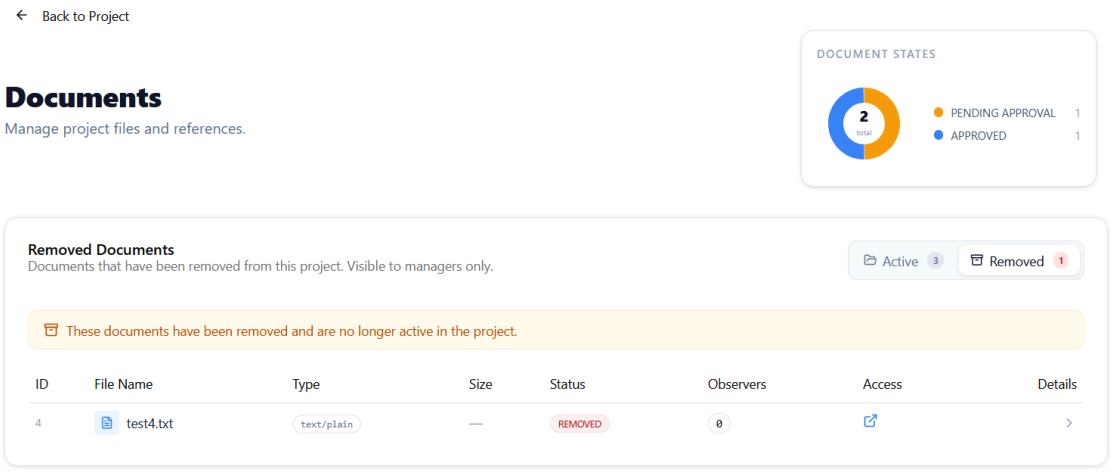

Figure 40: Removed documents view

Similarly, both functional and non functional requirements share the same design, as they are modeled in fundamentally identical ways.

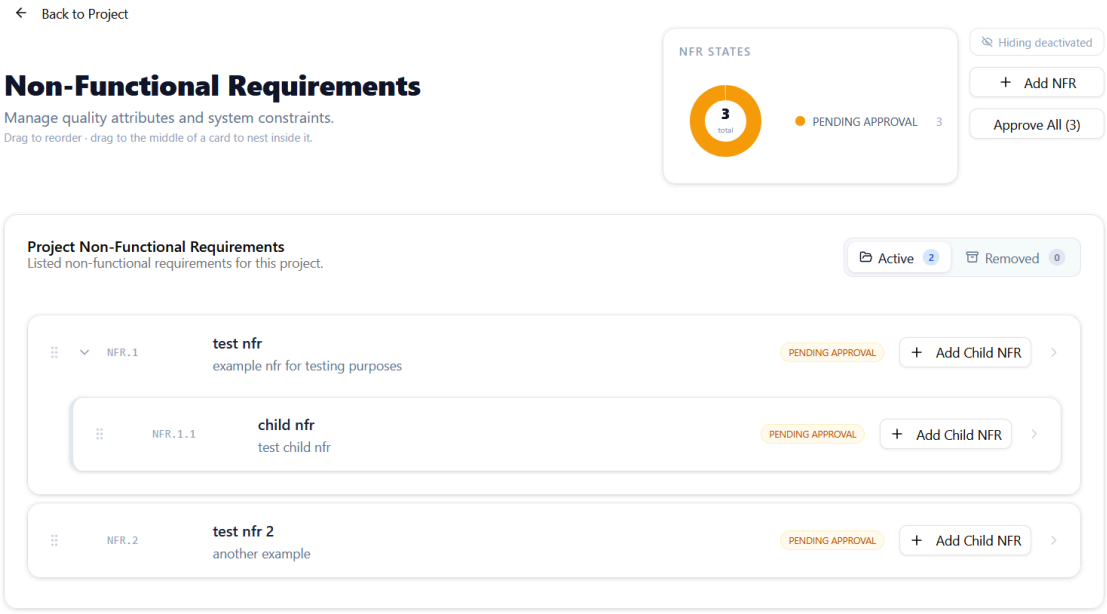

Figure 41: Nfr view

6.5.7 Entity Detail and Edit Views

Given the similarities between views, the most relevant example for an entity's detail view is the functional requirement's one:

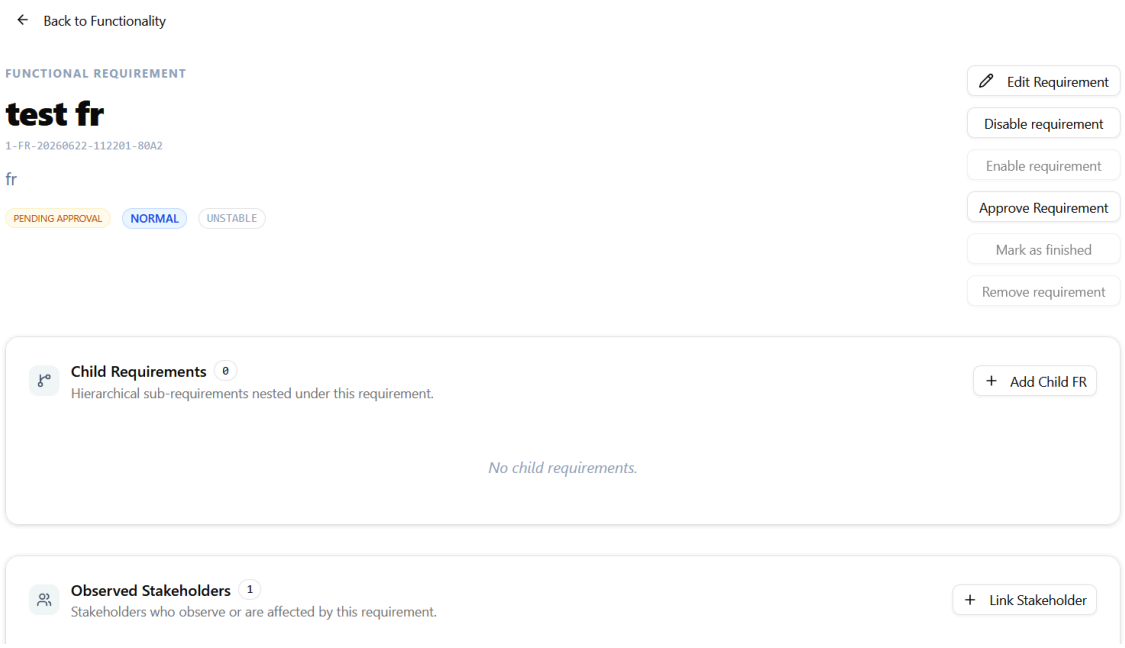

Figure 42: Functional requirement detail view

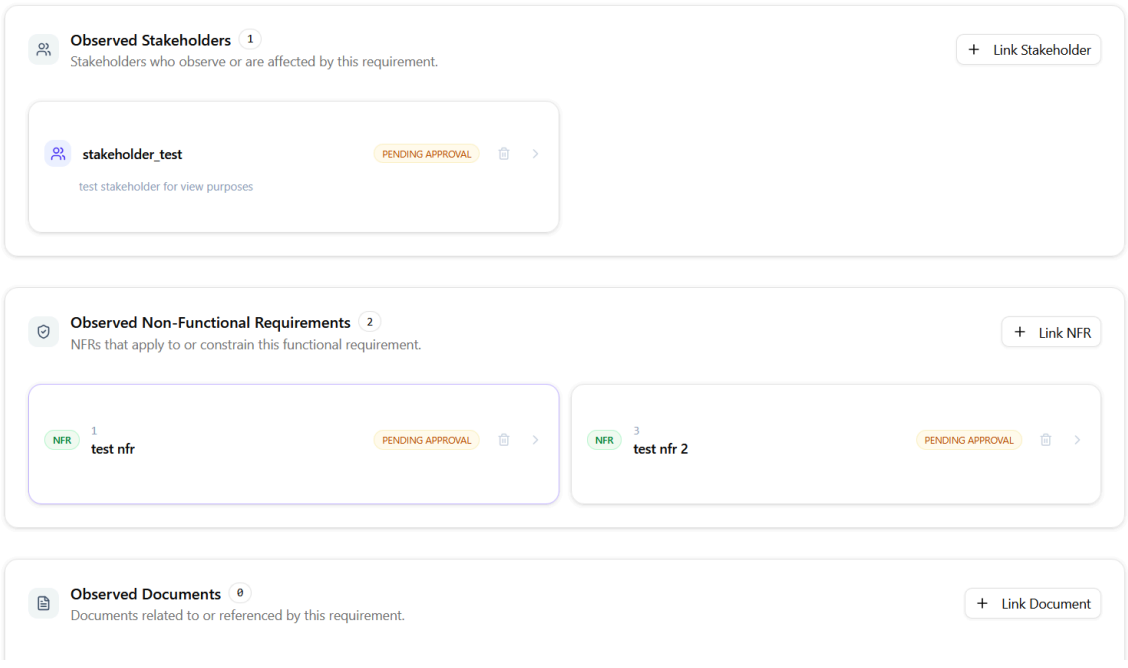

Figure 43: Functional requirement detail view, linked elements

And below would be the linked documents and other functional requirements, but are ommited as in the example none are provided and their sections have the exact same appearance.

6.5.8 User Management View

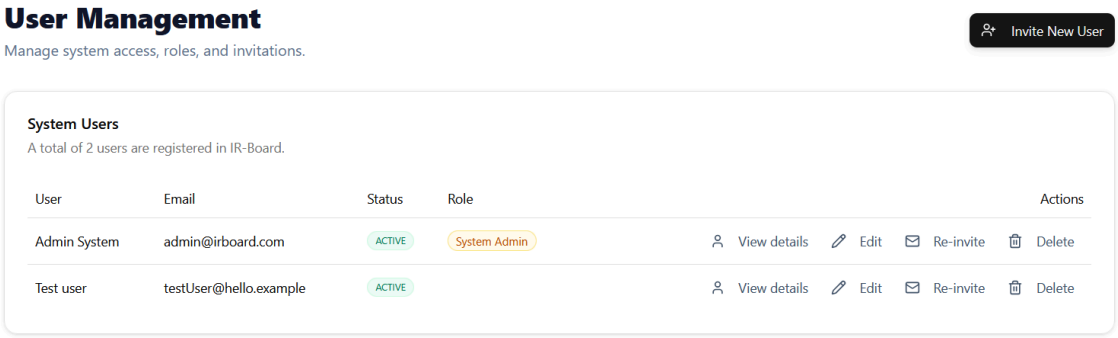

Figure 44: User management view

6.6 Test Plan Specification

The purpose of this section is to present the specification of the test plan that will be carried out to verify the correct functioning of the system's different components. The tests executed over the system aim to find and resolve errors in internal functionalities, infrastructure integration, and user interface design. The different types of tests, whose execution is reflected in the initial analysis presented earlier in this document, are described below.

6.6.1 Code quality and unit testing

Unit tests evaluate the isolated behavior of individual components within the codebase. In the particular case of this project, they are applied over the backend's domain model, drivers and service layers to verify that the adapters for the ory enviroment correctly translate the system calls to the appropiate set of calls, that state transition rules are correctly enforced, and that domain logic behaves as expected in isolation from external dependencies.

The selected tool for this layer is **SonarQube**, integrated into the development workflow as a Static Application Security Testing (SAST) tool. SonarQube will be configured to analyze the backend codebase on each significant development checkpoint, measuring code coverage, detecting code smells, identifying reliability issues through automated pattern matching, and scanning for common security vulnerabilities according to industry standards such as the OWASP Top 10. A minimum coverage threshold will be enforced to ensure that critical business logic is systematically exercised by automated tests before the system is considered ready for integration.

6.6.2 Integration and acceptance testing

Integration tests verify that independently developed components interact correctly with one another, targeting the boundaries between layers and services rather than the internal behavior of individual units.

For the **backend**, integration tests will be implemented using Spring Boot's testing support in combination with **Testcontainers**, which provisions isolated PostgreSQL container instances for each test run to guarantee full reproducibility. Each test scenario will exercise a complete vertical slice of the system, from the HTTP layer through the service and repository layers to the database, with the Ory ecosys-

tem components replaced by a controlled test security configuration that injects pre-authenticated contexts directly. This approach will validate that backend API contracts, requirement lifecycle state transitions, traceability relationships, concurrency control mechanisms, and document management operations all behave correctly when the full application context is active.

On the **frontend**, integration tests are written using React Testing Library in combination with Vitest, targeting individual pages and composite components in isolation from the live backend. API calls are intercepted by stubbing the global `fetch` function directly through Vitest's `vi.stubGlobal` mechanism, allowing each test scenario to return controlled responses for specific call sequences without requiring a running server. This approach validates component rendering under different response scenarios, form interaction behavior, error handling, and access-control-driven interface differences such as the conditional visibility of administrative actions.

Acceptance testing is treated as an extension of integration testing: each integration test scenario will be traceable to one or more functional requirements defined in the Requirements Specification, providing a structured verification record at the component level.

6.6.3 End-to-end testing

End-to-end tests validate the system as a deployed whole, exercising the complete stack from the browser through Traefik, Oathkeeper, Kratos, Keto, the backend, and the database, as a real user would interact with it. The selected tool for this layer is **Playwright**, which provides reliable cross-browser test execution, network interception capabilities, and a robust element selector model suited to the component structure of the React frontend.

Test scenarios are organized around the main user journeys identified in the requirements specification. Each scenario is executed against a deployment of the full Docker Compose stack, with the initial administrator account seeded by the same `initial-setup-admin` service used in regular deployment, ensuring that the infrastructure configuration is also exercised as part of each run. Timestamps are appended to dynamically generated entity names to allow parallelization and guarantee isolation between consecutive test executions without requiring a full database reset between runs.

The implemented scenarios are organized into two suites. The first covers the authentication flow:

Scenario	Description
Successful login	The administrator authenticates with valid credentials and is redirected to the home page, which is verified to render correctly.
Login page redirect when already authenticated	A user who already holds a valid session and navigates to the login page is automatically redirected to the home page without re-authenticating.

Scenario	Description
Logout removes session	An authenticated user logs out through the NavBar. The session is verified to be cleared by confirming that the home view is no longer accessible and the login form is presented again.
Invalid credentials	A login attempt with incorrect email and password is rejected. The user remains on the login page and an error is surfaced.

Table 20: E2E scenario specification: authentication

The second suite covers the core requirements engineering lifecycle, following the full create–operate–delete cycle for each entity type to ensure that creation, navigation, state transitions, and removal all function correctly end-to-end:

Scenario	Description
Project creation and deletion	An administrator creates a project with name, description, and stakeholder fields, verifies it appears on the home view, then takes it through the full removal lifecycle: disabling it, moving it to the removed archive, and permanently deleting it.
Stakeholder management	A project is created, a stakeholder is added to it with name and description, the stakeholder is navigated to and then taken through its full removal lifecycle from the stakeholder list view. The project is then cleaned up.
Non-functional requirement management	A project is created, a non-functional requirement is added with measurement unit, comparison operator, threshold, target, and actual values, and the requirement is taken through its full removal lifecycle. The project is then cleaned up.
Document management	A project is created, a plain-text file is uploaded as a document through the upload dialog, the document is verified to appear in the document list, and it is then taken through its full removal lifecycle. The project is then cleaned up.
Functionality management	A project is created, a functionality is added with a custom label, and the functionality view is confirmed to be reachable from the project dashboard. The project is then cleaned up.
Functional requirement management	A project is created, a functionality is added, and a functional requirement is created within it with name, description, priority, and stability values. The requirement is taken through its full removal lifecycle from the functionality view. The project is then cleaned up.

Table 21: E2E scenario specification: requirements engineering lifecycle

It should be noted that the concurrency control, slug-based search, and multi-user access control scenarios identified in the test plan analysis were not implemented within the scope of this project, and are documented as future work alongside the other testing improvements described in Conclusions and Future Work.

6.6.4 Load testing

Load testing will evaluate the robustness and responsiveness of the system under varying concurrent user conditions, focusing specifically on the critical path that passes through Traefik and Oathkeeper into the Spring Boot backend, as this chain involves the highest per-request overhead due to session validation against Kratos and permission checks against Keto.

The selected tool is **K6**, an open-source load testing framework that allows scripted execution of concurrent virtual user scenarios and produces detailed throughput, latency, and error-rate metrics. Tests will be executed directly against the deployed Docker Compose stack, simulating realistic usage patterns: gradual ramp-up of concurrent users, sustained load periods, and burst scenarios. During execution, the Grafana observability stack will be used to correlate K6 metrics with container-level resource usage and internal service latency, allowing the identification of bottlenecks at specific layers of the architecture rather than only at the system boundary.

The primary metrics to be observed are mean and percentile request latency, error rate under sustained load, session management stability in Kratos, and the behavior of the internal network routing under heavy pressure.

6.6.5 Usability and Accessibility testing

Usability testing evaluates the degree to which the system can be used by its target professional profiles efficiently, without unnecessary friction, and without requiring prior knowledge of its internal structure. Its objective is to identify interaction patterns, navigability issues, or interface elements that produce confusion, hesitation, or error in realistic usage scenarios.

Accessibility testing is the process of evaluating a website or application to ensure it is usable by people with disabilities. Given the MVP nature of this project, accessibility testing has remained minimal, as the color pallette from the frontend has remained primarily black and white. Despite this, test user behaviour will be analyzed to check whether they have been able to spot low contrast elements without being asked to.

Test participants will be selected from software-related professional profiles, including individuals with experience in software development, requirements engineering, or project management, as these represent the system's primary user groups. Participants with no prior familiarity with IR-Board will be preferred, and no time to explore or familiarize themselves with the system will be provided before the session begins, to avoid familiarity bias influencing the results.

Each session will follow a structured task-based protocol, during which the participant will be asked to complete the following sequence of operations without

assistance from the observer. Participants will be encouraged to verbalize their thought process using a think-aloud protocol throughout the session.

1. Sign in to the system using provided credentials.
2. Create a project, navigate to it and identify its dashboard metrics.
3. Add a new stakeholder to the project.
4. Add a new functionality to the project.
5. Create a functional requirement within the new functionality, filling in all available fields.
6. Link the created requirement to the previously created stakeholder.
7. Navigate to the stakeholder's detail view from the requirement's detail view.
8. Create a non-functional requirement with a measurement unit and a target value.
9. Upload a document and link it to the previously created functional requirement.
10. Approve all pending requirements in the functionality.
11. Search for a specific entity using its slug in the NavBar search field.
12. Deactivate the created functional requirement and verify its new state.
13. Log out of the system.

The observer will not intervene at any point during the session, unless the user locks up or is unwilling to go on. All observations are recorded using the following sheets.

The per-step recording sheet is used to track objective measurements and any remarks for each individual task:

Step	Task	Time (s)	Completed	Doubts / Issues / Comments
1	Sign in			
2	Create project and read dashboard metrics			
3	Add a new stakeholder			
4	Add a new functionality			
5	Create a functional requirement (all fields)			
6	Link requirement to stakeholder			
7	Navigate to stakeholder from requirement detail			
8	Create a non-functional requirement			

Step	Task	Time (s)	Completed	Doubts / Issues / Comments
9	Upload document and link to requirement			
10	Approve all pending requirements			
11	Search for entity by slug			
12	Deactivate requirement and verify state			
13	Log out			

Table 22: Usability test: per-step recording sheet

The general observation sheet is used to record broader behavioral impressions throughout the session:

Aspect observed	Always	Frequently	Occasionally	Never
Does the user know where they are within the application?				
Is navigation through the application intuitive?				
Does the user know how to authenticate and log out?				
Does each action produce the expected result?				
Does the user find the NavBar helpful for orientation?				
Does the user feel lost at any point during the session?				
Is the requirement creation form easy to complete?				
Are state labels and lifecycle transitions clearly communicated?				

Table 23: Usability test: general observation sheet

The session summary sheet is completed by the observer at the end of each session:

Aspect	Notes
Total session duration	

Aspect	Notes
Number of steps completed without assistance	
Steps that required the most time	
Recurring points of hesitation or confusion	
Errors committed and recovery behavior	
Overall impression of participant confidence	
Any additional free-form observations	

Table 24: Usability test: session summary sheet

Results will be analyzed across participants to identify recurring friction points, prioritized by frequency and severity of observed impact, and cross-referenced against the functional requirements specification and the navigability diagram to determine whether each finding represents an implementation gap or a design decision to be addressed in future work.

Chapter 7. System Implementation

7.1 Standards and regulations followed

This project follows requirements engineering principles defined by IEEE 830 and ISO/IEC/IEEE 29148, which establish recommendations for writing, managing, validating, and maintaining software requirements.

Security aspects follow principles such as Zero Trust and least privilege, ensuring that access is controlled and only granted according to user permissions and relationships. The system also applies general software quality principles related to reliability, maintainability, and security.

7.2 Programming languages used

To implement the system, the following programming languages were used:

7.2.1 Yaml

YAML (YAML Ain't Markup Language) is a human-readable data serialization format used for configuration files. In the project, YAML was used to define the Docker Compose environment and the configuration files of the Ory services, including Kratos, Keto, and Oathkeeper. These files describe service parameters, dependencies, networks, storage, authentication flows, and authorization-related settings.

7.2.2 Dockerfile

Dockerfile is a scripting format used to define the steps required to build Docker images. It specifies the base image, installed dependencies, copied files, exposed ports, and execution commands required to package an application into a container.

7.2.3 Shell Script (sh)

Shell Script is a scripting language used to automate tasks and execute commands through a command-line environment. In the project, shell scripts were used to define and automate deployment operations like inserting the initial admin.

7.2.4 TypeScript

TypeScript is a statically typed programming language built on top of JavaScript. It was used for the application frontend, providing type checking, improved maintainability, and safer implementation of the system logic.

7.2.5 Java

Java is an object-oriented programming language used for backend development. It was used to implement the server-side application logic and provide the main structure of the application.

7.2.6 JPQL

JPQL (Java Persistence Query Language) is an object-oriented query language used to retrieve and manipulate data through Java persistence entities instead of directly querying database tables. In the project, JPQL was used to define database queries through the application's entity model, allowing interaction with the persistence layer while maintaining independence from the underlying database structure.

7.2.7 Kratos and Keto, JSON schema and OPL

JSON Schema is a format used to define and validate the structure of JSON documents. In the project, it was used for defining the expected structure for kratos's identity schema.

Ory Keto uses the Ory Permission Language (OPL)[6] to define authorization rules and relationship-based access control models. The project used OPL to describe the relations between users and resources according to the ReBAC authorization model described on users and roles.

7.3 Tools and Software used

7.3.1 Visual Studio Code

Visual Studio Code is a source code editor used for developing and editing the project files. It was mainly used for frontend application development, configuration files, architecture definition and deployment, end-to-end testing, and documentation-related work.

7.3.2 Docker Desktop

Docker Desktop is a tool that provides an environment for creating and running Docker containers. It was used to manage the project containers, including the application services and Ory infrastructure components.

7.3.3 Intellij IDEA Ultimate

IntelliJ IDEA Ultimate is an integrated development environment for Java development. It was used to implement, debug, test, and manage the backend application code.

7.3.4 Git & Github

Git is a version control system used to track changes in the project source code. GitHub is a platform for hosting Git repositories and was used as the main remote repository, version management, and project backup.

7.3.5 PlantUML

PlantUML is a tool for generating diagrams from textual descriptions. It was used to create software architecture diagrams, UML diagrams, and other visual documentation elements.

7.3.6 Gatling

Gatling is a performance testing tool used to simulate user activity and measure application behaviour under load. It was used to evaluate system performance and response times.

7.3.7 Typst

Typst is a modern document preparation system used to create technical documentation. It was used for writing and formatting the project report.

7.3.8 MS Excel and MS Project

Microsoft Excel is a spreadsheet application used for data organisation, calculations, and budget-related analysis. Microsoft Project is a project management tool used for scheduling, task planning, and Gantt chart generation.

7.3.9 Azure

Microsoft Azure is a cloud computing platform that provides services for application hosting, database management, identity management, and system monitoring. In this project, Azure was the deployment environment due to its student plan.

7.4 Issues encountered

TODO

Here's the revised section with the SonarQube issue added:

Infrastructure and security architecture complexity The most significant implementation challenge was the integration of the Ory ecosystem within the Zero-Trust architecture. While the conceptual authorization model was compatible with Ory Keto and Oathkeeper, correctly configuring the surrounding infrastructure proved more complex than anticipated. Determining which services should sit behind the Oathkeeper gateway and which should remain on the internal network required substantial experimentation, as did correctly defining the trust boundaries between components. This was compounded by limited prior experience with multi-layered security architectures. A concrete manifestation of this was the integration of additional infrastructure components (particularly the observability stack (Grafana, Loki, Promtail) and the object storage service (MinIO)) where securely separating user-facing traffic from internal service communication required a deeper understanding of network segmentation than initially anticipated. SonarQube setup and CI integration The setup of SonarQube for quality assurance took considerably longer than estimated, which is reflected in the doubling of its planned hours from 3 to 6. Several issues were encountered during configuration: initial difficulties getting the SonarQube instance running correctly within the development environment, and a particularly time-consuming problem where the CI pipeline action was executing a standard test run rather than a coverage-instrumented one, meaning that no coverage data was being reported to SonarQube despite tests passing. Diagnosing and correcting this required iterating on the pipeline configuration until the correct test execution mode was in place and coverage metrics were being collected and forwarded as expected. Document management: architectural decision on file transfer During the implementation of document upload and download functionality, the file content was routed through the Spring Boot backend rather than using presigned URLs for direct client-to-storage transfers. This decision was made due to limited prior familiarity with S3-compatible object storage integration patterns at the time of implementation. While functional, this approach is not the standard pattern for systems built on an S3-compatible backend, and results in unnecessary load on the backend service. This is identified and documented as a future improvement. Cross-service consistency on partial failures Several operations involving coordinated changes across the relational database and the object storage backend (most visibly bulk document deletion) were found to lack

a distributed transaction or compensating-action mechanism. A failure partway through a multi-step operation can leave the system in an inconsistent state: for example, a document record deleted from the database but whose corresponding object was not successfully removed from MinIO, or vice versa. Manual intervention would be required to recover from such cases. This was recognized during development but left unresolved within the project scope. MinIO dependency maturity During the development period, MinIO's open-source Community Edition underwent significant changes: administrative functionality was removed from its web console in 2025, and the upstream project was subsequently placed into maintenance mode in December 2025. This meant that a dependency originally selected as a mature, actively maintained S3-compatible storage solution became effectively frozen during the course of the project. The current deployment uses unpinned latest tags for both minio/minio and minio/mc, which poses a risk in terms of reproducibility and future security patch availability. Migration to an actively maintained alternative is proposed as future work. Typst compatibility with nested requirement lists During the preparation of the requirements specification, a compatibility issue was encountered with Typst's support for nested lists. Since the requirements documentation relies heavily on hierarchical structures (such as identifiers following formats like UM.1.1) the absence of adequate native multilevel list support at the time created difficulties in maintaining the intended document structure. Resolving this required additional investigation and the adoption of an external Typst extension (efilrst) to restore the required nested list behavior. This added unforeseen effort to the documentation phase. Usability issues discovered during testing Several interface issues were identified during usability testing that required corrective action:

The entity identity slug was consistently missed or misunderstood by all four test users due to low contrast and a lack of explanatory labeling. The corrective measure was to increase contrast and add an explanatory tooltip. The filter for disabled requirements on the non-functional requirements view was placed unexpectedly relative to its counterpart on the functionality view, causing confusion. This was corrected by aligning the filter placement. State badges on entity detail views were positioned in a way that made them easy to overlook. They were moved to a more prominent position beside the identity slug. Clicking the user avatar in the navigation bar navigated to an unimplemented route, causing an error page. The redirect logic was commented out as the user profile editing view is outside the project scope. The slug search field did not respond to the Enter key, which users consistently expected. Keyboard navigation was documented as future work.

draw.io integration not fully completed The self-hosted draw.io instance and its supporting infrastructure (including Traefik routing, CORS configuration, and frontend embedding points) were fully provisioned as part of the architecture. However, the end-to-end integration was not brought to a fully operational state within the project timeline, as development effort was redirected toward ensuring testing coverage, usability testing, and the observability stack were complete. No architectural changes are required; this is a completion task deferred to future work. Requirements export to PDF not implemented PDF export of requirements

was a planned extension of the project scope but could not be implemented within the available development time. Project managers currently have no built-in mechanism to generate a portable snapshot of a project's requirements for external stakeholders. This is documented as future work. Incomplete test coverage

Several testing areas could not be completed within the project timeline. Multi-user and access-control E2E scenarios were not automated, meaning the permission boundaries between project managers, requirement engineers, and stakeholder users have not been validated end-to-end against the live Ory Keto layer. The concurrency control mechanism and the slug-based search flow also lack automated E2E coverage, though both were manually verified during development. Load testing was executed using Gatling against the deployed stack; its results and observations are documented in the corresponding test plan execution section.

Chapter 8. Test Plan Execution

8.1 Unit Testing

8.1.1 Frontend

Frontend unit tests were implemented using Vitest in combination with React Testing Library, targeting individual pages and components in isolation from the rest of the application. Since the frontend is composed of React components that encapsulate both rendering logic and user interaction, the boundary between unit and integration testing was deliberately blurry at this layer: a test that renders a single page component inevitably exercises its child components, hooks, and routing context simultaneously. For this reason, the distinction between unit and integration tests on the frontend is treated as a matter of scope rather than a strict structural boundary, and is discussed further in the integration testing section below.

Each test file renders the component under test within a controlled environment. Routing is simulated using React Router's MemoryRouter, allowing components that depend on navigation state or URL parameters to be exercised without a live browser. API calls and browser globals are replaced with Vitest's mocking primitives (vi.fn, vi.stubGlobal, vi.mock), ensuring that each test scenario exercises only the component's own logic and rendering behavior without introducing external dependencies.

A representative example is the ErrorPage component test suite, which validates that the page renders the correct heading and explanatory message when given a specific error state, and that the return home button triggers the expected navigation call:

```
it("calls window.location.replace when Return Home is clicked", () => {
  const replaceMock = vi.fn()
  vi.stubGlobal("window", {
    ...window,
    location: { ...window.location, replace: replaceMock },
  })
  renderError()
  const button = screen.getByRole("button", { name: /return home/i })
  button.click()
  expect(replaceMock).toHaveBeenCalledWith("/home")
})
```

This pattern is consistent across the test suite: the component is rendered with a controlled initial state, user interactions are triggered programmatically, and assertions are made against the resulting DOM or against mock call records. The use of `beforeEach(() => vi.restoreAllMocks())` ensures that mock state does not leak between test cases.

8.1.2 Backend

Backend unit tests were implemented using JUnit 5 as the test runner and Mockito as the mocking framework, applied across the distinct layers of the backend: the domain model, the service layer and the client implementations on the infrastructure layer.

All unit tests are executed in complete isolation, without loading the Spring application context or interacting with repositories, databases, or other external infrastructure. The unit under test is instantiated directly, while collaborators are replaced with Mockito mocks only when necessary to isolate the behavior being verified. Domain model classes are generally tested without mocks, as they represent self-contained business objects whose behavior can be exercised directly through their public API.

The unit test suite verifies a broad range of behaviors throughout the application. For domain entities, tests focus on validating business rules, state transition constraints, object identity semantics, observer notifications, validation logic, and the correct handling of edge cases such as null values or invalid state changes. For components with external dependencies, the use of mocks allows tests to verify that business logic is applied correctly, authorization and validation rules are enforced, lifecycle transitions occur as expected, exceptions are propagated or translated appropriately, and dependencies are invoked with the correct parameters and interaction sequence. This approach ensures that each unit is validated independently while remaining unaffected by the behavior of surrounding components.

8.1.3 SonarQube

Code coverage is measured and enforced by SonarQube as part of the continuous integration pipeline. A minimum coverage threshold of 80% is required for the quality gate to pass, while the analysis also detects code smells, reliability issues, duplicated code, and common security vulnerabilities at each significant development checkpoint.

Several code maintainability issues were encountered on the backend, and code duplication was found over the maximum threshold on the frontend upon initial testing. To prepare for the final delivery, all of these were mitigated to pass the quality gate enforced by the system.

8.2 Integration and Acceptance testing

8.2.1 Frontend

As noted in the unit testing section, the boundary between unit and integration testing on the frontend was intentionally blurred. Since React components are compositional by nature, a test that renders a page component exercises not only that page's own logic but also any child components, context providers, and hooks that it depends on. Tests that cross these boundaries (for example, rendering a page that coordinates multiple child components, shared context state, and a sequence of asynchronous fetch calls) are treated as integration tests, even though they use the same Vitest and React Testing Library tooling as the unit tests.

The main distinction applied in practice is one of scope and intent: unit tests target a single component's rendering and interaction behavior with all external dependencies mocked at the module boundary, while integration tests render a larger slice of the component tree and validate the coordination between components. This includes shared state managed through React Context, conditional rendering

driven by sequences of API responses, and navigation transitions triggered by user actions that span multiple components.

API calls are intercepted by stubbing the global fetch function via vi.stubGlobal, allowing each scenario to return a controlled sequence of responses for specific call patterns without requiring a running backend. This approach validates that the frontend correctly handles the full range of response scenarios it may encounter in practice, including successful responses, validation errors, unauthorized responses, and network failures, and that the correct interface state is presented in each case.

Access-control-driven interface differences are also covered at this layer: tests assert that administrative actions such as user invitation controls or project deactivation buttons are conditionally rendered based on the role information returned in the mocked session response, confirming that permission boundaries are enforced at the component level independently of the backend.

8.2.2 Backend

Backend integration tests verify that independently developed components interact correctly when the full Spring Boot application context is active, targeting the boundaries between the HTTP layer, the service layer, the repository layer, and the database rather than the internal behavior of individual units.

Tests are organized around a shared abstract base class, `IrBoardBaseTest`, annotated with `@SpringBootTest(webEnvironment = SpringBootTest.WebEnvironment.RANDOM_PORT)` and `@ActiveProfiles("test")`. This base class provisions all shared infrastructure: it starts a PostgreSQL instance through Testcontainers, whose connection properties are injected at runtime via `@DynamicPropertySource`, ensuring complete isolation between test runs. The Ory ecosystem components are replaced entirely by Mockito beans: `KetoClient`, `MinioClient`, and `MinioObjectStorageClient` are declared as `@MockitoBean` instances, allowing the test security configuration to inject pre-authenticated request contexts directly through the `X-User` header rather than requiring live Kratos session validation on every request.

The base class also provides a structured set of helpers that make individual test classes concise and focused. Authentication and permission state is controlled through a family of `allowX` methods that stub specific Keto permission checks for a given user and resource identifier, such as `allowEdit`, `allowEditProject`, and `allowViewRequirementsOfFunctionality`. A `RestClient` instance pre-configured with the random server port is exposed through typed `get`, `post`, `patch`, `delete`, and multipart request helpers that inject the `X-User` header automatically. Entity builder helpers (`buildProject`, `buildFunctionality`, `buildStakeholder`, `buildFr`, `buildNfr`, `buildDocument`) produce fully configured, persistable domain objects with sensible defaults, and `reload` helpers re-fetch each entity type from its repository by identifier to assert against the state actually persisted in the database after each operation.

Each test suite's `@BeforeEach` method delegates to `testSetup` in the base class, which clears all repository data in dependency order to guarantee a clean slate between

tests, seeds the fixed set of user identities referenced across suites by their `oryId` constants, and creates a baseline active project. Each concrete test class then implements the `setUp` hook to seed only the additional fixtures it needs, in a clear example of a template method pattern.

This structure allows each integration test to exercise a complete vertical slice of the system: an HTTP request is issued to a real controller endpoint, passes through the Spring Security filter chain with the injected identity, reaches the service and repository layers against the live Testcontainers database, and the response status, body, and resulting database state are all asserted. The following categories of behavior are covered across the test suites:

- **API contract validation:** each endpoint is verified to return the expected response structure and HTTP status code for both successful and error scenarios, including 400 for malformed input, 403 for insufficient permissions, 404 for missing entities, and 409 for constraint violations such as duplicate functionality labels.
- **Requirement lifecycle transitions:** the full state machine is exercised end-to-end, verifying that the correct successor states are persisted after approval, deactivation, reactivation, removal, and permanent deletion operations, and that transitions from invalid predecessor states are rejected.
- **Traceability relationship management:** linking and unlinking requirements to stakeholders, documents, and peer requirements is verified to correctly update the corresponding join tables in the database and to propagate pending-review flags to all observing entities.
- **Concurrency control:** entity lock acquisition is verified to succeed for the first requesting user and to be rejected for a second user targeting the same entity, and lock release on save or timeout is verified to restore the entity to an editable state.
- **Document management:** upload, association with requirements, state transitions, and permanent deletion are verified end-to-end through the controller, service, storage adapter, and database layers.
- **Access control boundaries:** each operation is verified to return 403 when the requesting user lacks the required Keto permission, confirming that authorization is enforced at the service layer before any domain logic is executed.

Acceptance testing is treated as a direct extension of integration testing: each integration test scenario is traceable to one or more functional requirements defined in the requirements specification, providing a structured verification record at the component boundary level.

8.3 Accessibility testing

TODO

8.4 Usability Testing

The results documented from the four rounds of usability testing are recorded below. As all users completed successfully the scenarios, the “completed” column was removed from the table.

Similarly, the errors are documented upon each step, and every user was quite confident on their actions and feedback until the identity slug step. Therefore, their appropiate rows have been removed from the session summary.

8.4.1 User 1

Step	Task	Time	Doubts / Issues / Comments
1	Sign in	3s	
2	Create project and read dashboard metrics	2s	Expected the project card to be clickable, not just the “more...” button on the bottom.
3	Add a new stakeholder	14s	Did not understand the states elements could be in.
4	Add a new functionality	31s	Struggled to see the functionalities a bit. expecting functionalities to be with the three links above (stakeholders, nfrs and documents).
5	Create a functional requirement (all fields)	35s	almost didnt put description but was forced to by the system
6	Link requirement to stakeholder	10s	-
7	Navigate to stakeholder from requirement detail	3s	Correctly used the shortcut from the detail view
8	Create a non-functional requirement	26s	Did not understand nfr numerical values until detail view.
9	Upload document and link to requirement	65s	Attempted to link it from document detail view.
10	Approve all pending requirements	20s	As only 3 requirements existed, (he created a child) he did each manually.
11	Search for entity by slug	200s	Did not understand what a slug was, had not seen it due to low contrast.
12	Deactivate requirement and verify state	114s	Looked at the child’s state badge instead of the current one, and did not see the filters at all. Found unexpected that the disabled requirements were hidden by default

Step	Task	Time	Doubts / Issues / Comments
13	Log out	4s	

Table 25: User 1 - per-step recording sheet

Aspect observed	Always	Frequently	Occasionally	Never
Does the user know where they are within the application?		■		
Is navigation through the application intuitive?		■		
Does the user know how to authenticate and log out?	■			
Does each action produce the expected result?			■	
Does the user find the NavBar helpful for orientation?		■		
Does the user feel lost at any point during the session?			■	
Is the requirement creation form easy to complete?		■		
Are state labels and lifecycle transitions clearly communicated?		■		

Table 26: User 1 - general observation sheet

Aspect	Notes
Total session duration	45 minutes, as issues with deployment caused the testing to be done with a LAN VPN
Number of steps completed without assistance	All
Steps that required the most time	The identity slug searching and deactivation. Clearly, the application does not explain correctly what one is.
Any additional free-form observations	The user commented how he did know where he was always, but could not grasp the actual size of the application.

Table 27: User 1 - session summary sheet

8.4.2 User 2

Step	Task	Time	Doubts / Issues / Comments
1	Sign in	5s	-
2	Create project and read dashboard metrics	35s	-
3	Add a new stakeholder	17s	-
4	Add a new functionality	29s	Expected a functionality link with the other top three.
5	Create a functional requirement (all fields)	44s	Complemented the requirement nesting and reordering.
6	Link requirement to stakeholder	12s	-
7	Navigate to stakeholder from requirement detail	2s	-
8	Create a non-functional requirement	31s	Thought he had to change the numerical values on the form.
9	Upload document and link to requirement	49s	Dislikes the linking being only allowed from the requirement.
10	Approve all pending requirements	13s	Used the approve all helper button.
11	Search for entity by slug	71s	Found counterintuitive that enter key does not goto the found entity.
12	Deactivate requirement and verify state	57s	Mistook the functionality deactivation for requirement.
13	Log out	12s	-

Table 28: User 2 - per-step recording sheet

Aspect observed	Always	Frequently	Occasionally	Never
Does the user know where they are within the application?		■		
Is navigation through the application intuitive?	■			
Does the user know how to authenticate and log out?	■			
Does each action produce the expected result?	■			
Does the user find the NavBar helpful for orientation?			■	

Aspect observed	Always	Frequently	Occasionally	Never
Does the user feel lost at any point during the session?				
Is the requirement creation form easy to complete?				
Are state labels and lifecycle transitions clearly communicated?				

Table 29: User 2 - general observation sheet

Aspect	Notes
Total session duration	20 minutes, the necessary setup time for the LAN VPN. Quicker due to prior experience of the observer with the setup.
Number of steps completed without assistance	All
Steps that required the most time	Mainly the slug searching, the requirement deactivation and the entity linking steps.
Any additional free-form observations	entity slug is not explained what is anywhere.

Table 30: User 2 - session summary sheet

8.4.3 User 3

Step	Task	Time	Doubts / Issues / Comments
1	Sign in	10s	-
2	Create and read dashboard metrics	12s	-
3	Add a new stakeholder	10s	-
4	Add a new functionality	25s	Expected a functionality link with the other top three.
5	Create a functional requirement (all fields)	61s	Did not knew where to create the fr whithin the functionality, and did not recognise FR as Functional Requirement
6	Link requirement to stakeholder	13s	-

Step	Task	Time	Doubts / Issues / Comments
7	Navigate to stakeholder from requirement detail	1s	-
8	Create a non-functional requirement	32s	Disliked the use of acronyms instead of full text on buttons.
9	Upload document and link to requirement	31s	-
10	Approve all pending requirements	17s	Used the approve all helper button
11	Search for entity by slug	117s	Missed the slug completely on every detail view. Attempted partial search, and due to the big screen, could not see the navigation bar.
12	Deactivate requirement and verify state	13s	-
13	Log out	17s	Clicked the user image and went into an invalid route.

Table 31: User 3 - per-step recording sheet

Aspect observed	Always	Frequently	Occasionally	Never
Does the user know where they are within the application?	■			
Is navigation through the application intuitive?		■		
Does the user know how to authenticate and log out?		■		
Does each action produce the expected result?	■			
Does the user find the NavBar helpful for orientation?				■
Does the user feel lost at any point during the session?			■	
Is the requirement creation form easy to complete?	■			
Are state labels and lifecycle transitions clearly communicated?			■	

Table 32: User 3 - general observation sheet

Aspect	Notes
Total session duration	15 minutes.
Number of steps completed without assistance	Every one except the slug search, as several reminders of the task were needed; reminding him to search on the nav bar with an entity's identity slug to nudge away from the project search and to search for an identifier that was not the dynamic identifier but rather something common to all project elements.
Steps that required the most time	The slug search and functional requirement creation step.
Any additional free-form observations	Finds unintuitive the collapsible nav bar, suggested that the badges should be besides the slug or right below the name

Table 33: User 3 - session summary sheet

8.4.4 User 4

Step	Task	Time	Doubts / Issues / Comments
1	Sign in	3s	-
2	Create and read dashboard metrics	17s	Attempted to enter the project by clicking the card.
3	Add a new stakeholder	6s	-
4	Add a new functionality	16s	-
5	Create a functional requirement (all fields)	13s	
6	Link requirement to stakeholder	6s	-
7	Navigate to stakeholder from requirement detail	4s	Correctly used the shortcut.
8	Create a non-functional requirement	6s	-
9	Upload document and link to requirement	28s	Looked for the linking within the document detail view.
10	Approve all pending requirements	4s	Approved every single requirement as he created them.
11	Search for entity by slug	84s	Does not know what a slug is. Saw it multiple times but did not know what it was

Step	Task	Time	Doubts / Issues / Comments
12	Deactivate requirement and verify state	179s	Disables it, but does not see the requirement state badge. Is confused due to the requirement dissapearing from the requirements view. Does not see the filter on nfr view.
13	Log out	12s	Pressed the user icon and navigated to an invalid route.

Table 34: User 4 - per-step recording sheet

Aspect observed	Always	Frequently	Occasionally	Never
Does the user know where they are within the application?	■			
Is navigation through the application intuitive?		■		
Does the user know how to authenticate and log out?	■			
Does each action produce the expected result?		■		
Does the user find the NavBar helpful for orientation?				■
Does the user feel lost at any point during the session?			■	
Is the requirement creation form easy to complete?	■			
Are state labels and lifecycle transitions clearly communicated?		■		

Table 35: User 4 - general observation sheet

Aspect	Notes
Total session duration	20 minutes between LAN VPN setup and actual testing.
Number of steps completed without assistance	11, all but the slug and deactivation.
Steps that required the most time	Entity slug searching and requirement deactivation due to low contrast.

Aspect	Notes
Any additional free-form observations	-

Table 36: User 4 - session summary sheet

8.4.5 Conclusions

Several issues arise from the usability and accessibility testing, from which we can obtain the following actions to mitigate their impact:

Issue found	Corrective measure
Low contrast on entity slug and project's functionality section labels	Increase contrast on each text, as well as correctly label the slug as "identity slug"
Unexpected placement of disabled filter on nfr view	Move the filter to a similar position as it is already done on functionality view
Unexpected placement of badges on detail view makes it easier for the user to miss them	Place the badges on the same row as the entity slug, to be right below the name and with the slug to its right, to be naturally discovered by following the elements.
Lack of understanding for nfr numerical values and entity slug's purpose	Add a ? symbol that upon hover shows a tooltip explaining what each is.
Image placeholder on the nav bar's user tab redirects to invalid/not implemented route	As a user modification view for non-admin users is not within scope (changing image and user details by yourself), the redirection logic is to be commented out.
Entity slug search does not work using the enter key	Add keybindings and keyboard navigation to future work.

Table 37: Usability testing's issues and corrective measures

8.4.6 Corrections

TODO

8.5 Load Testing

TODO

Chapter 9. System manuals

9.1 Installation Guide

TODO

9.2 User Manual

TODO

9.3 Developer Guide

TODO

Chapter 10. Project Closure

The final project closure analysis combines the schedule execution, budget execution, and risk outcomes as different representations of the same project reality. The deviations observed in the final schedule and budget reflect the actual distribution of effort during execution, while the risk analysis explains the factors that influenced the issues documented on the implementation issues section, which in turn caused those deviations.

10.1 Final Schedule

Once the project was carried out, the schedule was updated with the actual effort and duration recorded for each task, replacing the estimates used during initial planning. Unlike the initial planning, this final schedule distinguishes between **Work** (the actual effort invested in each task) and **Duration** (the elapsed time the task occupied on the calendar), since these values no longer coincide for higher-level summary tasks once real start/end dates and partial dedications are taken into account.

As with the initial planning, this final schedule should not be read as a literal record of the day-by-day timing of execution. Since the project was developed by a single student rather than a complete professional team, work was not necessarily carried out with the parallelization or strict role separation that the modeled professional profiles assume, so the distribution of tasks across the calendar is approximate. The Work values shown here, however, are the actual hours invested in each task; only their placement and overlap on the calendar (not the effort itself) should be read as an approximation aligned with the structure of the initial planning, rather than an exact, audit-level account of daily execution.

ID	Task	Work	Duration	Profile	Start	End
0	**IRBoard development**	310 hrs	250 hrs		Thu 01/01/26	Fri 13/02/26
1	Project management	24 hrs	248 hrs		Thu 01/01/26	Fri 13/02/26
1.1	Design project schedule	2 hrs	2 hrs	Project manager	Thu 01/01/26	Thu 01/01/26
1.2	Generate budget	2 hrs	2 hrs	Project manager	Thu 01/01/26	Thu 01/01/26
1.3	Periodic project management	20 hrs	242 hrs		Fri 02/01/26	Fri 13/02/26
1.3.1	Periodic project management 1	1 hr	1 hr	Project manager	Fri 02/01/26	Fri 02/01/26
1.3.2	Periodic project management 2	2 hrs	2 hrs	Project manager	Mon 05/01/26	Mon 05/01/26
1.3.3	Periodic project management 3	1 hr	1 hr	Project manager	Fri 09/01/26	Fri 09/01/26
1.3.4	Periodic project management 4	2 hrs	2 hrs	Project manager	Mon 12/01/26	Mon 12/01/26
1.3.5	Periodic project management 5	1 hr	1 hr	Project manager	Fri 16/01/26	Fri 16/01/26
1.3.6	Periodic project management 6	1 hr	1 hr	Project manager	Mon 19/01/26	Mon 19/01/26
1.3.7	Periodic project management 7	2 hrs	2 hrs	Project manager	Fri 23/01/26	Fri 23/01/26
1.3.8	Periodic project management 8	2 hrs	2 hrs	Project manager	Mon 26/01/26	Mon 26/01/26
1.3.9	Periodic project management 9	2 hrs	2 hrs	Project manager	Fri 30/01/26	Fri 30/01/26
1.3.10	Periodic project management 10	2 hrs	2 hrs	Project manager	Mon 02/02/26	Mon 02/02/26
1.3.11	Periodic project management 11	1 hr	1 hr	Project manager	Fri 06/02/26	Fri 06/02/26
1.3.12	Periodic project management 12	1 hr	1 hr	Project manager	Mon 09/02/26	Mon 09/02/26
1.3.13	Periodic project management 13	2 hrs	2 hrs	Project manager	Fri 13/02/26	Fri 13/02/26
2	Analysis/Software Requirements	57 hrs	57 hrs		Thu 01/01/26	Mon 12/01/26
2.1	Determine project scope	1 hr	1 hr	System Analyst	Thu 01/01/26	Thu 01/01/26

2.2	Review Regulatory Standards	4 hrs	4 hrs	Technology consultant	Thu 01/01/26	Thu 01/01/26
2.3	Identify required Documentation	1 hr	1 hr	System Analyst	Thu 01/01/26	Thu 01/01/26
2.4	Determine hand-ins for the project	1 hr	1 hr	System Analyst	Thu 01/01/26	Thu 01/01/26
2.5	Adapt project template to typst	2 hrs	2 hrs	System Analyst	Thu 01/01/26	Fri 02/01/26
2.6	Analyze existing systems	6 hrs	6 hrs	Technology consultant	Fri 02/01/26	Fri 02/01/26
2.7	Draft preliminary software requirements	22 hrs	22 hrs	System Analyst	Fri 02/01/26	Wed 07/01/26
2.8	Model auxiliary diagrams	8 hrs	8 hrs	System Analyst	Wed 07/01/26	Thu 08/01/26
2.9	Review software requirements	4 hrs	4 hrs	System Analyst	Thu 08/01/26	Fri 09/01/26
2.10	Modify requirements with feedback	8 hrs	8 hrs	System Analyst	Fri 09/01/26	Mon 12/01/26
3	Analysis complete	0 hrs	0 days		Mon 12/01/26	Mon 12/01/26
4	Design	53 hrs	43 hrs		Mon 12/01/26	Mon 19/01/26
4.1	Design architecture	10 hrs	10 hrs	Software architect	Mon 12/01/26	Tue 13/01/26
4.2	Design brand identity	3 hrs	3 hrs	Junior software engineer	Mon 12/01/26	Mon 12/01/26
4.3	Design project management	8 hrs	8 hrs	Senior software engineer	Mon 12/01/26	Tue 13/01/26
4.4	Design stakeholder management	4 hrs	4 hrs	Senior software engineer	Tue 13/01/26	Tue 13/01/26
4.5	Design requirement management	6 hrs	6 hrs	Senior software engineer	Wed 14/01/26	Wed 14/01/26
4.6	Design user management	12 hrs	12 hrs	Senior software engineer	Wed 14/01/26	Fri 16/01/26

4.7	Design document management and diagram modelling	10 hrs	10 hrs	Senior software engineer	Fri 16/01/26	Mon 19/01/26
5	Design complete	0 hrs	0 days		Mon 19/01/26	Mon 19/01/26
6	Set up SonarQube for Quality Assurance	6 hrs	6 hrs	Senior software engineer	Mon 19/01/26	Tue 20/01/26
7	Development	107 hrs	107 hrs		Tue 20/01/26	Fri 06/02/26
7.1	Set up architecture	6 hrs	6 hrs	Software architect	Tue 20/01/26	Tue 20/01/26
7.2	Set up development enviroment	1 hr	1 hr	Software architect	Wed 21/01/26	Wed 21/01/26
7.3	Develop code	100 hrs	100 hrs		Wed 21/01/26	Fri 06/02/26
7.3.1	Develop project management module	22 hrs	22 hrs	Junior software engineer	Wed 21/01/26	Fri 23/01/26
7.3.2	Develop stakeholder management module	6 hrs	6 hrs	Junior software engineer	Fri 23/01/26	Mon 26/01/26
7.3.3	Develop requirement management module	24 hrs	24 hrs	Senior software engineer	Mon 26/01/26	Thu 29/01/26
7.3.4	Develop user management module	12 hrs	12 hrs	Junior software engineer	Thu 29/01/26	Mon 02/02/26
7.3.5	Develop document management	20 hrs	20 hrs	Senior software engineer	Mon 02/02/26	Wed 04/02/26
7.3.6	Develop modifying concurrency system	10 hrs	10 hrs	Senior software engineer	Wed 04/02/26	Thu 05/02/26
7.3.7	Develop search and filtering	6 hrs	6 hrs	Junior software engineer	Thu 05/02/26	Fri 06/02/26
8	Development complete	0 hrs	0 days		Fri 06/02/26	Fri 06/02/26
9	Testing	35 hrs	121 hrs		Mon 19/01/26	Mon 09/02/26

9.1	Test project management module	6 hrs	6 hrs	Junior software engineer	Tue 27/01/26	Wed 28/01/26
9.2	Test stakeholder management module	4 hrs	4 hrs	Junior software engineer	Mon 26/01/26	Tue 27/01/26
9.3	Test requirement management module	5 hrs	5 hrs	Junior software engineer	Fri 30/01/26	Mon 02/02/26
9.4	Test user management module	3 hrs	3 hrs	Junior software engineer	Mon 02/02/26	Mon 02/02/26
9.5	Test document management	4 hrs	4 hrs	Junior software engineer	Wed 04/02/26	Thu 05/02/26
9.6	Test modifying concurrency system	5 hrs	5 hrs	Junior software engineer	Mon 09/02/26	Mon 09/02/26
9.7	Test search and filtering	3 hrs	3 hrs	Junior software engineer	Fri 06/02/26	Fri 06/02/26
9.8	Usability and accessibility testing	3 hrs	3 hrs	Junior software engineer	Mon 19/01/26	Mon 19/01/26
9.9	Load testing	2 hrs	8 hrs	Senior software engineer (25%)	Thu 22/01/26	Thu 22/01/26
10	Testing complete	0 hrs	0 hrs		Mon 09/02/26	Mon 09/02/26
11	Documentation	28 hrs	230 hrs		Thu 01/01/26	Tue 10/02/26
11.1	Document declaration of originality, abstract and keywords	1 hr	1 hr	Project manager	Thu 01/01/26	Thu 01/01/26
11.2	Document introduction	1 hr	1 hr	Project manager	Thu 01/01/26	Thu 01/01/26
11.3	Document theoretical background	1 hr	1 hr	Technology consultant	Thu 01/01/26	Thu 01/01/26

11.4	Document feasibility study and alternative analysis	2 hrs	2 hrs	Technology consultant	Thu 01/01/26	Thu 01/01/26
11.5	Document Initial project planning and management	3 hrs	3 hrs	Project manager	Fri 02/01/26	Fri 02/01/26
11.6	Document system analysis	2 hrs	2 hrs	System Analyst	Fri 02/01/26	Fri 02/01/26
11.7	Document system design	3 hrs	3 hrs	Software architect	Fri 16/01/26	Fri 16/01/26
11.8	Document system implementation	6 hrs	6 hrs	Senior software engineer	Fri 06/02/26	Mon 09/02/26
11.9	Document test plan execution	2 hrs	2 hrs	Senior software engineer	Mon 09/02/26	Mon 09/02/26
11.10	Document system manuals	5 hrs	5 hrs	Senior software engineer	Mon 09/02/26	Tue 10/02/26
11.11	Document final project closure	1 hr	1 hr	Project manager	Tue 10/02/26	Tue 10/02/26
11.12	Document conclusions and future work	1 hr	1 hr	Project manager	Tue 10/02/26	Tue 10/02/26
12	Documentation complete	0 hrs	0 days		Tue 10/02/26	Tue 10/02/26

Table 38: Project final schedule

Which gives the following Gantt chart:

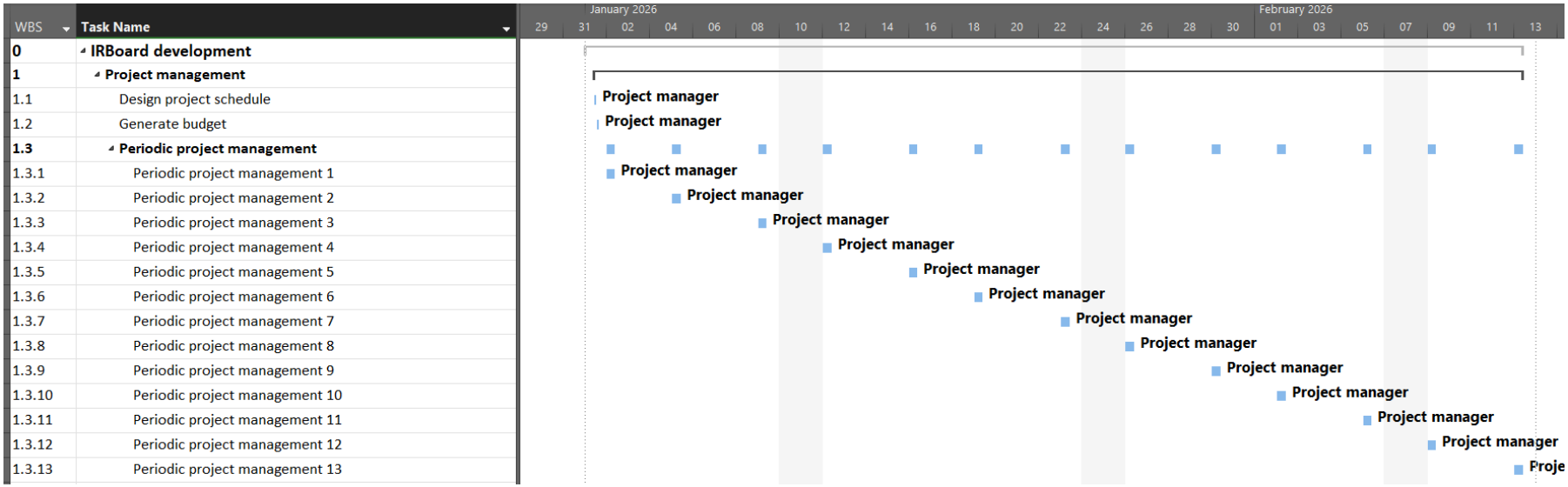

Figure 45: Project management Gantt chart (final)

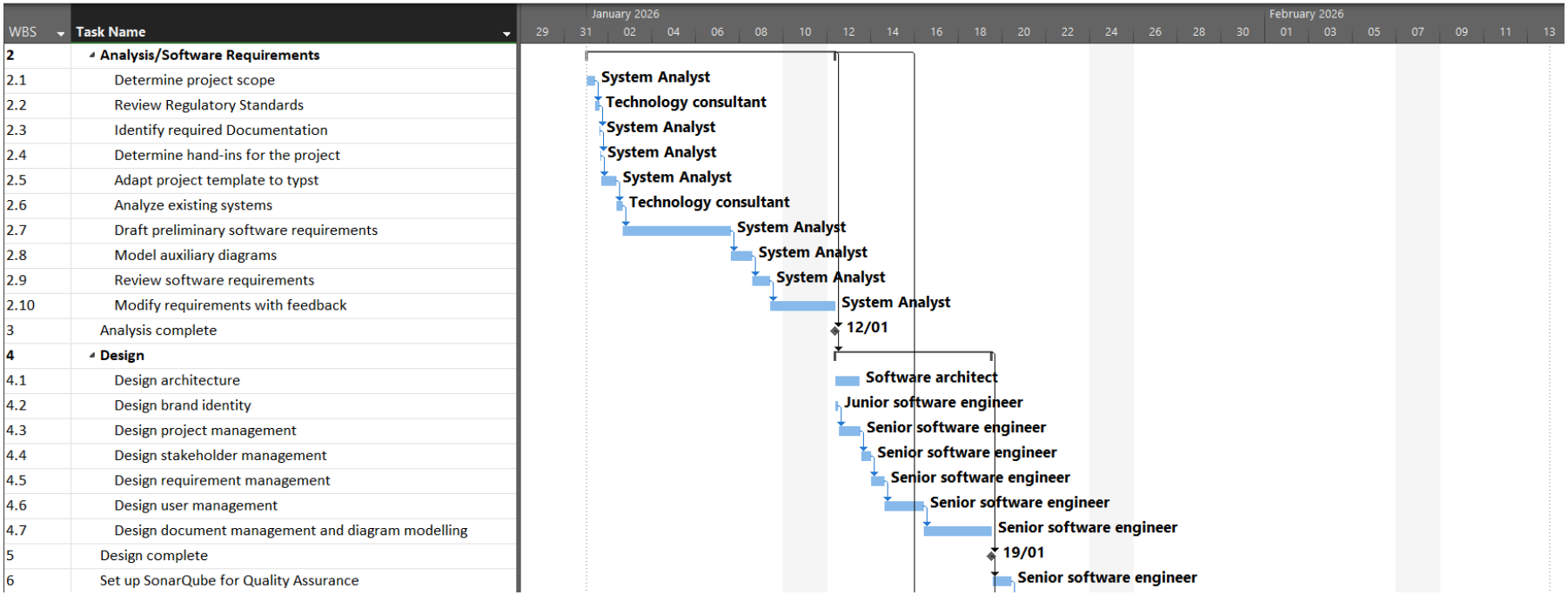

Figure 46: Analysis and design Gantt chart (final)

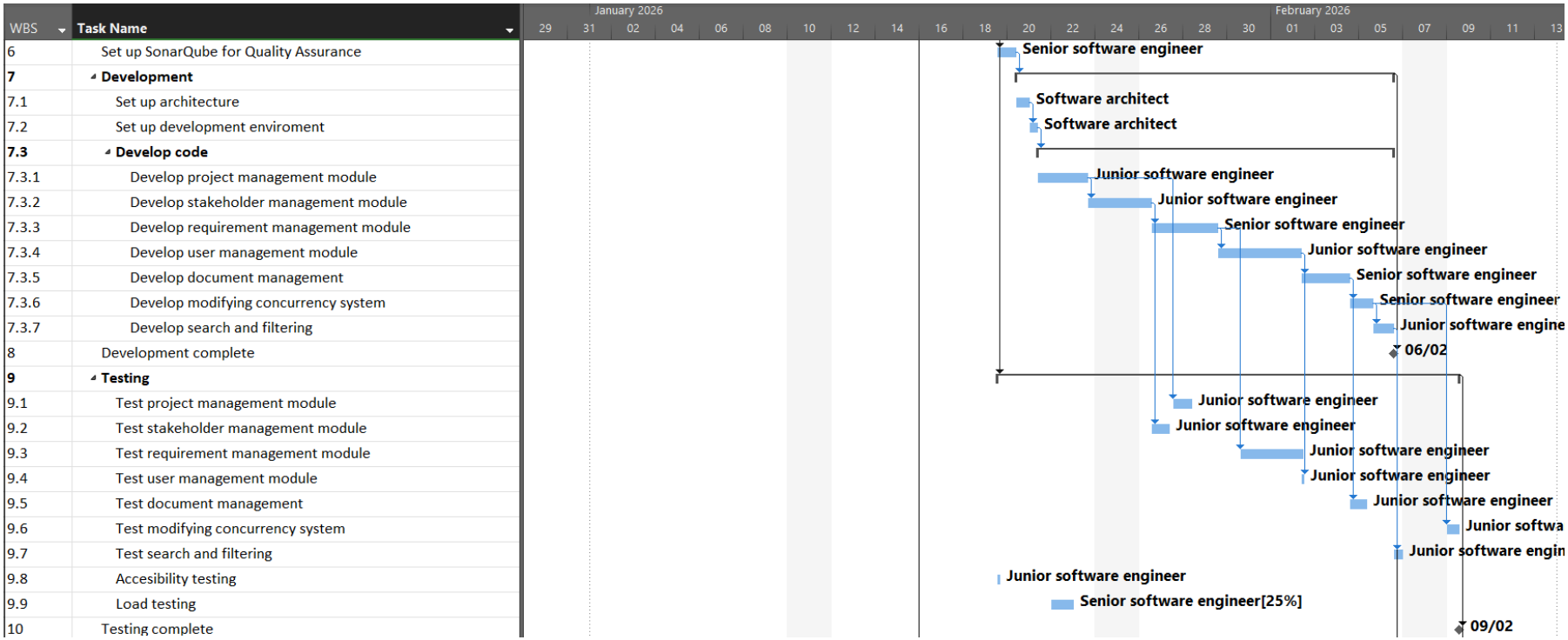

Figure 47: Development and testing Gantt chart (final)

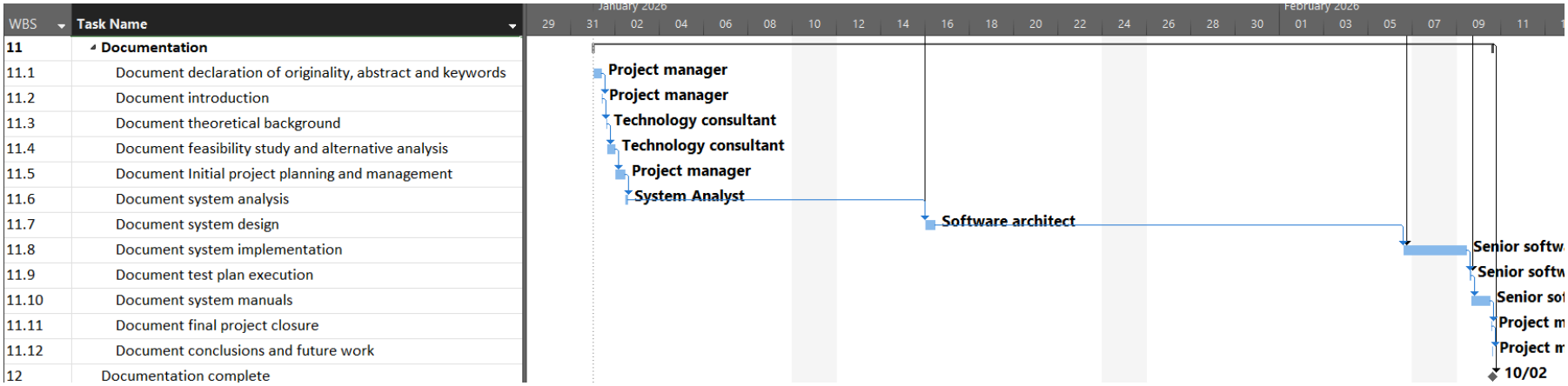

Figure 48: Documentation Gantt chart (final)

The final schedule was updated with the actual effort and duration recorded for each task, replacing the estimates used during the initial planning. Unlike the initial schedule, the final version distinguishes between Work (the actual effort invested) and Duration (the elapsed calendar time), as these values no longer necessarily coincide once real execution conditions and task distribution are considered.

The final schedule recorded 310 hours of total work compared with the initial estimate of 300 hours, representing an overall deviation of +3%. However, this global value hides several redistributions between project phases.

Project management decreased from 30 to 24 hours (-20%). The reduction was mainly concentrated in periodic management activities, where several planned sessions required less effort than originally estimated. The final execution required fewer coordination activities, resulting in a lower overall management workload.

The analysis phase increased from 54 to 57 hours (+6%). The main deviations occurred in modelling and requirement review activities. The auxiliary diagram modelling task increased from 6 to 8 hours, while software requirements review increased from 2 to 4 hours. These changes reflect a greater effort required to refine and structure the analysis documentation.

Design increased from 51 to 53 hours (+4%). Most activities remained close to their original estimates, with the largest variation occurring in user management design, which increased from 10 to 12 hours due to the additional complexity involved in defining the required structures and interactions.

The setup of quality assurance tooling increased from 3 to 6 hours (+100%), representing the largest relative deviation. This additional effort was associated with configuring and integrating the required quality assurance environment.

Development increased slightly from 105 to 107 hours (+2%). Although the overall phase remained stable, several internal adjustments occurred. Stakeholder management implementation decreased from 10 to 6 hours, and search functionality decreased from 10 to 6 hours. In contrast, user management increased from 6 to 12 hours, and document management increased from 15 to 20 hours. These variations balanced each other, resulting in a development phase close to the original estimate.

Testing increased from 27 to 35 hours (+29%), making it the largest phase deviation. Most testing activities required additional validation effort compared with the initial estimates. The most notable increase occurred in the concurrency system testing, which grew from 2 to 5 hours. The additional testing effort ensured that the implemented functionality met the expected quality standards.

Documentation decreased from 30 to 28 hours (-7%). The reduction was mainly concentrated in system design documentation and final closure activities, while the remaining documentation tasks remained aligned with their initial estimates.

Overall, the final schedule demonstrates that the project maintained strong alignment with the original planning. The deviations were mainly caused by differences in task-level complexity and effort distribution. Increased effort in technical and validation activities was compensated by reductions in management and documentation activities, resulting in a final execution close to the original estimate.

10.2 Final Risk Report

This section evaluates the outcome of the risks and opportunities identified during the planning phase (Risk analysis). It assesses the extent to which each risk or opportunity materialized, the effectiveness of the planned response strategies, and their overall impact on project execution.

The probability and impact assessments defined during planning are not repeated here. Instead, this section focuses on the actual events observed during development and on the effectiveness of the mitigation and exploitation strategies that were applied.

As the project was carried out by a single developer while simultaneously completing academic obligations and a professional internship, the observations presented below should be interpreted as qualitative assessments of project execution rather than precise quantitative measurements.

10.2.1 Modification of weekly working schedule : Materialized (significant but controlled)

This risk materialized throughout the project and represented the most visible deviation from the original plan. The simultaneous completion of the Bachelor's Degree and the external professional internship resulted in highly irregular weekly availability, with several periods of reduced activity and one particularly significant hiatus during the internship phase.

The mitigation strategy defined during planning proved effective. Rather than attempting to maintain a rigid weekly workload, development effort was redistributed across subsequent periods of higher availability. Although this introduced fluctuations in short-term progress, it avoided the need for scope reduction, additional resources, or major replanning of project objectives.

Consequently, while the risk materialized substantially, its impact remained primarily limited to schedule variability and did not compromise the final scope or quality of the delivered system.

10.2.2 Erroneous identification of requirements : Did not materialize

This risk did not materialize in any significant way during the project.

Although requirements engineering was initially identified as a potential source of uncertainty due to the developer's limited prior experience in formal requirements management, the continuous involvement of the project tutors throughout the analysis and validation phases proved sufficient to prevent misunderstandings from propagating into later stages of development.

Periodic reviews, clarification meetings, and iterative refinement of the requirements specification ensured that the functional and non-functional requirements remained stable throughout implementation. As a result, no substantial rework attributable to incorrect or conflicting requirements was required.

This outcome confirms the effectiveness of the mitigation strategy defined during planning and highlights the value of frequent stakeholder validation in requirements-intensive projects.

10.2.3 Lack of experience with the Ory ecosystem and ReBAC implementation: Materialized (low impact)

This risk materialized only to a limited extent during the initial integration phases.

Although the developer had no previous experience with the Ory ecosystem, the availability of official documentation, reference examples, and assisted problem-solving tools significantly reduced the expected learning curve. As a result, understanding the main concepts behind Ory Kratos, Ory Oathkeeper, and Ory Keto, as well as the fundamentals of Relation-Based Access Control, did not become a major source of delay.

The main challenges were not related to understanding the technologies themselves, but rather to correctly configuring and integrating them within the project's specific infrastructure. In particular, ensuring that the different components communicated correctly and that the authorization model matched the intended application behaviour required some additional experimentation.

The planned mitigation strategy based on early prototyping, documentation review, and incremental validation proved effective. The additional effort was contained within the security and infrastructure-related tasks and did not significantly affect the project schedule, scope, or quality objectives.

10.2.4 Incompatibility of expected workflow with the Ory ecosystem: Partially materialized (mitigated through architectural adaptation)

This risk partially materialized during implementation.

The conceptual authorization model designed for IR-Board was generally compatible with the capabilities provided by Ory Keto and Ory Oathkeeper. The intended Relation-Based Access Control model could be expressed using the selected components, and no fundamental limitation prevented the implementation of the planned permission system. However, several practical integration difficulties appeared when combining the application workflows with the operational requirements of the Ory ecosystem. The main challenge was not the authorization model itself, but rather the complexity of correctly configuring the surrounding infrastructure: service communication, network segmentation, reverse proxy behaviour, authentication flows, and the boundaries between protected and internal services.

The use of Ory Oathkeeper required additional architectural decisions regarding which components should be placed behind the authorization gateway and which should remain outside of it. Determining the appropriate request flow and service placement required additional experimentation, mainly due to the developer's limited previous experience with multi-layered security-oriented architectures. Examples of that particular situation occurred when introducing additional infrastructure components, particularly the observability stack and object storage. These services required careful consideration of trust boundaries, network exposure, and communication patterns. Their integration was more complex than initially expected, as securely separating user-facing traffic from internal service interactions required a stronger understanding of network segmentation and service isolation principles.

Despite these challenges, the architectural decisions adopted during development successfully contained the impact of this risk. The abstraction between the application domain and external infrastructure services allowed adjustments to be introduced without requiring significant changes to the core system design. Consequently, the risk resulted mainly in additional configuration effort and reduced extension capacity rather than functional limitations or project failure.

10.2.5 Opportunity: Acceleration and robustness through Ory ecosystem integration : Realized (high positive impact)

Despite the difficulties associated with learning and integrating the Ory ecosystem, the opportunity identified during planning was ultimately realized.

Once the initial configuration and architectural challenges had been overcome, the use of Ory Kratos, Ory Oathkeeper, and Ory Keto significantly reduced the amount of custom security code required within the application itself. Authentication, session management, authorization enforcement, and relationship-based permission evaluation could be delegated to specialized components designed specifically for those responsibilities.

This allowed development effort to remain focused on the core objectives of the project, including requirements lifecycle management, traceability, collaboration features, and workflow automation. Furthermore, the resulting architecture exhibits a clearer separation of concerns and follows security practices closer to those used in modern production environments.

Although the ecosystem introduced additional complexity during development, the final solution benefits from improved maintainability, stronger security guarantees, and greater architectural modularity than would likely have been achieved through a custom implementation.

10.2.6 Additional risk and opportunity encountered: Typst documentation workflow: Materialized (risk mitigated, opportunity exploited)

An additional risk and opportunity that was not identified during the initial planning phase emerged from the adoption of Typst as the documentation technology.

The decision to use Typst was initially motivated by the potential improvement over traditional documentation workflows. Its modern syntax, Markdown-like structure, fast compilation times, and extensibility provided a significantly more efficient environment for producing formal academic documentation compared with more traditional alternatives. This opportunity was successfully exploited, reducing the time required for writing, formatting, and maintaining the project documentation.

However, the adoption of a relatively new documentation ecosystem also introduced an unforeseen risk. During the preparation of the requirements documentation, a compatibility issue was encountered due to recent changes in Typst's support for nested lists. Since the requirements specification relied on hierarchical structures, such as identifiers following formats like UM.1.1, the absence of native multilevel list support created difficulties in maintaining the intended document structure.

Resolving this issue required additional investigation and adaptation of the documentation workflow. The extensibility of Typst proved essential in this situation, as external extensions and custom solutions allowed the required nested list behaviour to be restored without changing the documentation structure or reducing its quality.

Therefore, although the risk materialized during the project, its impact was limited and successfully mitigated. The final result confirmed both aspects of the decision: Typst introduced an unexpected compatibility challenge, but it also provided a substantial productivity improvement that outweighed the additional effort required to address the issue.

10.2.7 Risk and Opportunity Closure Summary

Risk / Opportunity	Outcome	Observed Impact
Modification of weekly working schedule	Materialized and mitigated	Irregular workload distribution caused by internship and academic commitments, controlled through schedule adaptation
Erroneous identification of requirements	Did not materialize	No significant impact due to continuous validation and requirement reviews
Lack of experience with Ory ecosystem and ReBAC	Materialized and contained	Limited additional effort thanks to documentation, examples, and early experimentation
Workflow incompatibility with Ory ecosystem	Partially materialized and mitigated	Additional infrastructure and integration complexity, addressed through architectural adaptation
Acceleration and robustness through Ory integration	Fully realized	Improved security architecture and reduced custom implementation effort
Typst documentation workflow	Risk mitigated/ Opportunity realized	Minor compatibility issues with nested lists, outweighed by faster documentation development

Table 39: Final status of identified risks and opportunities

10.2.8 Overall Conclusion

The risk management strategy defined during planning proved appropriate for the project's constraints. The main risk, the modification of the weekly working schedule, materialized due to the combination of academic and professional commitments, but its impact was controlled through workload redistribution.

The requirements-related risk did not significantly occur thanks to continuous validation, while the Ory-related risks mainly resulted in additional infrastructure and integration complexity rather than architectural limitations. These challenges were contained through adaptation of the system design and incremental validation.

The main planned opportunity was successfully exploited: the Ory ecosystem reduced the need for custom security logic and enabled a robust authorization architecture.

In addition, an unplanned factor emerged from the adoption of Typst as the documentation tool. Although not identified during risk planning, it introduced both minor compatibility challenges and a significant improvement in documentation efficiency, effectively acting as both a small operational risk and a productivity opportunity.

Overall, no risk required additional budget, scope reduction, or contingency activation, and all identified risks and opportunities remained within manageable limits.

10.3 Budget execution analysis

As noted in the initial budget, the provider's financial reality (the annual employment cost and hourly rates for each professional profile) depends on the structural cost of the organization rather than on the specific execution of this project, and therefore remains unchanged. For this reason it is not repeated here.

What follows is the recalculation of the cost breakdown based on the actual hours invested during execution, together with a deviation analysis of each budget line.

10.3.1 Final project cost

This section lays out the final cost structure, recalculated with the actual hours recorded for each task, and the corresponding proportional amount to be diluted.

Category 1: IrBoard development costs breakdown											
I1	**I2**	**I3**	**Description**	**Resource**	**Quantity**	**Units**	**Price**	**Subtotal**	**Subtotal (2)**	**Total**	**Deviation**
01			Project management							1.194,77 €	−20%
	001		Design project schedule	Project manager	2	Hours	49,78 €	99,56 €	99,56 €		0%
	002		Generate budget	Project manager	2	Hours	49,78 €	99,56 €	99,56 €		0%
	003		Periodic project management						995,64 €		−23%
		01	Periodic project management 1	Project manager	1	Hours	49,78 €	49,78 €			−50%
		02	Periodic project management 2	Project manager	2	Hours	49,78 €	99,56 €			0%
		03	Periodic project management 3	Project manager	1	Hours	49,78 €	49,78 €			−50%
		04	Periodic project management 4	Project manager	2	Hours	49,78 €	99,56 €			0%
		05	Periodic project management 5	Project manager	1	Hours	49,78 €	49,78 €			−50%
		06	Periodic project management 6	Project manager	1	Hours	49,78 €	49,78 €			−50%
		07	Periodic project management 7	Project manager	2	Hours	49,78 €	99,56 €			0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
		08	Periodic project management 8	Project manager	2	Hours	49,78 €	99,56 €			0%
		09	Periodic project management 9	Project manager	2	Hours	49,78 €	99,56 €			0%
		10	Periodic project management 10	Project manager	2	Hours	49,78 €	99,56 €			0%
		11	Periodic project management 11	Project manager	1	Hours	49,78 €	49,78 €			−50%
		12	Periodic project management 12	Project manager	1	Hours	49,78 €	49,78 €			−50%
		13	Periodic project management 13	Project manager	2	Hours	49,78 €	99,56 €			0%
02			Analysis / Software Requirements							2.433,69 €	5%
	001		Determine project scope	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €		0%
	002		Review Regulatory Standards	Technology consultant	4	Hours	45,22 €	180,88 €	180,88 €		0%
	003		Identify required Documentation	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €		−50%
	004		Determine hand-ins for the project	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €		0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
	005		Adapt project template to Typst	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €		0%
	006		Analyze existing systems	Technology consultant	6	Hours	45,22 €	271,32 €	271,32 €		0%
	007		Draft preliminary software requirements	System Analyst	22	Hours	42,16 €	927,50 €	927,50 €		0%
	008		Model auxiliary diagrams	System Analyst	8	Hours	42,16 €	337,27 €	337,27 €		33%
	009		Review software requirements	System Analyst	4	Hours	42,16 €	168,64 €	168,64 €		100%
	010		Modify requirements with feedback	System Analyst	8	Hours	42,16 €	337,27 €	337,27 €		0%
03			Design							2.361,06 €	4%
	001		Design architecture	Software architect	10	Hours	49,15 €	491,54 €	491,54 €		0%
	002		Design brand identity	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €		0%
	003		Design project management	Senior software engineer	8	Hours	43,85 €	350,77 €	350,77 €		0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
	004		Design stakeholder management	Senior software engineer	4	Hours	43,85 €	175,39 €	175,39 €		0%
	005		Design requirement management	Senior software engineer	6	Hours	43,85 €	263,08 €	263,08 €		0%
	006		Design user management	Senior software engineer	12	Hours	43,85 €	526,16 €	526,16 €		20%
	007		Design document management	Senior software engineer	10	Hours	43,85 €	438,47 €	438,47 €		0%
04			Set up SonarQube for Quality Assurance	Senior software engineer	6	Hours	43,85 €	263,08 €	263,08 €	263,08 €	100%
05			Development							4.485,18 €	2%
	001		Set up architecture	Software architect	6	Hours	49,15 €	294,92 €	294,92 €		50%
	002		Set up development environment	Software architect	1	Hours	49,15 €	49,15 €	49,15 €		−67%
	003		Develop code						4.141,10 €		2%
		01	Develop project management module	Junior software engineer	22	Hours	38,55 €	848,14 €			0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
		02	Develop stakeholder management module	Junior software engineer	6	Hours	38,55 €	231,31 €			−40%
		03	Develop requirement management module	Senior software engineer	24	Hours	43,85 €	1.052,32 €			−4%
		04	Develop user management module	Junior software engineer	12	Hours	38,55 €	462,62 €			100%
		05	Develop document management module	Senior software engineer	20	Hours	43,85 €	876,93 €			33%
		06	Develop modifying concurrency system	Senior software engineer	10	Hours	43,85 €	438,47 €			0%
		07	Develop search and filtering	Junior software engineer	6	Hours	38,55 €	231,31 €			−40%
06			Testing							1.359,90 €	29%
	001		Test project management module	Junior software engineer	6	Hours	38,55 €	231,31 €	231,31 €		50%
	002		Test stakeholder management module	Junior software engineer	4	Hours	38,55 €	154,21 €	154,21 €		33%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
	003		Test requirement management module	Junior software engineer	5	Hours	38,55 €	192,76 €	192,76 €		0%
	004		Test user management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €		0%
	005		Test document management module	Junior software engineer	4	Hours	38,55 €	154,21 €	154,21 €		33%
	006		Test modifying concurrency system	Junior software engineer	5	Hours	38,55 €	192,76 €	192,76 €		150%
	007		Test search and filtering	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €		0%
	008		Usability and accessibility testing	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €		50%
	009		Load testing	Senior software engineer	2	Hours	43,85 €	87,69 €	87,69 €		0%
07			Documentation							1.285,92 €	−7%
	001		Document declaration of originality, abstract and keywords	Project manager	1	Hours	49,78 €	49,78 €	49,78 €		0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
	002		Document introduction	Project manager	1	Hours	49,78 €	49,78 €	49,78 €		0%
	003		Document theoretical background	Technology consultant	1	Hours	45,22 €	45,22 €	45,22 €		0%
	004		Document feasibility study and alternative analysis	Technology consultant	2	Hours	45,22 €	90,44 €	90,44 €		0%
	005		Document initial project planning and management	Project manager	3	Hours	49,78 €	149,35 €	149,35 €		50%
	006		Document system analysis	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €		−33%
	007		Document system design	Software architect	3	Hours	49,15 €	147,46 €	147,46 €		−40%
	008		Document system implementation	Senior software engineer	6	Hours	43,85 €	263,08 €	263,08 €		20%
	009		Document test plan execution	Senior software engineer	2	Hours	43,85 €	87,69 €	87,69 €		0%
	010		Document system manuals	Senior software engineer	5	Hours	43,85 €	219,23 €	219,23 €		0%

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total	Deviation
	011		Document final project closure	Project manager	1	Hours	49,78 €	49,78 €	49,78 €		−50%
	012		Document conclusions and future work	Project manager	1	Hours	49,78 €	49,78 €	49,78 €		0%
Total										13.383,61 €	3%

Table 40: Budget execution with deviations

Given that the second category was not linked to actual execution but rather theoretical assumptions, we can refrain from repeating the budget line as no deviations are recorded.

Therefore we obtain the following:

IrBoard costs			
Cat. Num	**Category**	**Total**	**Deviation**
01	IrBoard development costs breakdown	13.383,61 €	3%
02	Other	172,50 €	0%
total		13.556,11 €	3%

Table 41: Provider's final budget summary

Which gives us an **actual profit** of 2.944,40€, a deviation of −11%, in other words, instead of a profit of 25% of the total costs, a profit of 22%.

10.4 Project Closure Analysis

The final distribution of effort shows that the deviations were not caused by a general increase in project scope, but by changes in the relative complexity of individual activities. Some areas required additional work, while others required less effort than initially estimated, producing a redistribution of resources between phases.

The analysis phase increased from 54 to 57 hours, mainly due to additional effort in requirement modelling and review activities. This was reflected in the final cost of the analysis category, which increased by approximately +5%.

The design phase increased from 51 to 53 hours (+4%), with the main variation concentrated in user management design. The additional effort required for this activity was reflected both in the final schedule and in the corresponding budget line.

The quality assurance setup presented the highest relative deviation. SonarQube configuration increased from 3 to 6 hours (+100%), producing the same deviation in its budget line. Although this represented a large percentage change at task level, its contribution to the overall project deviation was limited due to the small size of the activity.

Development remained close to the original estimation, increasing from 105 to 107 hours (+2%). However, the final execution showed significant redistribution between modules. User management and document management required more effort than planned, while stakeholder management and search functionality required less effort. These changes were reflected in the final cost distribution of the development activities.

Testing showed the largest phase-level deviation, increasing from 27 to 35 hours (+29%). The additional validation effort was reflected in the increased testing cost and represents one of the main contributors to the final deviation.

Documentation decreased from 30 to 28 hours (-7%), producing a corresponding reduction in its final budget. This offset part of the additional effort required in technical phases.

The risk outcomes provide the explanation behind these deviations. The risks that materialized were mainly related to areas where additional effort was eventually required, such as technology integration, infrastructure configuration, and validation activities. These effects appeared simultaneously in the schedule and budget because they were different measurements of the same execution changes. Likewise, risks that did not materialize corresponded to areas where the initial assumptions remained valid and therefore did not generate relevant deviations.

Chapter 11. Conclusions and Future Work

11.1 Conclusions

Overall, the project closure confirms that the initial planning was sufficiently accurate at a global level. The final deviations resulted from variations in task complexity and execution conditions rather than changes in objectives or scope. The project maintained its planned functionality and quality goals while keeping schedule and budget deviations within acceptable limits.

TODO

11.2 Keyboard navigation and sound feedback

During the usability and accessibility testing, it was observed that the system would benefit from keyboard-based navigation and shortcut support, both to improve general workflow efficiency and to reduce reliance on a pointing device for users who prefer or require keyboard-driven interaction.

Additionally, the addition of subtle sound feedback tied to key operations such as saving, approving, or locking an entity could reinforce the sense of action completion and improve the overall user experience. **SoundCN** is proposed as a candidate library for this addition.

Both improvements are outside the scope of this project and are proposed as future extensions.

11.3 Extended E2E scenario coverage

The E2E test suite implemented within the scope of this project covers the core requirements engineering lifecycle and the authentication flow, as documented in the test plan specification. Several scenarios identified during planning were not implemented due to time constraints and are proposed here as future work. It should be noted that all of the areas described below have been manually validated during development and are confirmed to work correctly in the deployed system; the absence of automated coverage represents a gap in regression safety rather than unverified functionality.

The most significant gap is the absence of multi-user and access control scenarios. The current suite exercises only the administrator role, meaning that the permission boundaries between project managers, requirement engineers, and stakeholder users have not been validated end-to-end against the live Ory Keto authorization layer. Future E2E tests should cover user invitation and the signup flow triggered by a received signup code, linking a user to a project functionality in each available role, verifying that each role can access only the operations permitted by the ReBAC model, and confirming that cross-functionality access restrictions are correctly enforced at the infrastructure level rather than only at the frontend.

The concurrency control mechanism also lacks E2E coverage, despite having been manually verified to behave correctly during development. Testing this scenario in an automated context requires two browser contexts to be active simultaneously against the same deployment, which Playwright supports natively through its multi-page and browser context APIs. A future test should verify that a second user

attempting to edit an entity already held by another user is presented with the lock indicator, that the lock is released when the first user saves or navigates away, and that changes submitted by a non-holding user are correctly rejected by the backend.

Finally, the slug-based search navigation flow has not been covered at the E2E level, though it has been manually exercised across all supported entity types. A future scenario should verify that entering a valid slug in the NavBar search field navigates the user to the correct entity detail view, and that invalid slugs, access-restricted slugs, and partial slugs produce the correct feedback states described in the navigability diagram.

11.4 Load testing

Load testing was planned as part of the test strategy but could not be executed within the available project timeline. Manual observation during development suggests that the system responds within acceptable latency bounds under light, single-user load, but no systematic measurement has been performed under concurrent usage conditions. The following describes the intended approach, which is proposed as future work.

The selected tool is **K6**, an open-source, script-based load testing framework that integrates well with the Grafana observability stack already deployed as part of the system. K6 allows virtual user scenarios to be defined as JavaScript scripts, executed from the command line, and configured with ramp-up profiles, sustained load phases, and burst scenarios using its built-in executor model.

Tests should be executed directly against the full Docker Compose stack, targeting the critical path that passes through Traefik and Oathkeeper into the Spring Boot backend, since this chain carries the highest per-request overhead due to the session validation call to Kratos and the permission check against Keto on every authorized request. A representative scenario would simulate a realistic number of concurrent authenticated users performing a mix of read and write operations across different project entities, reflecting the expected usage pattern of a small development team working simultaneously on the same project.

The primary metrics to be collected and analyzed are mean and percentile request latency at each layer, error rate under sustained and burst load, session management stability in Kratos, and container-level resource utilization as observed through the Grafana and Prometheus stack already provisioned. The Loki log aggregation layer should also be monitored during test runs to surface any error patterns or retry storms that do not appear in the top-level latency metrics.

A minimum acceptance threshold should be defined before running the tests, for example a p95 latency below 500ms under a load representative of the expected user base, so that results can be evaluated against a concrete criterion rather than interpreted subjectively.

11.5 Dependency maturity: migration away from MinIO

At the time of writing, MinIO's open-source Community Edition is in a fragile position. Following the removal of administrative functionality from its web console in

2025 and the subsequent transition of the upstream project to “maintenance mode” in December 2025, MinIO is no longer actively developed as a community project, even though its AGPLv3-licensed source remains available and usable.

Since IR-Board’s `object-storage` service is used purely as an S3-compatible backend for document persistence, accessed entirely through the standard S3 API and the `mc` CLI rather than through the now-deprecated web console, the immediate functional impact on the project is limited. However, relying on an unmaintained dependency for production deployment is not a sustainable long-term decision, particularly regarding unpatched security vulnerabilities.

As future work, two paths are proposed:

- **Pin and isolate the current dependency.** In the short term, replace the unpinned `minio/minio:latest` and `minio/mc:latest` images used in the current deployment with explicit, audited version tags, to avoid unpredictable behaviour from future upstream changes to an effectively frozen codebase.
- **Evaluate actively maintained alternatives.** In the medium term, migrate the object storage layer to an actively maintained, S3-compatible alternative, such as Garage, SeaweedFS, or Ceph’s RGW component. Given that the backend already interacts with object storage exclusively through the standard S3 API using a driver, this migration is expected to require minimal changes to the application layer, mainly limited to deployment configuration and credentials management.

11.6 Standardization of frontend data-fetching

The current frontend implementation relies on a `useBackendResource`-style hook pattern for interacting with the backend API, but error handling is not yet centralized. As a result, different parts of the application may handle failed requests inconsistently, with no guarantee that all unrecoverable errors lead to the same user-facing behaviour.

As future work, a custom fetch wrapper is proposed, shared across all data-fetching hooks, that intercepts failed requests and uniformly redirects the user to the `/error` page (or an equivalent contextual error state) according to the type of failure encountered. This would centralize error handling logic, reduce duplicated boilerplate across components, and ensure a consistent user experience regardless of which part of the system triggered the failure.

11.7 Hardening of cross-service consistency on partial API failures

Several operations exposed by the backend involve coordinated changes across more than one external service, most notably the combination of the relational database and the object storage backend used for document management. These operations are not currently wrapped in a distributed transaction or compensating-action mechanism, meaning that a failure partway through a multi-step operation can leave the system in an inconsistent, though recoverable, state.

A representative example is the bulk deletion of documents linked to a project: if the deletion of a given document succeeds in the database but the corresponding object fails to be removed from MinIO (or vice versa), documents already processed before the failure remain deleted, while the failing document and any subsequent

ones in the batch remain present in one system without their counterpart in the other. The system does not currently detect or repair this kind of divergence automatically; recovering from it requires manual intervention.

As future work, this should be addressed either through an explicit compensating-transaction (saga) pattern around multi-step operations, or through a periodic reconciliation process that detects and reports orphaned records between the database and object storage. At minimum, this limitation should be made explicit to administrators through observability tooling, so that inconsistencies can be identified and resolved before they affect traceability guarantees.

11.8 Adoption of presigned URLs for document transfer

The current implementation of document upload and download routes the full file content through the backend API, meaning that documents are passed through the Spring Boot service before reaching, or after leaving, the object storage backend. While functional, this is not the standard approach for systems built around an S3-compatible object storage layer, and was a result of limited prior experience with object storage integration patterns at the time the document management module was designed.

The industry-standard approach is to use presigned URLs, generated by the backend (which holds the necessary credentials) but used by the client to upload or download a document directly against the object storage service, without the file ever passing through the backend itself. This reduces backend load, avoids unnecessary memory and bandwidth usage on an intermediate service, and is the pattern generally expected when integrating with S3-compatible storage.

As future work, document upload and download flows should be refactored to use presigned URLs for both storing and retrieving objects, with the backend's role limited to authorization, presigned URL generation, and persistence of document metadata.

11.9 Adoption of a safer deployment strategy

The current deployment model, based on a single Docker Compose stack brought up and down as a whole, follows what is effectively a big-bang deployment strategy: all services are replaced simultaneously, with no intermediate state between the previous and new versions of the system. While acceptable for the development and demonstration context of this project, this approach is not appropriate for a production environment, as it offers no rollback path, no traffic shifting, and a non-trivial downtime window during redeployment.

As future work, a more resilient deployment strategy should be adopted, such as a blue-green or rolling deployment model, where a new version of the system is brought up alongside the previous one and traffic is gradually or atomically switched over only once the new version is confirmed healthy. This would also be a natural point to reconsider container orchestration beyond Docker Compose, as discussed as outside the scope of this project in the Scope section, since orchestrators such as Kubernetes or Docker Swarm provide native primitives for these deployment strategies.

11.10 Completion of the draw.io integration

The self-hosted draw.io (diagrams.net) container and its supporting infrastructure, including routing through Traefik, CORS configuration, and the corresponding frontend embedding points, are already fully provisioned as part of the system's architecture. However, the integration was not brought to a fully operational state by the end of this project, as development time was prioritized toward ensuring testing coverage, usability testing and the observability stack were present and complete instead.

As future work, the remaining effort consists of completing and validating the end-to-end integration of the self-hosted draw.io instance with the rest of the platform, confirming that diagrams can be created, embedded, and persisted correctly from within the frontend, and that the existing infrastructure configuration (CORS, routing, and iframe embedding) behaves as intended in practice. No architectural changes are expected to be required; this is treated as a completion task rather than an open design problem.

11.11 Completion of requirements export to PDF

Due to time constraints during development, this functionality was not implemented within the scope of this project, even though it was a planned extension of the scope. Currently, a project manager has no built-in way to generate a portable, shareable snapshot of a project's requirements outside the platform itself.

As future work, this export capability should be implemented, most likely by composing a structured PDF document from the existing requirement, functionality, and stakeholder data already exposed by the backend, following a presentation format suitable for external stakeholders who do not have direct access to the platform. This would also be a natural point to consider exposing additional export formats (such as a traceability matrix or a Word document), aligned with the documentation standards discussed in the Theoretical Background section.

Chapter 12. References

- [1] “Welcome to Ory! | Ory,” Ory.com, Oct. 15, 2025. https://www.ory.com/docs/ (accessed Mar. 07, 2026).
- [2] “Configuring Vite,” vitejs, 2025. https://vite.dev/config/
- [3] “Typst Documentation,” Typst, 2024. https://typst.app/docs/
- [4] “hexagonal-architecture.” https://alistair.cockburn.us/hexagonal-architecture
- [5] R. C. Martin, Clean architecture: A Craftsman’s Guide to Software Structure and Design. Pearson Professional, 2018.
- [6] “Ory Permission Language specification | Ory,” Jun. 17, 2024. https://www.ory.com/docs/keto/reference/ory-permission-language

Chapter 13. Appendices

13.1 Budget

13.1.1 Provider finantial reality

Personnel	Num	Gross annual salary	annual employment cost	total
General manager	1	76.243,00 €	91.781,35 €	91.781,35 €
Project manager	1	40.550,00 €	53.039,40 €	53.039,40 €
Service coordinator	1	25.135,00 €	32.876,58 €	32.876,58 €
Systems analyst	1	35.574,00 €	46.530,79 €	46.530,79 €
Technology consultant	1	41.988,00 €	54.920,30 €	54.920,30 €
Software architect	1	44.369,00 €	58.034,65 €	58.034,65 €
Senior developer	2	39.879,00 €	52.161,73 €	104.323,46 €
Junior developer	4	25.872,00 €	33.840,58 €	135.362,32 €
Sales representative	1	26.849,00 €	35.118,49 €	35.118,49 €
TOTAL	13			611.987,34 €

Table 42: Initial Budget: Personnel basic information

Personnel	TOTAL	Prod (%)	Direct cost	IC (%)	Indirect cost
General manager	91.781,35 €	0,00%	-	100,00%	91.781,35 €
Project manager	53.039,40 €	65,00%	34.475,61 €	35,00%	18.563,79 €
Service coordinator	32.876,58 €	50,00%	16.438,29 €	50,00%	16.438,29 €
Systems analyst	46.530,79 €	80,00%	37.224,63 €	20,00%	9.306,16 €
Technology consultant	54.920,30 €	85,00%	46.682,26 €	15,00%	8.238,05 €
Software architect	58.034,65 €	75,00%	43.525,99 €	25,00%	14.508,66 €
Senior developer	52.161,73 €	85,00%	44.337,47 €	15,00%	7.824,26 €
Junior developer	33.840,58 €	70,00%	23.688,41 €	30,00%	10.152,17 €
Sales representative	35.118,49 €	0,00%	-	100,00%	35.118,49 €

Personnel	TOTAL	Prod (%)	Direct cost	IC (%)	Indirect cost
TOTAL	458.303,87 €		246.372,66 €		211.931,22 €

Table 43: Initial Budget: Allocation of personnel costs between direct and indirect costs

The Indirect Cost (CI) percentage stands for the percentage of the professional profile's workload related to non project-specific tasks, like general management or human resources, for example.

Service	Monthly cost	Annual cost
Cleaning	850,00 €	10.200,00 €
Advertising	800,00 €	9.600,00 €
Accounting services	550,00 €	6.600,00 €
Quality control	400,00 €	4.800,00 €
Taxes and fees	120,00 €	1.440,00 €
Rent / lease	1.350,00 €	16.200,00 €
Vehicle leasing fees	900,00 €	10.800,00 €
Maintenance, repair and upkeep	140,00 €	1.680,00 €
Small tools and equipment	120,00 €	1.440,00 €
Electricity consumption	120,00 €	1.440,00 €
Heating energy consumption	180,00 €	2.160,00 €
Water consumption	55,00 €	660,00 €
Consulting, auditing and professional fees	400,00 €	4.800,00 €
Insurance premiums	300,00 €	3.600,00 €
Freight and transportation costs	150,00 €	1.800,00 €
Communications expenses	500,00 €	6.000,00 €
Travel, subsistence and accommodation expenses for non-productive staff	350,00 €	4.200,00 €
Advertising and public relations expenses	300,00 €	3.600,00 €

Service	Monthly cost	Annual cost
Office supplies	200,00 €	2.400,00 €
Postal and courier expenses	50,00 €	600,00 €
Professional and business publication subscriptions	150,00 €	1.800,00 €
Financial expenses	250,00 €	3.000,00 €
Depreciation expense of fixed assets	685,00 €	8.220,00 €
TOTAL		107.040,00 €

Table 44: Initial Budget: Annual indirect operating expenses

Equipment / Licenses	Units	Price	Total cost	Annual cost	Type	Term (years)
Azure data center	1	8.000,00 €	8.000,00 €	8.000,00 €	Rental	
Administrative equipment	3	1.000,00 €	3.000,00 €	750,00 €	Depreciation	4
Development equipment	6	1.400,00 €	8.400,00 €	2.100,00 €	Depreciation	4
Laptops	5	1.500,00 €	7.500,00 €	1.875,00 €	Depreciation	4
Development licenses	10	200,00 €	2.000,00 €	2.000,00 €	Rental	
Apple developer program annual fee	1	99,00 €	99,00 €	99,00 €	Rental	
Google Play Console registration	1	25,00 €	25,00 €	25,00 €	Rental	
Collaboration tools	14	120,00 €	1.680,00 €	1.680,00 €	Rental	
TOTAL			22.704,00 €	16.529,00 €		

Table 45: Initial Budget: Equipment and software licenses costs

Personnel	Prod (%)	Hours / year	Productive hours per year (per person)	Total productive hours
General manager	0,00%			0
Project manager	65,00%	2008	1305,2	1305,2
Service coordinator	50,00%	2008	1004,0	1004,0
Systems analyst	80,00%	2008	1606,4	1606,4

Personnel	Prod (%)	Hours / year	Productive hours per year (per person)	Total productive hours
Technology consultant	85,00%	2008	1706,8	1706,8
Software architect	75,00%	2008	1506,0	1506,0
Senior developer	85,00%	2008	1706,8	3413,6
Junior developer	70,00%	2008	1405,6	5622,4
Sales representative	0,00%			0
TOTAL				16.164,4

Table 46: Initial Budget: Annual productive hours by personnel profile

Concept	Value
Indirect costs	335.500,22 €
Productive employees	11
Indirect costs per productive employee	30.500,02 €
Indirect costs including profit	38.125,03 €

Table 47: Initial Budget: Indirect cost allocation per productive employee

Personnel	Direct cost / hour	Hourly rate (with profit)	Productive hours	Revenue	Hourly rate (without profit)
General manager	-	-	0	-	-
Project manager	26,41 €	55,62 €	1305,2	72.600,64 €	49,78 €
Service coordinator	16,37 €	54,35 €	1004,0	54.563,32 €	46,75 €
Systems analyst	23,17 €	46,91 €	1606,4	75.349,66 €	42,16 €
Technology consultant	27,35 €	49,69 €	1706,8	84.807,29 €	45,22 €
Software architect	28,90 €	54,22 €	1506,0	81.651,02 €	49,15 €
Senior developer	25,98 €	48,31 €	3413,6	164.924,99 €	43,85 €
Junior developer	16,85 €	43,98 €	5622,4	247.253,74 €	38,55 €

Personnel	Direct cost / hour	Hourly rate (with profit)	Productive hours	Revenue	Hourly rate (without profit)
Sales representative	-	-	0	-	-
TOTAL			16.164,4	781.150,64 €	

Table 48: Initial Budget: Hourly rates and projected annual revenue by personnel profile

No.	Concept	Value	Amount
1	Total direct costs		246.372,66 €
2	Total indirect costs		335.500,22 €
3	Sum of direct and indirect costs		581.872,88 €
4	Desired profit	25%	145.468,22 €
5	Required billing / revenue		727.341,10 €
6	Possible billing based on productive hours and calculated hourly rates		781.150,64 €
7	Margin between total cost and billing		7,40%

Table 49: Initial Budget: Financial viability and profitability analysis

13.1.2 Initial provider cost breakdown

Category 1: IrBoard development costs breakdown										
I1	**I2**	**I3**	**Description**	**Resource**	**Quantity**	**Units**	**Price**	**Subtotal**	**Subtotal (2)**	**Total**
01			Project management							1.493,46 €
	001		Design project schedule	Project manager	2	Hours	49,78 €	99,56 €	99,56 €	
	002		Generate budget	Project manager	2	Hours	49,78 €	99,56 €	99,56 €	
	003		Periodic project management						1.294,34 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
		01	Periodic project management 1	Project manager	2	Hours	49,78 €	99,56 €		
		02	Periodic project management 2	Project manager	2	Hours	49,78 €	99,56 €		
		03	Periodic project management 3	Project manager	2	Hours	49,78 €	99,56 €		
		04	Periodic project management 4	Project manager	2	Hours	49,78 €	99,56 €		
		05	Periodic project management 5	Project manager	2	Hours	49,78 €	99,56 €		
		06	Periodic project management 6	Project manager	2	Hours	49,78 €	99,56 €		
		07	Periodic project management 7	Project manager	2	Hours	49,78 €	99,56 €		
		08	Periodic project management 8	Project manager	2	Hours	49,78 €	99,56 €		
		09	Periodic project management 9	Project manager	2	Hours	49,78 €	99,56 €		
		10	Periodic project management 10	Project manager	2	Hours	49,78 €	99,56 €		
		11	Periodic project management 11	Project manager	2	Hours	49,78 €	99,56 €		
		12	Periodic project management 12	Project manager	2	Hours	49,78 €	99,56 €		

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
		13	Periodic project management 13	Project manager	2	Hours	49,78 €	99,56 €		
02			Analysis / Software Requirements							2.307,21 €
	001		Determine project scope	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €	
	002		Review Regulatory Standards	Technology consultant	4	Hours	45,22 €	180,88 €	180,88 €	
	003		Identify required Documentation	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	
	004		Determine hand-ins for the project	System Analyst	1	Hours	42,16 €	42,16 €	42,16 €	
	005		Adapt project template to Typst	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	
	006		Analyze existing systems	Technology consultant	6	Hours	45,22 €	271,32 €	271,32 €	
	007		Draft preliminary software requirements	System Analyst	22	Hours	42,16 €	927,50 €	927,50 €	
	008		Model auxiliary diagrams	System Analyst	6	Hours	42,16 €	252,96 €	252,96 €	
	009		Review software requirements	System Analyst	2	Hours	42,16 €	84,32 €	84,32 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	010		Modify requirements with feedback	System Analyst	8	Hours	42,16 €	337,27 €	337,27 €	
03			Design							2.273,37 €
	001		Design architecture	Software architect	10	Hours	49,15 €	491,54 €	491,54 €	
	002		Design brand identity	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	003		Design project management	Senior software engineer	8	Hours	43,85 €	350,77 €	350,77 €	
	004		Design stakeholder management	Senior software engineer	4	Hours	43,85 €	175,39 €	175,39 €	
	005		Design requirement management	Senior software engineer	6	Hours	43,85 €	263,08 €	263,08 €	
	006		Design user management	Senior software engineer	10	Hours	43,85 €	438,47 €	438,47 €	
	007		Design document management	Senior software engineer	10	Hours	43,85 €	438,47 €	438,47 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
04			Set up SonarQube for Quality Assurance	Senior software engineer	3	Hours	43,85 €	131,54 €	131,54 €	131,54 €
05			Development							4.386,90 €
	001		Set up architecture	Software architect	4	Hours	49,15 €	196,62 €	196,62 €	
	002		Set up development environment	Software architect	3	Hours	49,15 €	147,46 €	147,46 €	
	003		Develop code						4.042,82 €	
		01	Develop project management module	Junior software engineer	22	Hours	38,55 €	848,14 €		
		02	Develop stakeholder management module	Junior software engineer	10	Hours	38,55 €	385,52 €		
		03	Develop requirement management module	Senior software engineer	25	Hours	43,85 €	1.096,17 €		
		04	Develop user management module	Junior software engineer	6	Hours	38,55 €	231,31 €		
		05	Develop document management module	Senior software engineer	15	Hours	43,85 €	657,70 €		

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
		06	Develop modifying concurrency system	Senior software engineer	10	Hours	43,85 €	438,47 €		
		07	Develop search and filtering	Junior software engineer	10	Hours	38,55 €	385,52 €		
06			Testing							1.051,49 €
	001		Test project management module	Junior software engineer	4	Hours	38,55 €	154,21 €	154,21 €	
	002		Test stakeholder management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	003		Test requirement management module	Junior software engineer	5	Hours	38,55 €	192,76 €	192,76 €	
	004		Test user management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	005		Test document management module	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	006		Test modifying concurrency system	Junior software engineer	2	Hours	38,55 €	77,10 €	77,10 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	007		Test search and filtering	Junior software engineer	3	Hours	38,55 €	115,66 €	115,66 €	
	008		Usability and accessibility testing	Junior software engineer	2	Hours	38,55 €	77,10 €	77,10 €	
	009		Load testing	Senior software engineer	2	Hours	43,85 €	87,69 €	87,69 €	
07			Documentation							1.382,54 €
	001		Document declaration of originality, abstract and keywords	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	
	002		Document introduction	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	
	003		Document theoretical background	Technology consultant	1	Hours	45,22 €	45,22 €	45,22 €	
	004		Document feasibility study and alternative analysis	Technology consultant	2	Hours	90,44 €	90,44 €	90,44 €	
	005		Document initial project planning and management	Project manager	2	Hours	99,56 €	99,56 €	99,56 €	
	006		Document system analysis	System Analyst	3	Hours	126,48 €	126,48 €	126,48 €	

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	007		Document system design	Software architect	5	Hours	245,77 €	245,77 €	245,77 €	
	008		Document system implementation	Senior software engineer	5	Hours	219,23 €	219,23 €	219,23 €	
	009		Document test plan execution	Senior software engineer	2	Hours	87,69 €	87,69 €	87,69 €	
	010		Document system manuals	Senior software engineer	5	Hours	219,23 €	219,23 €	219,23 €	
	011		Document final project closure	Project manager	2	Hours	99,56 €	99,56 €	99,56 €	
	012		Document conclusions and future work	Project manager	1	Hours	49,78 €	49,78 €	49,78 €	
Total										13.026,52 €

Table 50: Initial Budget: Provider's budget category 1

Category 2: Other										
I1	**I2**	**I3**	**Description**	**Resource**	**Quantity**	**Units**	**Price**	**Subtotal**	**Subtotal (2)**	**Total**
01			Travel and transportation expenses						172,50 €	172,50 €

I1	I2	I3	Description	Resource	Quantity	Units	Price	Subtotal	Subtotal (2)	Total
	001		Travel to client facilities (deployment/support)		150	km	0,35 €	52,50 €		
	002		Meals/daily allowance		6	meals	20,00 €	120,00 €		

Table 51: Initial Budget: Provider's budget category 2

IrBoard costs		
Cat. Num	**Category**	**Total**
01	IrBoard development costs breakdown	13.026,52 €
02	Other	172,50 €
total		13.199,02 €

Table 52: Initial Budget: Provider's budget summary

Profit (25%)	3.299,75 €
Amount to be increased on the first category:	3.472,25 €
Total billable hours:	300,00 hours
Dilution increase per billable hour:	11,58 €

Table 53: Initial Budget: Provider's final notes

13.1.3 Client’s budget

Detailed client budget				
I1	**I2**	**Description**	**Cost**	**Total**
01		Project management		1.840,86 €
	001	Design project schedule	122,72 €	
	002	Generate budget	122,72 €	

I1	I2	Description	Cost	Total
	003	Periodic project management	1.595,42 €	
02		Analysis/Software Requirements		2.932,53 €
	001	Determine project scope	53,74 €	
	002	Review Regulatory Standards	227,20 €	
	003	Identify required Documentation	107,48 €	
	004	Determine hand-ins for the project	53,74 €	
	005	Adapt project template to typst	107,48 €	
	006	Analyze existing systems	340,80 €	
	007	Draft preliminary software requirements	1.182,26 €	
	008	Model auxiliary diagrams	322,44 €	
	009	Review software requirements	107,48 €	
	010	Modify requirements with feedback	429,91 €	
03		Design		2.863,95 €
	001	Design architecture	607,34 €	
	002	Design brand identity	150,40 €	
	003	Design project management	443,41 €	
	004	Design stakeholder management	221,71 €	
	005	Design requirement management	332,56 €	
	006	Design user management	554,27 €	
	007	Design document management	554,27 €	
04		Set up SonarQube for Quality Assurance	166,28 €	166,28 €
05		Development		5.602,80 €
	001	Set up architecture	242,94 €	
	002	Set up development enviroment	182,20 €	

I1	I2	Description	Cost	Total
	003	Develop code	5.177,66 €	
06		Testing		1.364,15 €
	001	Test project management module	200,53 €	
	002	Test stakeholder management module	150,40 €	
	003	Test requirement management module	250,66 €	
	004	Test user management module	150,40 €	
	005	Test document management	150,40 €	
	006	Test modifying concurrency system	100,26 €	
	007	Test search and filtering	150,40 €	
	008	Usability and accessibility testing	100,26 €	
	009	Load testing	110,85 €	
07		Documentation		1.729,94 €
	001	Document declaration of originality, abstract and keywords	61,36 €	
	002	Document introduction	61,36 €	
	003	Document theoretical background	56,80 €	
	004	Document feasibility study and alternative analysis	113,60 €	
	005	Document Initial project planning and management	122,72 €	
	006	Document system analysis	161,22 €	
	007	Document system design	303,67 €	
	008	Document system implementation	277,13 €	
	009	Document test plan execution	110,85 €	
	010	Document system manuals	277,13 €	
	011	Document final project closure	122,72 €	
	012	Document conclusions and future work	61,36 €	

I1	I2	Description	Cost	Total
Total				16.500,52 €

Table 54: Initial Budget: Detailed client budget

Simplified client budget		
Cat. Num	**Category**	**Total**
01	IrBoard development costs breakdown	16.500,52 €
total		16.500,52 €

Table 55: Initial Budget: Simplified client budget

13.2 Licensing of the used open source components

IR-Board relies on a set of third-party open source components, both for its core security architecture and for its observability and supporting infrastructure. Although none of these components are distributed as part of the final product (they run as independent containers integrated through APIs), their licenses determine the conditions under which they can be used, self-hosted, and, where applicable, modified.

Open source licenses can be broadly grouped into two categories relevant to this project. **Permissive licenses** (such as MIT, Apache 2.0, or the PostgreSQL License) allow free use, modification, and redistribution, including in commercial or proprietary contexts, generally only requiring preservation of copyright and license notices. **Copyleft licenses**, and in particular network copyleft licenses such as the GNU Affero General Public License (AGPLv3), additionally require that, if a modified version of the software is made available to users over a network, the corresponding source code must also be made available to those users. This distinction is particularly relevant for the observability stack, as detailed below.

Component	Role in IR-Board	License	Relevant conditions
Ory Kratos	Identity and session management	Apache 2.0	Permissive. Free use, modification, and redistribution; requires preservation of copyright notices and a copy of the license.
Ory Oathkeeper	Authorization gateway	Apache 2.0	Same conditions as above.
Ory Keto	ReBAC authorization server	Apache 2.0	Same conditions as above.
Traefik	API gateway / reverse proxy	MIT	Permissive. Minimal restrictions; requires preservation of copyright notice and license text.
draw.io	Self-hosted diagramming tool embedded in the frontend	Apache 2.0	Permissive; self-hosting and embedding via iframe is explicitly supported by the project's deployment model.
Mailpit	Development email testing server	MIT	Permissive. Used strictly as a development-only component, not intended for production deployment.
Grafana	Observability dashboard	AGPLv3	Network copyleft. Since self-hosting Grafana and exposing it to users over a network constitutes "making the software available" under the AGPL, the source code of any modified version must be made available to those users. As IR-Board uses an unmodi-

Component	Role in IR-Board	License	Relevant conditions
			fied upstream image, no source disclosure obligation is triggered, but this constraint must be respected if the component is ever customized.
Loki	Log aggregation	AGPLv3	Same network copyleft conditions as Grafana, since both were relicensed by Grafana Labs from Apache 2.0 to AGPLv3 in 2021.
Promtail	Log shipping agent (Docker log collection)	AGPLv3	Distributed as part of the Loki repository and therefore subject to the same AGPLv3 conditions.
Prometheus	Metrics collection	Apache 2.0	Permissive; not affected by the Grafana Labs relicensing, as it is a CNCF project independent of Grafana Labs.
PostgreSQL	Relational database	PostgreSQL License	Permissive, similar in spirit to the MIT/BSD family. Free use, modification, and redistribution with minimal conditions.
Spring Boot (and Spring ecosystem)	Backend application framework	Apache 2.0	Permissive.
React	Frontend application framework	MIT	Permissive.
MinIO	Object storage for documents	AGPLv3	Network copyleft, same conditions as Grafana/Loki. Additionally, the upstream open-source project was placed into maintenance mode and effectively archived in December 2025 after earlier (2025) removal of administrative features from the Community Edition's web console; treated here as a frozen, unmaintained-but-licensed dependency rather than an actively supported one. The project pins `minio/minio` and `minio/mc` to `latest`, which should be revisited given this status.

Table 56: License summary of open source components used by IR-Board

13.2.1 Implications for the project

The large majority of the components used by IR-Board (the Ory ecosystem, Traefik, draw.io, Mailpit, Prometheus, PostgreSQL, Spring Boot, and React) are licensed under permissive terms, imposing no meaningful restriction on their use within the project, whether for academic purposes or in a hypothetical commercial deployment for the theoretical client described in the initial planning.

The exception is the logging and dashboarding portion of the observability stack (Grafana, Loki, and Promtail), licensed under AGPLv3 since Grafana Labs' 2021 relicensing. Because these components are deployed unmodified, directly from their official container images, and are not redistributed as part of IR-Board's own codebase, no source-disclosure obligation is triggered by their use within this project. This distinction is, however, an important constraint to document: were the project to fork or modify any of these components (for example, to build a custom plugin or a tailored dashboard distribution) and expose that modified version to other users over the network, the AGPLv3 would require the corresponding source code to be made available to those users. This has been factored into the architecture by keeping observability components clearly isolated as independent, unmodified infrastructure services, consistent with the scope boundaries already established in the Scope section.

The same network-copyleft reasoning applies to MinIO, used for document object storage: it is deployed unmodified and not redistributed, so no source-disclosure obligation is triggered. Independently of licensing, MinIO's open-source edition has been in maintenance mode since December 2025, following the earlier removal of administrative features from its Community Edition console. This is noted here as a dependency-maturity concern rather than a licensing issue; its implications for the project are discussed as a future extension in Conclusions and Future Work.

No component used by the project carries licensing terms that would prevent its use in an academic deliverable, and none requires the payment of licensing fees for self-hosted deployment.